

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №7

з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Розробка моделей на основі технології Петрі-об’єктного моделювання

1. Текст завдання:

1. Розглянути алгоритм Петрі-об'єктного моделювання, реалізований в бібліотеці PetriObjModelPaint (див. github StetsenkoInna). Виконати тестування запропонованого алгоритму на моделі мережі масового обслуговування. 15 балів.
2. За текстом завдання 2 практикуму 5 розробити відповідні Петрі-об'єкти та побудувати Петрі-об'єктну модель системи. Отримати результати імітаційного моделювання. Зробити висновки про функціонування моделі. 30 балів.
3. За текстом завдання 3 практикуму 5 розробити відповідні Петрі-об'єкти та побудувати Петрі-об'єктну модель системи. Отримати результати імітаційного моделювання. Зробити висновки про функціонування моделі. 30 балів.
4. Побудувати математичні рівняння, що описують побудовану за текстом завдання 1 практикуму 5 Петрі-об'єктну модель. 20 балів.
5. Сформулювати переваги та недоліки використання технології Петрі-об'єктного моделювання. 5 балів.

Задача 1 практикуму 5. Конвеєрна система складається з п'ятьох обслуговуючих пристроїв, розташованих уздовж стрічки конвеєра. Деталі надходять на опрацювання на перший пристрій із постійною швидкістю, рівної 4 одиниці за 1 хвилину. Тривалість обслуговування на кожному пристрої розподілена за експоненціальним законом з математичним сподіванням 1 хвилина. Вільного місця перед кожним конвеєром немає, тому пристрій може зняти деталь із конвеєра, тільки якщо знаходиться в стані «вільний». Якщо перший пристрій вільний, то деталь обробляється на ньому. По закінченні обробляння деталь залишає систему. Якщо перший пристрій зайнятий у момент надходження деталі, деталь по конвеєру надходить до другого пристрою. Інтервал проходження деталі між пристроями дорівнює 1 хвилина. Якщо при прямуванні деталі по конвеєру всі пристрої були зайняті, вона повертається до першого пристрою з затримкою 5 хвилин. Метою моделювання є визначення статистичних характеристик часу перебування деталі в системі, завантаження обслуговуючих пристроїв і кількості зайнятих пристроїв.

Задача 2 практикуму 5. Експериментальна роботизована гнучка виробнича система має два верстати із числовим пультом керування, три роботи, пункт прибуття і склад оброблених деталей. Деталі прибувають на пункт прибуття кожні 40 секунд згідно з експоненціальним законом розподілу, захоплюються одним з вільних роботів і переміщуються ним до першого верстата, після чого робот звільняється. Після завершення обробки на першому верстаті деталь захоплюється одним з роботів і переміщується на другий верстат, а після обробки на другому верстаті – одним з роботів переміщується на склад оброблених деталей. Кожний з верстатів може одночасно обробляти до трьох деталей. Час переміщення робота між пунктом прибуття та першим верстатом, першим і другим верстатом, другим верстатом та пунктом зберігання оброблених деталей складає відповідно 6, 7, і 5 секунд незалежно від того, холостий це хід, чи ні. Роботу потрібний час 8±1 секунд на захоплення або вивільнення деталей. Час обробки на першому верстаті розподілений за нормальним законом із середнім значення 60 секунд і стандартним відхиленням 10 секунд. Середній час обробки на другому верстаті дорівнює 100 секунд і має експоненціальний закон розподілу. Метою моделювання є визначення найкращого (з точки зору підвищення пропускної здатності гнучкої виробничої системи) способу закріплення роботів до операцій. Можливі варіанти закріплення: 1) по одному роботу на кожний з трьох шляхів переміщення деталей (пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат, склад); 2) кожний робот може використовуватися на кожному шляху переміщення деталей (при цьому повинен займатися найближчий з роботів).

Задача 3 практикуму 5. На маршруті приміського сполучення працюють два мікроавтобуси (А і В), кожний з яких має n місць. Мікроавтобус А користується більшою популярністю, ніж автобус В, оскільки водій мікроавтобуса А їздить акуратніше і швидше. Тому пасажир, який підійшов до зупинки, сідає в мікроавтобус В тільки у випадку, коли автобуса А немає. Мікроавтобус відправляється на маршрут, якщо всі місця в ньому зайняті. Пасажири підходять до зупинки через 0,5±0,2 хвилин і, якщо немає мікроавтобусів, утворюють чергу. Якщо черга більша, ніж 30 осіб, то пасажир не стає у чергу і йде до іншого маршруту. Припускається, що всі пасажири їдуть до кінця маршруту. На проходження маршруту мікроавтобус А витрачає 20±5 хвилин, а мікроавтобус В – 30±5 хвилин. Після того, як пасажири звільнили автобус (протягом часу 5±1

хвилин), він їде у зворотному напрямку тим же чином. Плата за проїзд складає 20 гривень. Авто підприємство стільки ж втрачає (недоотримує), якщо пасажир, прийшовши на зупинку, не стає у чергу і обирає інший маршрут. Метою моделювання є визначення таких характеристик: час очікування пасажира у черзі; кількість місць n (не більше 25), при якому час очікування в черзі пасажира буде мінімальним; виручку автопідприємства за день від маршруту, якщо мікроавтобуси працюють 10 годин на добу.

2. Код програми:

Моделі написано мовою Java 23 з бібліотекою PetriObjModelPaint версії 2.3.0 (модуль `petri-math` з класами `PetriP`, `PetriT`, `ArcIn`, `ArcOut`, `PetriNet`, `PetriSim`, `PetriObjModel` і модуль `petri-model` з графічною мережею та форматом PNML). Петрі-об'єкти – це об'єкти класу `PetriSim`, модель – об'єкт класу `PetriObjModel`, імітацію виконує метод `PetriObjModel.go()` без змін. Вихідний код, моделі у форматі PNML і результати розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab7`). Файли програми:

- `NetBuilder.java`, `Delay.java` – побудова мережі Петрі Петрі-об'єкта з координатами елементів і переліком дуг, що переносять мітку маркера;
- `PetriObject.java`, `ObjectModel.java` – базовий клас Петрі-об'єкта і Петрі-об'єктна модель: список об'єктів, об'єднання позицій через `linkObjectsCombiningPlaces`, доступ до елементів за іменами;
- `Collector.java`, `Tracker.java` – збирач статистики для `PetriObjModel.go()` (інтерфейс бібліотеки `SimulationStatisticCollector`): середні з розгоном, кількості входів і виходів, трасування, перевірки після кожної події, мітки маркерів;
- `Generator.java`, `Smo.java`, `Fork.java`, `Machine.java`, `Transfer.java`, `AssignedRobot.java`, `RobotPool.java`, `Stop.java`, `Bus.java`, `Device.java` – класи Петрі-об'єктів;
- `QueueNetwork.java`, `Fms.java`, `Minibus.java`, `Conveyor.java` – Петрі-об'єктні моделі завдань 1–4, їх перевірка та експерименти;
- `Export.java` – збереження моделей у PNML з перевіркою зворотного читання, зображення мереж засобами редактора, опис мереж у JSON;
- `Queueing.java`, `Checks.java`, `Stats.java`, `Json.java`, `Output.java`, `Main.java` – формули теорії масового обслуговування, перевірки, довірчі інтервали, запис результатів і запуск;

- `compare_lab5.py` – порівняння мереж і результатів з практикумом 5; `plots.py` – графіки; `run.sh` – завантаження бібліотеки, компіляція і запуск.

Файл NetBuilder.java

```
package lab7;

import java.awt.geom.Point2D;
import java.util.ArrayList;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;

import ua.stetsenkoinna.petriobj.ArcIn;
import ua.stetsenkoinna.petriobj.ArcOut;
import ua.stetsenkoinna.petriobj.ExceptionInvalidTimeDelay;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Побудова мережі Петрі з класів бібліотеки PetriObjModelPaint (PetriP, PetriT, ArcIn, ArcOut)
 * разом з координатами елементів для графічного зображення і переліком дуг, що переносять
 * мітку маркера (момент його появи в моделі) з вхідної позиції у вихідну.
 */
public final class NetBuilder {

    /** Вихідна дуга переходу t у позицію to переносить мітку маркера, взятого з позиції from. */
    public record Carry(PetriT t, PetriP to, PetriP from) {
    }

    private final String name;
    private final List<PetriP> places = new ArrayList<>();
    private final List<PetriT> transitions = new ArrayList<>();
    private final List<ArcIn> arcsIn = new ArrayList<>();
    private final List<ArcOut> arcsOut = new ArrayList<>();
    private final List<Carry> carries = new ArrayList<>();
    private final Map<Object, Point2D> layout = new IdentityHashMap<>();

    public NetBuilder(String name) {
        this.name = name;
        // номери позицій і переходів бібліотека бере з глобальних лічильників, а дуги посилаються
        // на ці номери, тому кожна мережа нумерується з нуля
        PetriP.initNext();
        PetriT.initNext();
        ArcIn.initNext();
        ArcOut.initNext();
    }

    public PetriP place(String placeName, int mark, double x, double y) {
        PetriP p = new PetriP(placeName, mark);
        places.add(p);
        layout.put(p, new Point2D.Double(x, y));
        return p;
    }

    /** Перехід зі сталою затримкою delay і пріоритетом priority. */
    public PetriT transition(String transitionName, double delay, int priority, double x, double y) {
        PetriT t = new PetriT(transitionName, delay);
        t.setPriority(priority);
        transitions.add(t);
        layout.put(t, new Point2D.Double(x, y));
        return t;
    }

    public PetriT transition(String transitionName, double delay, double x, double y) {
        return transition(transitionName, delay, 0, x, y);
    }

    /** Перехід із затримкою delay (стала або випадкова) і пріоритетом priority. */
    public PetriT transition(String transitionName, Delay delay, int priority, double x, double y) {
        PetriT t = transition(transitionName, delay.mean(), priority, x, y);
        if (delay.distribution() != null) {
```

```
            t.setDistribution(delay.distribution(), delay.mean());
            t.setParamDeviation(delay.dev());
        }
        return t;
    }

    public PetriT transition(String transitionName, Delay delay, double x, double y) {
        return transition(transitionName, delay, 0, x, y);
    }

    public void arc(PetriP p, PetriT t, int k) {
        arcsIn.add(new ArcIn(p, t, k));
    }

    public void arc(PetriP p, PetriT t) {
        arc(p, t, 1);
    }

    public void arc(PetriT t, PetriP p, int k) {
        arcsOut.add(new ArcOut(t, p, k));
    }

    public void arc(PetriT t, PetriP p) {
        arc(t, p, 1);
    }

    /** Вихідна дуга t → to, що переносить у позицію to мітки маркерів, взятих переходом з позиції from. */
    public void carry(PetriT t, PetriP to, int k, PetriP from) {
        arc(t, to, k);
        carries.add(new Carry(t, to, from));
    }

    public void carry(PetriT t, PetriP to, PetriP from) {
        carry(t, to, 1, from);
    }

    public PetriNet build() {
        try {
            return new PetriNet(name, new ArrayList<>(places), new ArrayList<>(transitions),
                    new ArrayList<>(arcsIn), new ArrayList<>(arcsOut));
        } catch (ExceptionInvalidTimeDelay e) {
            throw new IllegalStateException(e);
        }
    }

    public String name() {
        return name;
    }

    public List<PetriP> places() {
        return places;
    }

    public List<PetriT> transitions() {
        return transitions;
    }

    public List<ArcIn> arcsIn() {
        return arcsIn;
    }

    public List<ArcOut> arcsOut() {
        return arcsOut;
    }

    public List<Carry> carries() {
        return carries;
    }

    public Point2D position(Object element) {
        return layout.get(element);
    }

    public void move(Object element, double x, double y) {
        layout.put(element, new Point2D.Double(x, y));
    }
}
```

Файл Delay.java

```java
package lab7;

import java.util.Locale;

/** Затримка переходу: стала (const), exp(mean), unif(mean ± dev) або norm(mean, dev). */
public record Delay(String distribution, double mean, double dev) {

    public static Delay fixed(double mean) {
        return new Delay(null, mean, 0);
    }

    public static Delay exp(double mean) {
        return new Delay("exp", mean, 0);
    }

    public static Delay unif(double mean, double dev) {
        return new Delay("unif", mean, dev);
    }

    public static Delay norm(double mean, double dev) {
        return new Delay("norm", mean, dev);
    }

    @Override
    public String toString() {
        String m = String.format(Locale.ROOT, "%g", mean).replaceAll("\\.?0+$", "");
        String d = String.format(Locale.ROOT, "%g", dev).replaceAll("\\.?0+$", "");
        return switch (distribution == null ? "const" : distribution) {
            case "exp" -> "exp(" + m + ")";
            case "unif" -> m + " ± " + d;
            case "norm" -> "N(" + m + "; " + d + ")";
            default -> m;
        };
    }
}
```

Файл PetriObject.java

```java
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт: об'єкт класу, полем якого є мережа Петрі, що описує його динаміку. Підклас будує мережу
 * в конструкторі через поле net; позиції, які об'єкт ділить з іншими об'єктами моделі, адресуються за
 * іменем. Імітатор PetriSim створюється один раз – під час збирання моделі.
 */
public abstract class PetriObject {

    private final String name;
    protected final NetBuilder net;
    private int priority;
    private PetriSim sim;

    protected PetriObject(String name) {
        this.name = name;
        this.net = new NetBuilder(name);
    }

    public String name() {
        return name;
    }

    /** Назва класу Петрі-об'єкта для опису моделі. */
    public abstract String kind();

    public int priority() {
        return priority;
    }

    /** Пріоритет об'єкта: об'єкти з вищим пріоритетом першими виконують вхід маркерів у переходи. */
    public PetriObject priority(int value) {
```

```
        if (sim != null) {
            throw new IllegalStateException("пріоритет задається до збирання моделі");
        }
        priority = value;
        return this;
    }

    public NetBuilder builder() {
        return net;
    }

    public PetriP place(String placeName) {
        return net.places().get(placeIndex(placeName));
    }

    public boolean hasPlace(String placeName) {
        return net.places().stream().anyMatch(p -> p.getName().equals(placeName));
    }

    public int placeIndex(String placeName) {
        for (int i = 0; i < net.places().size(); i++) {
            if (net.places().get(i).getName().equals(placeName)) {
                return i;
            }
        }
        throw new IllegalArgumentException(name + ": немає позиції «" + placeName + "»");
    }

    public PetriT transition(String transitionName) {
        return net.transitions().stream().filter(t -> t.getName().equals(transitionName)).findFirst()
                .orElseThrow(() -> new IllegalArgumentException(name + ": немає переходу «"
                        + transitionName + "»"));
    }

    PetriSim sim() {
        if (sim == null) {
            sim = new PetriSim(net.build());
            sim.setName(name);
            sim.setPriority(priority);
        }
        return sim;
    }
}
```

Файл ObjectModel.java

```
package lab7;

import java.awt.geom.Point2D;
import java.util.ArrayList;
import java.util.IdentityHashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import ua.stetsenkoinna.petriobj.ArcIn;
import ua.stetsenkoinna.petriobj.ArcOut;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriObjModel;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єктна модель: список Петрі-об'єктів і зв'язки між ними. Зв'язок – об'єднання позицій
 * (PetriObjModel.linkObjectsCombiningPlaces): позиція-копія одного об'єкта стає тим самим екземпляром
 * PetriP, що й позиція-джерело іншого. Після збирання модель надає доступ до елементів за іменами і
 * опис мережі, яку утворює об'єднання мереж усіх об'єктів.
 */
public final class ObjectModel {

    /** Зв'язок: позиція copyPlace об'єкта copy збігається з позицією sourcePlace об'єкта source. */
    public record Link(PetriObject copy, String copyPlace, PetriObject source, String sourcePlace) {
    }

    /** Дуга мережі моделі після об'єднання позицій. */
    public record Arc(PetriP place, PetriT transition, int k, boolean info) {
```

```
}

private final String name;
private final List<PetriObject> objects = new ArrayList<>();
private final Map<PetriObject, Point2D> origins = new IdentityHashMap<>();
private final List<Link> links = new ArrayList<>();
private final Map<String, Point2D> drawAt = new LinkedHashMap<>();
private PetriObjModel model;
private final Map<String, PetriP> places = new LinkedHashMap<>();
private final Map<String, PetriT> transitions = new LinkedHashMap<>();
private final Map<PetriT, PetriObject> transitionOwner = new IdentityHashMap<>();
private final Map<PetriP, PetriObject> placeOwner = new IdentityHashMap<>();
private final Map<PetriT, List<Arc>> inputs = new IdentityHashMap<>();
private final Map<PetriT, List<Arc>> outputs = new IdentityHashMap<>();
private final Map<PetriT, Map<PetriP, PetriP>> carries = new IdentityHashMap<>();

public ObjectModel(String name) {
    this.name = name;
}

/** Додає об'єкт; (x, y) - зсув його мережі на загальному зображенні моделі. */
public <O extends PetriObject> O add(O object, double x, double y) {
    objects.add(object);
    origins.put(object, new Point2D.Double(x, y));
    return object;
}

public void link(PetriObject copy, String copyPlace, PetriObject source, String sourcePlace) {
    source.placeIndex(sourcePlace);
    if (copy.place(copyPlace).getMark() != 0) {
        // маркування позиції-копії втрачається, початкове маркування задає джерело
        throw new IllegalArgumentException("позиція-копія з маркерами: " + copyPlace);
    }
    for (Link l : links) {
        if (l.copy() == source && l.copyPlace().equals(sourcePlace)) {
            throw new IllegalArgumentException("ланцюжок об'єднань: " + sourcePlace);
        }
    }
    links.add(new Link(copy, copyPlace, source, sourcePlace));
}

/**
 * Об'єднує з позицією placeName об'єкта source однойменні позиції всіх інших об'єктів моделі
 * (у кожному об'єкті позиція спільного маркування має те саме ім'я, що й у джерела).
 */
public void share(PetriObject source, String... placeNames) {
    for (String placeName : placeNames) {
        boolean linked = false;
        for (PetriObject o : objects) {
            if (o != source && o.hasPlace(placeName)) {
                link(o, placeName, source, placeName);
                linked = true;
            }
        }
        if (!linked) {
            throw new IllegalArgumentException("позицію «" + placeName
                    + "» не використовує жоден інший об'єкт");
        }
    }
}

/** Збирає PetriObjModel: створює імітатори об'єктів і об'єднує позиції. */
public PetriObjModel build() {
    if (model != null) {
        return model;
    }
    ArrayList<PetriSim> sims = new ArrayList<>();
    for (PetriObject o : objects) {
        sims.add(o.sim());
    }
    model = new PetriObjModel(sims);
    model.setIsProtokol(false);
    for (Link l : links) {
        model.linkObjectsCombiningPlaces(objects.indexOf(l.copy()), l.copy().placeIndex(l.copyPlace()),
                objects.indexOf(l.source()), l.source().placeIndex(l.sourcePlace()));
    }
    index();
```

```
        return model;
    }

    private void index() {
        for (PetriObject o : objects) {
            for (PetriP p : o.sim().getNet().getListP()) {
                if (o.builder().places().contains(p)) {
                    put(places, p.getName(), p);
                    placeOwner.put(p, o);
                }
            }
        }
        for (PetriObject o : objects) {
            PetriNet net = o.sim().getNet();
            PetriP[] p = net.getListP();
            PetriT[] t = net.getListT();
            for (PetriT tr : t) {
                put(transitions, tr.getName(), tr);
                transitionOwner.put(tr, o);
                inputs.put(tr, new ArrayList<>());
                outputs.put(tr, new ArrayList<>());
                carries.put(tr, new IdentityHashMap<>());
            }
            for (ArcIn a : net.getArcIn()) {
                inputs.get(t[a.getNumT()]).add(new Arc(p[a.getNumP()], t[a.getNumT()], a.getQuantity(),
                        a.getIsInf()));
            }
            for (ArcOut a : net.getArcOut()) {
                outputs.get(t[a.getNumT()]).add(new Arc(p[a.getNumP()], t[a.getNumT()],
                        a.getQuantity(), false));
            }
            // мітки переносяться між позиціями моделі, тому позиції-копії замінюються джерелами
            for (NetBuilder.Carry c : o.builder().carries()) {
                carries.get(c.t()).put(resolve(o, c.to()), resolve(o, c.from()));
            }
        }
    }

    private static PetriP resolve(PetriObject o, PetriP local) {
        return o.sim().getNet().getListP()[o.builder().places().indexOf(local)];
    }

    private static <T> void put(Map<String, T> map, String key, T value) {
        if (map.put(key, value) != null) {
            throw new IllegalStateException("неоднозначне ім'я елемента моделі: " + key);
        }
    }

    public String name() {
        return name;
    }

    public List<PetriObject> objects() {
        return objects;
    }

    public List<Link> links() {
        return links;
    }

    /** Інше місце елемента на загальному зображенні моделі (абсолютні координати). */
    public void drawAt(String element, double x, double y) {
        drawAt.put(element, new Point2D.Double(x, y));
    }

    public Point2D drawAt(String element, Point2D fallback) {
        return drawAt.getOrDefault(element, fallback);
    }

    public Point2D origin(PetriObject o) {
        return origins.get(o);
    }

    public PetriP place(String placeName) {
        PetriP p = places.get(placeName);
        if (p == null) {
            throw new IllegalArgumentException("немає позиції «" + placeName + "»");
```

```
        }
        return p;
    }

    public PetriT transition(String transitionName) {
        PetriT t = transitions.get(transitionName);
        if (t == null) {
            throw new IllegalArgumentException("немає переходу «" + transitionName + "»");
        }
        return t;
    }

    /** Позиції моделі без копій (кожна спільна позиція – один раз, під іменем джерела). */
    public List<PetriP> places() {
        return new ArrayList<>(places.values());
    }

    public List<PetriT> transitions() {
        return new ArrayList<>(transitions.values());
    }

    public PetriObject owner(PetriT t) {
        return transitionOwner.get(t);
    }

    public PetriObject owner(PetriP p) {
        return placeOwner.get(p);
    }

    public List<Arc> inputs(PetriT t) {
        return inputs.get(t);
    }

    public List<Arc> outputs(PetriT t) {
        return outputs.get(t);
    }

    /** Для переходу t: вихідна позиція → вхідна позиція, з якої переноситься мітка маркера. */
    public Map<PetriP, PetriP> carries(PetriT t) {
        return carries.get(t);
    }

    public PetriObjModel model() {
        return model;
    }
}
```

Файл Collector.java

```
package lab7;

import java.util.ArrayList;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Consumer;
import java.util.function.ToDoubleFunction;

import ua.stetsenkoinna.petriobj.FiringPhase;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriObjModel;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;
import ua.stetsenkoinna.petriobj.SimulationStatisticCollector;

/**
 * Збирач статистики для методу PetriObjModel.go(). Імітацію виконує сама бібліотека, а збирач через
 * інтерфейс SimulationStatisticCollector отримує: виклик flush(t) на початку кожної ітерації циклу
 * go() – стан моделі після події в момент t і входу маркерів у переходи всіх об'єктів, який не
 * змінюється до наступної події; повідомлення про кожен вхід маркерів у перехід і вихід з нього.
 * З цього обчислюються середні за часом значення з урахуванням періоду розгону, кількості входів і
 * виходів, мітки маркерів (Tracker), трасування подій і перевірки після кожної події.
 */
public final class Collector implements SimulationStatisticCollector {
```

```
/** Дія під час виходу маркера з переходу: момент виходу і мітка маркера (NaN, якщо її немає). */
public interface OutAction {
    void accept(double time, double stamp);
}

/** Середнє за часом довільної функції стану моделі та її розподіл за часом. */
public static final class Watch {
    private final ToDoubleFunction<Collector> fn;
    private double value;
    private double area;
    private double span;
    private final Map<Double, Double> histogram = new TreeMap<>();

    private Watch(ToDoubleFunction<Collector> fn) {
        this.fn = fn;
    }

    public double mean() {
        return span > 0 ? area / span : Double.NaN;
    }

    public Map<Double, Double> distribution() {
        Map<Double, Double> d = new TreeMap<>();
        histogram.forEach((v, dt) -> d.put(v, dt / span));
        return d;
    }
}

private final ObjectModel model;
private final double warmup;
private final PetriP[] places;
private final PetriT[] transitions;
private final Map<PetriP, Integer> placeIndex = new IdentityHashMap<>();
private final Map<PetriT, Integer> transitionIndex = new IdentityHashMap<>();
private final int[] mark;
private final int[] buffer;
private final double[] markArea;
private final double[] bufferArea;
private final int[] entered;
private final int[] fired;
private final List<Watch> watches = new ArrayList<>();
private final List<Consumer<Collector>> afterEvent = new ArrayList<>();
private final Map<PetriT, List<OutAction>> outActions = new IdentityHashMap<>();
private final Tracker tracker;
private final List<String> inLog = new ArrayList<>();
private final List<String> outLog = new ArrayList<>();
private double end;
private double last;
private long events = -1;
private int traceEvents;
private Consumer<String> traceOut = s -> { };

public Collector(ObjectModel model, double warmup) {
    this.model = model;
    this.warmup = warmup;
    model.build();
    places = model.places().toArray(PetriP[]::new);
    transitions = model.transitions().toArray(PetriT[]::new);
    for (int i = 0; i < places.length; i++) {
        placeIndex.put(places[i], i);
    }
    for (int i = 0; i < transitions.length; i++) {
        transitionIndex.put(transitions[i], i);
    }
    mark = new int[places.length];
    buffer = new int[transitions.length];
    markArea = new double[places.length];
    bufferArea = new double[transitions.length];
    entered = new int[transitions.length];
    fired = new int[transitions.length];
    tracker = new Tracker(model);
}

/** Імітація моделі методом PetriObjModel.go() бібліотеки до моменту time. */
public Collector run(double time) {
    end = time;
```

```
        PetriObjModel m = model.model();
        m.setStatisticCollector(this);
        m.go(time);
        return this;
    }

    @Override
    public boolean shouldCollect(double currentTime) {
        return true;
    }

    @Override
    public void onTimeStep(double currentTime, PetriNet net, int petriObjId) {
        // стан моделі зчитується один раз за ітерацію – у flush
    }

    @Override
    public void flush(double currentTime) {
        integrate(currentTime);
        for (int i = 0; i < places.length; i++) {
            mark[i] = places[i].getMark();
        }
        for (int i = 0; i < transitions.length; i++) {
            buffer[i] = transitions[i].getBuffer();
        }
        for (Watch w : watches) {
            w.value = w.fn.applyAsDouble(this);
        }
        events++;
        for (Consumer<Collector> hook : afterEvent) {
            hook.accept(this);
        }
        if (events < traceEvents) {
            traceOut.accept(state(currentTime));
        }
        inLog.clear();
        outLog.clear();
    }

    /** Накопичення інтегралів за інтервал [last, t), протягом якого стан моделі не змінювався. */
    private void integrate(double t) {
        double from = Math.max(last, warmup);
        if (t > from) {
            double dt = t - from;
            for (int i = 0; i < places.length; i++) {
                markArea[i] += mark[i] * dt;
            }
            for (int i = 0; i < transitions.length; i++) {
                bufferArea[i] += buffer[i] * dt;
            }
            for (Watch w : watches) {
                w.area += w.value * dt;
                w.span += dt;
                w.histogram.merge(w.value, dt, Double::sum);
            }
        }
        last = Math.max(last, t);
    }

    @Override
    public void onFiringPhase(double time, FiringPhase phase, PetriT t, Iterable<PetriSim> objects) {
        switch (phase) {
            case AFTER_ACT_IN -> {
                tracker.in(t);
                if (time >= warmup) {
                    entered[transitionIndex.get(t)]++;
                }
                if (events + 1 < traceEvents) {
                    inLog.add(label(t));
                }
            }
            case BEFORE_ACT_OUT -> {
                double stamp = tracker.out(t, time);
                for (OutAction a : outActions.getOrDefault(t, List.of())) {
                    a.accept(time, stamp);
                }
            }
```

```
            case AFTER_ACT_OUT -> {
                if (time >= warmup) {
                    fired[transitionIndex.get(t)]++;
                }
                if (events + 1 < traceEvents) {
                    outLog.add(label(t));
                }
            }
            default -> {
            }
        }
    }

    @Override
    public void onSimulationEnd(double simulationEndTime, Iterable<PetriSim> objects) {
        // go() повідомляє момент наступної, вже не виконаної події, тому інтервал завершується в end
        integrate(end);
    }

    @Override
    public void shutdown() {
    }

    private String label(PetriT t) {
        return model.owner(t).name() + ": " + t.getName();
    }

    private String state(double t) {
        StringBuilder sb = new StringBuilder(String.format(Locale.ROOT, "t = %.4f%n", t));
        if (!outLog.isEmpty()) {
            sb.append("  вихід: ").append(grouped(outLog)).append('\n');
        }
        if (!inLog.isEmpty()) {
            sb.append("  вхід:  ").append(grouped(inLog)).append('\n');
        }
        List<String> marks = new ArrayList<>();
        for (PetriP p : places) {
            if (p.getMark() > 0) {
                marks.add(p.getName() + "=" + p.getMark());
            }
        }
        List<String> busy = new ArrayList<>();
        for (PetriT tr : transitions) {
            if (tr.getBuffer() > 0) {
                busy.add(String.format(Locale.ROOT, "%s=%d (%.3f)", tr.getName(), tr.getBuffer(),
                        tr.getMinTime()));
            }
        }
        sb.append("  маркування: ").append(String.join(", ", marks)).append('\n');
        sb.append("  у переходах (найближчий вихід): ").append(String.join(", ", busy)).append('\n');
        return sb.toString();
    }

    private static String grouped(List<String> names) {
        List<String> parts = new ArrayList<>();
        for (int i = 0; i < names.size(); ) {
            int j = i;
            while (j < names.size() && names.get(j).equals(names.get(i))) {
                j++;
            }
            parts.add(j - i == 1 ? names.get(i) : names.get(i) + " ×" + (j - i));
            i = j;
        }
        return String.join(", ", parts);
    }

    public void trace(int count, Consumer<String> out) {
        traceEvents = count;
        traceOut = out;
    }

    public void afterEvent(Consumer<Collector> hook) {
        afterEvent.add(hook);
    }

    public void onOut(String transition, OutAction action) {
        outActions.computeIfAbsent(model.transition(transition), k -> new ArrayList<>()).add(action);
```

```
    }

    public Watch watch(ToDoubleFunction<Collector> fn) {
        Watch w = new Watch(fn);
        watches.add(w);
        return w;
    }

    public int mark(String place) {
        return model.place(place).getMark();
    }

    public int buffer(String transition) {
        return model.transition(transition).getBuffer();
    }

    public double meanMark(String place) {
        return markArea[placeIndex.get(model.place(place))] / period();
    }

    public double meanBuffer(String transition) {
        return bufferArea[transitionIndex.get(model.transition(transition))] / period();
    }

    public int fired(String transition) {
        return fired[transitionIndex.get(model.transition(transition))];
    }

    public int entered(String transition) {
        return entered[transitionIndex.get(model.transition(transition))];
    }

    /**
     * Найбільша розбіжність між середніми, які обчислює бібліотека в PetriObjModel.go()
     * (PetriP.getMean, PetriT.getMean), і власними; має сенс для прогону без розгону.
     */
    public double libraryDifference() {
        double max = 0;
        for (int i = 0; i < places.length; i++) {
            max = Math.max(max, Math.abs(markArea[i] / period() - places[i].getMean()));
        }
        for (int i = 0; i < transitions.length; i++) {
            max = Math.max(max, Math.abs(bufferArea[i] / period() - transitions[i].getMean()));
        }
        return max;
    }

    public boolean tracksConsistently() {
        return tracker.consistent();
    }

    public ObjectModel model() {
        return model;
    }

    /** Поточний модельний час (момент останньої події). */
    public double time() {
        return last;
    }

    /** Тривалість збору статистики (без розгону). */
    public double period() {
        return end - warmup;
    }

    /** Кількість подій – ітерацій циклу go(), у кожній з яких вихід маркерів виконує один об'єкт. */
    public long events() {
        return events;
    }
}
```

Файл Tracker.java

```
package lab7;

import java.util.ArrayDeque;
import java.util.ArrayList;
```

```java
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Мітки маркерів. Маркери бібліотеки неіменовані, тому для вимірювання часу між подіями з однією
 * вимогою (деталлю, пасажиром) мітка – момент появи вимоги в моделі – зберігається поза мережею:
 * для кожної позиції, через яку проходить вимога, – черга міток у порядку надходження маркерів, для
 * кожного переходу – мітки маркерів у його каналах у тому самому порядку, що й моменти виходу в масиві
 * PetriT.getTimeOut(). Під час виходу маркера бібліотека повідомляє номер каналу (PetriT.getNum()),
 * тож мітки переходять разом з маркерами так само, як у мережі практикуму 5.
 */
final class Tracker {

    private final ObjectModel model;
    private final Map<PetriP, ArrayDeque<Double>> queues = new IdentityHashMap<>();
    private final Map<PetriT, List<double[][]>> channels = new IdentityHashMap<>();

    Tracker(ObjectModel model) {
        this.model = model;
        Set<PetriP> tracked = java.util.Collections.newSetFromMap(new IdentityHashMap<>());
        for (PetriT t : model.transitions()) {
            for (Map.Entry<PetriP, PetriP> c : model.carries(t).entrySet()) {
                tracked.add(c.getKey());
                tracked.add(c.getValue());
            }
        }
        for (PetriP p : tracked) {
            ArrayDeque<Double> q = new ArrayDeque<>();
            for (int i = 0; i < p.getMark(); i++) {
                q.add(0.0);
            }
            queues.put(p, q);
        }
        for (PetriT t : model.transitions()) {
            channels.put(t, new ArrayList<>());
        }
    }

    /** Вхід маркера в перехід t: мітки забираються з черг вхідних позицій. */
    void in(PetriT t) {
        List<ObjectModel.Arc> in = model.inputs(t);
        double[][] taken = new double[in.size()][];
        for (int i = 0; i < in.size(); i++) {
            ObjectModel.Arc a = in.get(i);
            ArrayDeque<Double> q = queues.get(a.place());
            if (q != null && !a.info()) {
                taken[i] = new double[a.k()];
                for (int j = 0; j < a.k(); j++) {
                    taken[i][j] = q.removeFirst();
                }
            }
        }
        channels.get(t).add(taken);
    }

    /**
     * Вихід маркера з каналу t.getNum() переходу t у момент time: мітки переносяться у вихідні позиції
     * (або створюються нові з міткою time). Повертає мітку першої відстежуваної вхідної позиції.
     */
    double out(PetriT t, double time) {
        List<double[][]> ch = channels.get(t);
        double[][] taken = ch.remove(t.getNum());
        List<ObjectModel.Arc> in = model.inputs(t);
        Map<PetriP, PetriP> carry = model.carries(t);
        for (ObjectModel.Arc a : model.outputs(t)) {
            ArrayDeque<Double> q = queues.get(a.place());
            if (q == null) {
                continue;
            }
            double[] moved = null;
            PetriP from = carry.get(a.place());
            for (int i = 0; i < in.size() && from != null; i++) {
```

```
                if (in.get(i).place() == from) {
                    moved = taken[i];
                }
            }
            for (int j = 0; j < a.k(); j++) {
                q.add(moved != null && j < moved.length ? moved[j] : time);
            }
        }
        for (double[] stamps : taken) {
            if (stamps != null && stamps.length > 0) {
                return stamps[0];
            }
        }
        return Double.NaN;
    }

    /** Узгодженість з бібліотекою: міток стільки, скільки маркерів у позиціях і каналах переходів. */
    boolean consistent() {
        for (Map.Entry<PetriP, ArrayDeque<Double>> e : queues.entrySet()) {
            if (e.getKey().getMark() != e.getValue().size()) {
                return false;
            }
        }
        for (Map.Entry<PetriT, List<double[][]>> e : channels.entrySet()) {
            if (e.getKey().getBuffer() != e.getValue().size()) {
                return false;
            }
        }
        return true;
    }
}
```

Файл Generator.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Генератор»: перехід із затримкою, що повертає маркер у свою вхідну позицію, створює
 * вимогу в позиції output через інтервали delay. Початкове маркування позиції генератора 0 вимикає
 * генератор (режим необмеженого надходження, коли вимоги заздалегідь лежать у позиції output).
 */
public final class Generator extends PetriObject {

    public Generator(String name, String source, String transition, String output, Delay delay,
                     int sourceMark, int outputMark) {
        super(name);
        PetriP gen = net.place(source, sourceMark, 0, 0);
        PetriP out = net.place(output, outputMark, 260, 0);
        PetriT t = net.transition(transition, delay, 130, 0);
        net.arc(gen, t);
        net.arc(t, gen);
        net.arc(t, out);
    }

    public Generator(String name, String source, String transition, String output, Delay delay) {
        this(name, source, transition, output, delay, 1, 0);
    }

    @Override
    public String kind() {
        return "Генератор";
    }
}
```

Файл Smo.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «СМО»: система масового обслуговування з channels каналами, експоненційним часом
```

```
 * обслуговування і необмеженою чергою. Позиція «Черга X» – вхід об'єкта, «Обслужені X» – вихід.
 */
public final class Smo extends PetriObject {

    public Smo(String label, int channels, double mean) {
        super("СМО " + label);
        PetriP queue = net.place("Черга " + label, 0, 0, 0);
        PetriP free = net.place("Вільні канали " + label, channels, 340, 95);
        PetriP done = net.place("Обслужені " + label, 0, 300, 0);
        PetriT service = net.transition("Обслуговування " + label, Delay.exp(mean), 150, 0);
        net.arc(queue, service);
        net.arc(free, service);
        net.arc(service, free);
        net.carry(service, done, queue);
    }

    @Override
    public String kind() {
        return "СМО";
    }
}
```

Файл Fork.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Розгалуження»: вимога з позиції входу миттєво направляється в одну з вихідних
 * позицій outputs[i] з ймовірністю probabilities[i] (конфлікт переходів з однаковим пріоритетом).
 */
public final class Fork extends PetriObject {

    public Fork(String label, String input, String[] outputs, String[] routes, double[] probabilities) {
        super("Розгалуження " + label);
        PetriP in = net.place(input, 0, 0, 0);
        double top = -150.0 * (outputs.length - 1) / 2;
        for (int i = 0; i < outputs.length; i++) {
            PetriP out = net.place(outputs[i], 0, 260, top + 150 * i);
            PetriT t = net.transition(routes[i], 0, 130, top + 150 * i);
            t.setProbability(Math.round(probabilities[i] * 1e9) / 1e9);
            net.arc(in, t);
            net.carry(t, out, in);
        }
    }

    @Override
    public String kind() {
        return "Розгалуження";
    }
}
```

Файл Machine.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Верстат»: одночасно обробляє до slots деталей. Позиція «Вільні місця» – ресурс
 * верстата, який займають і звільняють роботи (місце резервується при призначенні робота, що везе
 * деталь на верстат, і звільняється, коли наступний робот захоплює оброблену деталь).
 */
public final class Machine extends PetriObject {

    public Machine(String label, int slots, Delay processing) {
        super("Верстат " + label);
        PetriP in = net.place("Деталь на " + label, 0, 0, 0);
        net.place("Вільні місця " + label, slots, 120, -110);
        PetriP done = net.place("Оброблена на " + label, 0, 240, 0);
        PetriT t = net.transition("Обробка на " + label, processing, 120, 0);
        net.arc(in, t);
        net.carry(t, done, in);
```

```
    }

    @Override
    public String kind() {
        return "Верстат";
    }
}
```

Файл Transfer.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Переміщення деталі» на шляху route: робот, що прибув до деталі, захоплює її (8 ± 1 с),
 * переміщується до пункту призначення (move с) і вивільняє деталь (8 ± 1 с). Захоплення звільняє місце
 * на верстаті, з якого забрано деталь (freed, якщо є); після вивільнення деталь потрапляє в позицію
 * result, а робот – у позицію robot (повертається або чекає на новий виклик залежно від закріплення).
 */
public final class Transfer extends PetriObject {

    public Transfer(String route, double move, Delay handle, String freed, String result, String robot) {
        super("Шлях " + route);
        PetriP atPart = net.place("Робот біля деталі (" + route + ")", 0, 0, 0);
        PetriP held = net.place("Деталь захоплена (" + route + ")", 0, 340, 0);
        PetriP delivered = net.place("Деталь доставлена (" + route + ")", 0, 680, 0);
        PetriP out = net.place(result, 0, 1020, 0);
        PetriP back = net.place(robot, 0, 1020, 140);
        PetriT grab = net.transition("Захоплення (" + route + ")", handle, 170, 0);
        PetriT go = net.transition("Переміщення (" + route + ")", Delay.fixed(move), 510, 0);
        PetriT release = net.transition("Вивільнення (" + route + ")", handle, 850, 0);
        net.arc(atPart, grab);
        net.carry(grab, held, atPart);
        if (freed != null) {
            net.arc(grab, net.place(freed, 0, 170, 140));
        }
        net.arc(held, go);
        net.carry(go, delivered, held);
        net.arc(delivered, release);
        net.carry(release, out, delivered);
        net.arc(release, back);
    }

    @Override
    public String kind() {
        return "Переміщення деталі";
    }
}
```

Файл AssignedRobot.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Закріплений робот» (варіант 1): робот k обслуговує лише свій шлях route. Вільний робот
 * призначається на деталь з позиції request, якщо на верстаті призначення є вільне місце (slot, якщо
 * є; місце резервується), а після вивільнення деталі повертається на початок шляху за back с.
 */
public final class AssignedRobot extends PetriObject {

    public AssignedRobot(int k, String route, double back, String request, String slot) {
        super("Робот " + k);
        PetriP req = net.place(request, 0, 0, 0);
        PetriP free = net.place("Робот " + k + " вільний", 1, 0, 330);
        PetriP returning = net.place("Робот " + k + " повертається", 0, 330, 330);
        PetriP atPart = net.place("Робот біля деталі (" + route + ")", 0, 400, 0);
        PetriT assign = net.transition("Призначення робота " + k, 0, 140, 170);
        PetriT ret = net.transition("Повернення робота " + k, back, 170, 330);
        net.arc(req, assign);
        if (slot != null) {
            net.arc(net.place(slot, 0, 0, 170), assign);
```

```
        }
        net.arc(free, assign);
        net.carry(assign, atPart, req);
        net.arc(returning, ret);
        net.arc(ret, free);
    }

    @Override
    public String kind() {
        return "Закріплений робот";
    }
}
```

Файл RobotPool.java

```
package lab7;

import java.util.List;
import java.util.Map;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Парк роботів» (варіант 2): будь-який вільний робот може виконати переміщення на будь-якому
 * шляху. Позиція «Вільний робот: L» містить вільних роботів, що стоять у точці L. Виклик робота з точки L
 * на шлях з початком O триває стільки, скільки холостий хід з L до O, і має пріоритет 100 − відстань,
 * тому деталь отримує найближчий вільний робот. Усі виклики − переходи одного об'єкта, тож конфлікт за
 * робота між шляхами розв'язується за пріоритетами так само, як у мережі практикуму 5.
 */
public final class RobotPool extends PetriObject {

    /** Шлях переміщення: початок, позиція із запитом, позиція місць на верстаті призначення (або null). */
    public record Route(String name, String origin, String request, String slot) {
    }

    /** band − відстань між рядами шляхів на рисунку: виклики кожного шляху стоять у своєму ряду. */
    public RobotPool(Map<String, Double> position, Map<String, Integer> robots, List<Route> routes,
            double band) {
        super("Парк роботів");
        Map<String, PetriP> free = new java.util.LinkedHashMap<>();
        int i = 0;
        for (String loc : position.keySet()) {
            free.put(loc, net.place("Вільний робот: " + loc, robots.getOrDefault(loc, 0), 340 + 200 * (i % 2),
                    band / 2 + band * (i / 2)));
            i++;
        }
        int r = 0;
        for (Route route : routes) {
            double y = band * r++;
            PetriP req = net.place(route.request(), 0, -160, y);
            PetriP slot = route.slot() == null ? null : net.place(route.slot(), 0, -280, y + 150);
            PetriP atPart = net.place("Робот біля деталі (" + route.name() + ")", 0, 600, y);
            int j = 0;
            for (Map.Entry<String, PetriP> e : free.entrySet()) {
                double d = Math.abs(position.get(e.getKey()) - position.get(route.origin()));
                PetriT call = net.transition("Виклик робота: " + e.getKey() + " → " + route.name(), d,
                        (int) (100 - d), 220, y + (j++ - 1.5) * 125);
                net.arc(req, call);
                if (slot != null) {
                    net.arc(slot, call);
                }
                net.arc(e.getValue(), call);
                net.carry(call, atPart, req);
            }
        }
    }

    @Override
    public String kind() {
        return "Парк роботів";
    }
}
```

Файл Stop.java

```java
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Зупинка»: пасажир, якого не забрав мікроавтобус, стає в чергу, якщо в ній є місце
 * (не більше limit осіб), інакше йде до іншого маршруту. Посадку виконують об'єкти «Мікроавтобус», що
 * мають вищий пріоритет, тому до черги потрапляє лише пасажир, який не зміг сісти.
 */
public final class Stop extends PetriObject {

    public Stop(int s, int limit) {
        super("Зупинка З" + s);
        PetriP passenger = net.place("Пасажир на З" + s, 0, 0, 0);
        PetriP queue = net.place("Черга на З" + s, 0, 240, 0);
        PetriP room = net.place("Місця в черзі З" + s, limit, 120, -130);
        PetriP left = net.place("Пішли з З" + s, 0, 240, 150);
        PetriT join = net.transition("Стати в чергу на З" + s, 0, 2, 120, 0);
        PetriT leave = net.transition("Відмова на З" + s, 0, 1, 120, 150);
        net.arc(passenger, join);
        net.arc(room, join);
        net.carry(join, queue, passenger);
        net.arc(passenger, leave);
        net.arc(leave, left);
    }

    @Override
    public String kind() {
        return "Зупинка";
    }
}
```

Файл Bus.java

```java
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Мікроавтобус» з n місцями, що курсує між зупинками З1 і З2. На зупинці автобус приймає
 * пасажирів – спершу з черги (пріоритет queuePriority), потім тих, хто щойно підійшов (directPriority);
 * заповнений автобус вирушає в рейс (trip), на кінцевій зупинці пасажири виходять (unload), і автобус
 * знову стає на посадку. Позиції пасажирів і черг належать об'єктам «Зупинка».
 */
public final class Bus extends PetriObject {

    private static final double WIDTH = 1240;

    public Bus(String b, int n, int start, Delay trip, Delay unload, int queuePriority, int directPriority) {
        super("Мікроавтобус " + b);
        PetriP[] at = new PetriP[3];
        PetriP[] seats = new PetriP[3];
        PetriP[] inside = new PetriP[3];
        PetriP[] arrived = new PetriP[3];
        for (int s = 1; s <= 2; s++) {
            at[s] = net.place(b + " на З" + s, s == start ? 1 : 0, x(s, 290), 200);
            seats[s] = net.place("Вільні місця " + b + " на З" + s, s == start ? n : 0, x(s, 290), -200);
            inside[s] = net.place("Пасажири в " + b + " на З" + s, 0, x(s, 290), 0);
            arrived[s] = net.place(b + " прибув на З" + s, 0, x(s, 560), -100);
            PetriP passenger = net.place("Пасажир на З" + s, 0, x(s, 0), -80);
            PetriP queue = net.place("Черга на З" + s, 0, x(s, 0), 80);
            PetriP room = net.place("Місця в черзі З" + s, 0, x(s, 0), 200);
            String suffix = " в " + b + " на З" + s;
            PetriT fromQueue = net.transition("Посадка з черги" + suffix, 0, queuePriority, x(s, 130), 80);
            PetriT direct = net.transition("Посадка" + suffix, 0, directPriority, x(s, 130), -80);
            net.arc(queue, fromQueue);
            net.arc(seats[s], fromQueue);
            net.carry(fromQueue, inside[s], queue);
            net.arc(fromQueue, room);
            net.arc(passenger, direct);
            net.arc(seats[s], direct);
            net.carry(direct, inside[s], passenger);
        }
        PetriP carried = net.place("Перевезені пасажири", 0, WIDTH / 2, 330);
        for (int s = 1; s <= 2; s++) {
```

```
            int t = 3 - s;
            PetriT ride = net.transition("Рейс " + b + " З" + s + "→З" + t, trip, x(s, 450), 100);
            PetriT off = net.transition("Висадка " + b + " на З" + t, unload, x(t, 450), -100);
            net.arc(at[s], ride);
            net.arc(inside[s], ride, n);
            net.arc(ride, arrived[t]);
            net.carry(ride, carried, n, inside[s]);
            net.arc(arrived[t], off);
            net.arc(off, at[t]);
            net.arc(off, seats[t], n);
        }
    }

    /** Координата елемента: частина мережі для зупинки З2 – дзеркальне відображення частини для З1. */
    private static double x(int stop, double x) {
        return stop == 1 ? x : WIDTH - x;
    }

    @Override
    public String kind() {
        return "Мікроавтобус";
    }
}
```

Файл Device.java

```
package lab7;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Петрі-об'єкт «Пристрій» конвеєрної системи. Деталь біля вільного пристрою знімається з конвеєра й
 * обробляється (пріоритет 1), інакше рухається конвеєром до позиції next за move хв (пріоритет 0).
 */
public final class Device extends PetriObject {

    public Device(int i, Delay service, String moveName, double move, String next) {
        super("Пристрій П" + i);
        PetriP part = net.place("Деталь біля П" + i, 0, 0, 0);
        PetriP free = net.place("П" + i + " вільний", 1, 240, -140);
        PetriP done = net.place("Оброблені деталі", 0, 240, -280);
        PetriP nextPart = net.place(next, 0, 240, 0);
        PetriT serve = net.transition("Обробка П" + i, service, 1, 120, -140);
        PetriT go = net.transition(moveName, move, 0, 120, 0);
        net.arc(part, serve);
        net.arc(free, serve);
        net.arc(serve, free);
        net.carry(serve, done, part);
        net.arc(part, go);
        net.carry(go, nextPart, part);
    }

    @Override
    public String kind() {
        return "Пристрій";
    }
}
```

Файл QueueNetwork.java

```
package lab7;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.ToDoubleFunction;

/**
 * Завдання 1: тестування алгоритму Петрі-об'єктного моделювання на мережі масового обслуговування,
 * для якої відомі аналітичні результати (відкрита мережа Джексона). Модель складено з Петрі-об'єктів
 * трьох класів: Generator, Smo (використано чотири рази) і Fork (двічі).
 */
public final class QueueNetwork {
```

```
static final double ARRIVAL = 1.0;
static final double P_B = 0.7;
static final double P_BACK = 0.2;
static final double S_A = 0.6;
static final double S_B = 1.0;
static final double S_C = 1.5;
static final double TIME = 100000;
static final double WARMUP = 1000;
static final int RUNS = 20;
static final String[] NODES = {"A", "B", "C"};

private QueueNetwork() {
}

/**
 * priorityB1 – пріоритет об'єкта «СМО B1» відносно «СМО B2», що мають спільну чергу; back – ймовірність
 * повернення вимоги з вузла B на вузол A.
 */
static ObjectModel model(int priorityB1, double back) {
    ObjectModel m = new ObjectModel("Мережа масового обслуговування");
    Generator g = m.add(new Generator("Генератор", "Генератор", "Надходження", "Черга A",
            Delay.exp(ARRIVAL)), 0, 520);
    Smo a = m.add(new Smo("A", 1, S_A), 330, 520);
    Fork f1 = m.add(new Fork("1", "Обслужені A", new String[]{"Черга B1", "Черга C"},
            new String[]{"На B", "На C"}, new double[]{P_B, 1 - P_B}), 740, 520);
    Smo b1 = m.add(new Smo("B1", 1, S_B), 1080, 260);
    Smo b2 = m.add(new Smo("B2", 1, S_B), 1080, 520);
    Smo c = m.add(new Smo("C", 1, S_C), 1080, 800);
    Fork f2 = m.add(new Fork("2", "Обслужені B1", new String[]{"Черга A", "Вийшли з мережі"},
            new String[]{"Повтор на A", "Вихід з B"}, new double[]{back, 1 - back}), 1500, 260);
    b1.priority(priorityB1);
    m.drawAt("Повтор на A", 1630, 0);
    m.link(g, "Черга A", a, "Черга A");
    m.link(f1, "Обслужені A", a, "Обслужені A");
    m.link(f1, "Черга B1", b1, "Черга B1");
    m.link(f1, "Черга C", c, "Черга C");
    m.link(b2, "Черга B2", b1, "Черга B1");
    m.link(b2, "Обслужені B2", b1, "Обслужені B1");
    m.link(f2, "Обслужені B1", b1, "Обслужені B1");
    m.link(f2, "Черга A", a, "Черга A");
    return m;
}

/** Інтенсивності потоків вимог у вузлах з рівнянь балансу λ_i = λ_0i + Σ λ_j p_ji. */
static Map<String, Double> rates(double back) {
    double lambda = 1 / ARRIVAL;
    double a = lambda / (1 - P_B * back);
    return Map.of("A", a, "B", P_B * a, "C", (1 - P_B) * a);
}

static int channels(String node) {
    return node.equals("B") ? 2 : 1;
}

static double service(String node) {
    return switch (node) {
        case "A" -> S_A;
        case "B" -> S_B;
        default -> S_C;
    };
}

/** Аналітичні характеристики вузлів мережі Джексона (кожен вузол – система M/M/c). */
static Map<String, Object> analytic() {
    Map<String, Object> out = new LinkedHashMap<>();
    double total = 0;
    for (String n : NODES) {
        double lambda = rates(P_BACK).get(n);
        int c = channels(n);
        double s = service(n);
        double lq = Queueing.queueMMc(c, lambda, s);
        double l = lq + lambda * s;
        total += l;
        out.put(n, Json.obj("rate", lambda, "load", lambda * s / c, "queue", lq, "in_node", l,
                "sojourn", l / lambda));
    }
    out.put("in_network", total);
```

```
        out.put("sojourn", total * ARRIVAL);
        return out;
    }

    /** Показники одного прогону. */
    record Run(Map<String, double[]> node, double inNetwork, double sojournLittle, double sojourn,
               double shareB, double shareBack, double shareB1, long events) {
    }

    static Run run(Collector s) {
        List<Double> sojourns = new ArrayList<>();
        Collector.OutAction exit = (t, born) -> {
            if (born >= WARMUP) {
                sojourns.add(t - born);
            }
        };
        s.onOut("Вихід з B", exit);
        s.onOut("Обслуговування C", exit);
        s.run(TIME);
        Map<String, double[]> node = new LinkedHashMap<>();
        double total = 0;
        for (String n : NODES) {
            String[] servers = n.equals("B") ? new String[]{"B1", "B2"} : new String[]{n};
            double busy = 0;
            double served = 0;
            for (String k : servers) {
                busy += s.meanBuffer("Обслуговування " + k);
                served += s.fired("Обслуговування " + k);
            }
            double queue = s.meanMark("Черга " + servers[0]);
            double rate = served / s.period();
            node.put(n, new double[]{rate, busy / channels(n), queue, queue + busy, (queue + busy) / rate});
            total += queue + busy;
        }
        double arrivals = s.fired("Надходження") / s.period();
        double toB = s.fired("На B");
        double back = s.fired("Повтор на A");
        double b1 = s.fired("Обслуговування B1");
        return new Run(node, total, total / arrivals,
                sojourns.stream().mapToDouble(x -> x).average().orElse(0),
                toB / (toB + s.fired("На C")), back / (back + s.fired("Вихід з B")),
                b1 / (b1 + s.fired("Обслуговування B2")), s.events());
    }

    static Map<String, Object> main(Output out) {
        out.println("ЗАВДАННЯ 1. Мережа масового обслуговування з Петрі-об'єктів (PetriObjModel.go)");
        out.printf("Надходження exp(%.1f); СМО A: 1 канал, exp(%.1f); розгалуження 1: %.1f на B, %.1f на C; "
                + "B: два об'єкти СМО B1, B2 зі спільною чергою, exp(%.1f); C: 1 канал, exp(%.1f); "
                + "після B з ймовірністю %.1f повтор на A%n%n", ARRIVAL, S_A, P_B, 1 - P_B, S_B, S_C, P_BACK);
        out.println("Трасування перших подій (у дужках – найближчий момент виходу маркера з переходу):\n");
        Collector traced = new Collector(model(0, P_BACK), 0);
        traced.trace(10, out::println);
        traced.run(5);

        Collector checked = new Collector(model(0, P_BACK), 0);
        long[] checks = {0};
        checked.afterEvent(s -> {
            Checks.check(s.mark("Генератор") + s.buffer("Надходження") == 1, "генератор");
            for (String k : new String[]{"A", "B1", "B2", "C"}) {
                Checks.check(s.mark("Вільні канали " + k) + s.buffer("Обслуговування " + k) == 1,
                        "канал " + k);
            }
            int inside = s.mark("Черга A") + s.mark("Черга B1") + s.mark("Черга C") + s.mark("Обслужені A")
                    + s.mark("Обслужені B1");
            for (String t : new String[]{"Обслуговування A", "Обслуговування B1", "Обслуговування B2",
                    "Обслуговування C", "На B", "На C", "Повтор на A", "Вихід з B"}) {
                inside += s.buffer(t);
            }
            Checks.check(s.fired("Надходження") == inside + s.mark("Вийшли з мережі") + s.mark("Обслужені C"),
                    "збереження вимог");
            Checks.check(s.tracksConsistently(), "мітки маркерів");
            checks[0]++;
        });
        checked.run(TIME);
        out.printf("Перевірка інваріантів після кожної з %d подій: порушень немає%n", checks[0]);
        out.printf("Середні значення бібліотеки (PetriP.getMean, PetriT.getMean, обчислені в "
                + "PetriObjModel.go) і власні статистики прогону без розгону: найбільша розбіжність %.2e%n",
```

```
                checked.libraryDifference());

        List<Run> runs = new ArrayList<>();
        for (int r = 0; r < RUNS; r++) {
            runs.add(run(new Collector(model(0, P_BACK), WARMUP)));
        }
        Map<String, Object> analytic = analytic();
        String[] metrics = {"інтенсивність потоку", "завантаження каналу", "середня черга",
                "середня кількість у вузлі", "середній час у вузлі"};
        String[] keys = {"rate", "load", "queue", "in_node", "sojourn"};
        out.printf("%nРЕЗУЛЬТАТИ (%d прогонів по %.0f, розгін %.0f, 95 %% довірчий інтервал) і теорія:%n",
                RUNS, TIME, WARMUP);
        Map<String, Object> nodes = new LinkedHashMap<>();
        for (String n : NODES) {
            Map<String, Object> rows = new LinkedHashMap<>();
            @SuppressWarnings("unchecked")
            Map<String, Object> th = (Map<String, Object>) analytic.get(n);
            out.printf("  вузол %s (M/M/%d):%n", n, channels(n));
            for (int i = 0; i < metrics.length; i++) {
                int k = i;
                Stats st = stats(runs, r -> r.node().get(n)[k]);
                double theory = (double) th.get(keys[i]);
                rows.put(keys[i], Json.obj("sim", st, "theory", theory));
                out.printf("    %-28s %-20s теорія %.4f%n", metrics[i], st, theory);
            }
            nodes.put(n, rows);
        }
        Stats inNet = stats(runs, Run::inNetwork);
        Stats little = stats(runs, Run::sojournLittle);
        Stats direct = stats(runs, Run::sojourn);
        out.printf("  середня кількість вимог у мережі     %-20s теорія %.4f%n", inNet,
                analytic.get("in_network"));
        out.printf("  час перебування в мережі (L / λ)     %-20s теорія %.4f%n", little,
                analytic.get("sojourn"));
        out.printf("  час перебування (за мітками вимог)   %-20s%n", direct);
        Stats shareB = stats(runs, Run::shareB);
        Stats shareBack = stats(runs, Run::shareBack);
        Stats shareB1 = stats(runs, Run::shareB1);
        out.printf("Частки маршрутів: на B %s (задано %.1f), повтор на A %s (задано %.1f)%n", shareB, P_B,
                shareBack, P_BACK);
        out.printf("Частка вимог вузла B, обслужених об'єктом СМО B1 (однакові пріоритети B1, B2): %s%n",
                shareB1);

        // перевага одного з двох об'єктів зі спільною чергою задається пріоритетом об'єкта
        List<Object> priority = new ArrayList<>();
        for (double back : new double[]{P_BACK, 0}) {
            List<Run> prior = new ArrayList<>();
            for (int r = 0; r < RUNS; r++) {
                prior.add(run(new Collector(model(1, back), WARMUP)));
            }
            Stats share = stats(prior, Run::shareB1);
            Stats load = stats(prior, r -> r.node().get("B")[1]);
            Stats queue = stats(prior, r -> r.node().get("B")[2]);
            double lambdaB = rates(back).get("B");
            double theory = Queueing.preferredShare(lambdaB, S_B);
            out.printf("Пріоритет об'єкта СМО B1 = 1, повтор на A %.1f (λ_B = %.4f): частка B1 %s "
                    + "(M/M/2 з пуассонівським потоком: %.4f), завантаження B %s (%.4f), черга B %s (%.4f)%n",
                    back, lambdaB, share, theory, load, lambdaB * S_B / 2, queue,
                    Queueing.queueMMc(2, lambdaB, S_B));
            priority.add(Json.obj("back", back, "rate_b", lambdaB, "share_b1", share, "share_theory", theory,
                    "load_b", load, "load_theory", lambdaB * S_B / 2, "queue_b", queue,
                    "queue_theory", Queueing.queueMMc(2, lambdaB, S_B)));
        }
        double events = runs.stream().mapToLong(Run::events).average().orElse(0);
        out.printf("Подій за прогін: %.0f%n", events);

        return Json.obj("nodes", nodes, "analytic", analytic, "in_network", inNet, "sojourn_little", little,
                "sojourn_direct", direct, "share_b", shareB, "share_back", shareBack, "share_b1", shareB1,
                "priority", priority,
                "check_events", checks[0], "library_difference", checked.libraryDifference(), "runs", RUNS,
                "events", events);
    }

    static Stats stats(List<Run> runs, ToDoubleFunction<Run> f) {
        return Stats.of(runs.stream().map(f::applyAsDouble).toList());
    }
}
```

Файл Fms.java

```
package lab7;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Завдання 2: Петрі-об'єктна модель роботизованої гнучкої виробничої системи (задача 2 практикуму 5).
 * Варіант 1 – по одному закріпленому роботу на кожному шляху, варіант 2 – спільний парк роботів.
 */
public final class Fms {

    static final double ARRIVAL = 40;
    static final Delay HANDLE = Delay.unif(8, 1);
    static final int SLOTS = 3;
    static final Map<String, Double> POSITION = new LinkedHashMap<>();
    /** Шлях: назва, початок, кінець, запит, місця на верстаті призначення, звільнені місця, результат. */
    static final String[][] ROUTES = {
        {"вхід-B1", "вхід", "B1", "Деталь на вході", "Вільні місця B1", null, "Деталь на B1"},
        {"B1-B2", "B1", "B2", "Оброблена на B1", "Вільні місця B2", "Вільні місця B1", "Деталь на B2"},
        {"B2-склад", "B2", "склад", "Оброблена на B2", null, "Вільні місця B2", "Склад"},
    };
    static final double BAND = 620;
    static final double TIME = 100000;
    static final double WARMUP = 10000;
    static final double TAIL = 5000;
    static final int RUNS = 20;
    static final int SATURATION = 100000;
    static final double[] INTERVALS = {45, 50, 55, 60, 70, 80};
    static final String[][] METRICS = {
        {"throughput", "пропускна здатність, деталей/год"},
        {"time_in_system", "середній час перебування деталі, с"},
        {"input_queue", "середня черга на вході"},
        {"robots_busy", "середня кількість зайнятих роботів"},
        {"m1_slots", "зайняті місця B1 (з резервом)"},
        {"m2_slots", "зайняті місця B2 (з резервом)"},
        {"m1_processing", "деталей в обробці на B1"},
        {"m2_processing", "деталей в обробці на B2"},
        {"m1_done_waiting", "оброблені на B1 чекають робота"},
        {"m2_done_waiting", "оброблені на B2 чекають робота"},
        {"idle_moves", "роботів у холостому ході"},
    };

    static {
        POSITION.put("вхід", 0.0);
        POSITION.put("B1", 6.0);
        POSITION.put("B2", 13.0);
        POSITION.put("склад", 18.0);
    }

    private Fms() {
    }

    static double distance(String a, String b) {
        return Math.abs(POSITION.get(a) - POSITION.get(b));
    }

    /** arrival – середній інтервал надходження деталей, 0 – необмежене надходження. */
    static ObjectModel model(int variant, double arrival) {
        ObjectModel m = new ObjectModel("Гнучка виробнича система, варіант " + variant);
        boolean saturated = arrival == 0;
        Delay interval = Delay.exp(saturated ? ARRIVAL : arrival);
        Generator gen = m.add(new Generator("Генератор", "Генератор", "Надходження", "Деталь на вході",
                interval, saturated ? 0 : 1, saturated ? SATURATION : 0), 0, 0);
        double shift = variant == 1 ? 0 : 520;
        Machine m1 = m.add(new Machine("B1", SLOTS, Delay.norm(60, 10)), 20, BAND);
        Machine m2 = m.add(new Machine("B2", SLOTS, Delay.exp(100)), 20, 2 * BAND);
        List<Transfer> transfers = new ArrayList<>();
        List<RobotPool.Route> poolRoutes = new ArrayList<>();
        for (int k = 1; k <= ROUTES.length; k++) {
            String[] r = ROUTES[k - 1];
```

```
            double move = distance(r[1], r[2]);
            String robot = variant == 1 ? "Робот " + k + " повертається" : "Вільний робот: " + r[2];
            double y = BAND * (k - 1);
            transfers.add(m.add(new Transfer(r[0], move, HANDLE, r[5], r[6], robot), 660 + shift, y));
            if (variant == 1) {
                AssignedRobot a = m.add(new AssignedRobot(k, r[0], move, r[3], r[4]), 260, y);
                m.share(a, "Робот " + k + " повертається");
            } else {
                poolRoutes.add(new RobotPool.Route(r[0], r[1], r[3], r[4]));
            }
        }
        if (variant == 2) {
            Map<String, Integer> robots = Map.of("вхід", 1, "В1", 1, "В2", 1);
            RobotPool pool = m.add(new RobotPool(POSITION, robots, poolRoutes, BAND), 420, 0);
            // після вивільнення деталі робот стає вільним у кінці шляху
            for (String[] r : ROUTES) {
                m.share(pool, "Вільний робот: " + r[2]);
            }
        }
        m.share(gen, "Деталь на вході");
        m.share(m1, "Деталь на В1", "Оброблена на В1", "Вільні місця В1");
        m.share(m2, "Деталь на В2", "Оброблена на В2", "Вільні місця В2");
        for (int k = 0; k < ROUTES.length; k++) {
            m.share(transfers.get(k), "Робот біля деталі (" + ROUTES[k][0] + ")");
        }
        return m;
    }

    static int robotsBusy(Collector s, int variant) {
        int free = 0;
        if (variant == 1) {
            for (int k = 1; k <= 3; k++) {
                free += s.mark("Робот " + k + " вільний");
            }
        } else {
            for (String loc : POSITION.keySet()) {
                free += s.mark("Вільний робот: " + loc);
            }
        }
        return 3 - free;
    }

    /** Кількість маркерів у переходах, назва яких починається з prefix і закінчується suffix. */
    static int inside(Collector s, String prefix, String suffix) {
        int n = 0;
        for (var t : s.model().transitions()) {
            if (t.getName().startsWith(prefix) && t.getName().endsWith(suffix)) {
                n += t.getBuffer();
            }
        }
        return n;
    }

    static int inside(Collector s, String prefix) {
        return inside(s, prefix, "");
    }

    /** Робот або деталь на шляху route: від прибуття робота до деталі до вивільнення деталі. */
    static int onRoute(Collector s, String route) {
        int n = 0;
        for (String stage : new String[]{"Робот біля деталі", "Деталь захоплена", "Деталь доставлена"}) {
            n += s.mark(stage + " (" + route + ")");
        }
        for (String stage : new String[]{"Захоплення", "Переміщення", "Вивільнення"}) {
            n += s.buffer(stage + " (" + route + ")");
        }
        return n;
    }

    /** Переходи призначення робота на шлях route (у них маркер перебуває під час виклику). */
    static int assigning(Collector s, int variant, int k) {
        return variant == 1 ? s.buffer("Призначення робота " + k) : inside(s, "Виклик", ROUTES[k - 1][0]);
    }

    /** P-інваріанти роботів і місць на верстатах та збереження деталей; перевіряються після кожної події. */
    static void checkInvariants(Collector s, int variant) {
        if (variant == 1) {
```

```
            for (int k = 1; k <= 3; k++) {
                int idle = s.mark("Робот " + k + " вільний") + s.mark("Робот " + k + " повертається")
                        + s.buffer("Повернення робота " + k);
                int robot = idle + assigning(s, 1, k) + onRoute(s, ROUTES[k - 1][0]);
                Checks.check(robot == 1, "робот " + k);
            }
        } else {
            int robots = inside(s, "Виклик");
            for (String loc : POSITION.keySet()) {
                robots += s.mark("Вільний робот: " + loc);
            }
            for (String[] r : ROUTES) {
                robots += onRoute(s, r[0]);
            }
            Checks.check(robots == 3, "роботи");
        }
        for (int i = 1; i <= 2; i++) {
            String m = "B" + i;
            int onMachine = s.mark("Деталь на " + m) + s.buffer("Обробка на " + m)
                    + s.mark("Оброблена на " + m);
            int beforeGrab = s.mark("Робот біля деталі (" + ROUTES[i][0] + ")")
                    + s.buffer("Захоплення (" + ROUTES[i][0] + ")") + assigning(s, variant, i + 1);
            int reserved = assigning(s, variant, i) + onRoute(s, ROUTES[i - 1][0]);
            int free = s.mark("Вільні місця " + m);
            Checks.check(free + reserved + onMachine + beforeGrab == SLOTS, "місця " + m);
        }
        int parts = s.mark("Деталь на вході") + s.mark("Склад");
        for (int i = 1; i <= 2; i++) {
            String m = "B" + i;
            parts += s.mark("Деталь на " + m) + s.buffer("Обробка на " + m) + s.mark("Оброблена на " + m);
        }
        for (int k = 1; k <= 3; k++) {
            parts += onRoute(s, ROUTES[k - 1][0]) + assigning(s, variant, k);
        }
        Checks.check(parts == s.fired("Надходження"), "збереження деталей");
        Checks.check(s.tracksConsistently(), "мітки маркерів");
    }

    static Map<String, Object> run(int variant, double arrival, Collector s) {
        List<Double> times = new ArrayList<>();
        s.onOut("Вивільнення (B2-склад)", (t, born) -> {
            if (born >= WARMUP && born <= TIME - TAIL) {
                times.add(t - born);
            }
        });
        Collector.Watch busy = s.watch(c -> robotsBusy(c, variant));
        s.run(TIME);
        double hour = s.period() / 3600;
        Map<String, Object> r = new LinkedHashMap<>();
        r.put("throughput", s.fired("Вивільнення (B2-склад)") / hour);
        r.put("arrivals", arrival > 0 ? s.fired("Надходження") / hour : null);
        r.put("time_in_system", times.isEmpty() ? null : Minibus.mean(times));
        r.put("input_queue", s.meanMark("Деталь на вході"));
        r.put("robots_busy", busy.mean());
        r.put("m1_slots", SLOTS - s.meanMark("Вільні місця B1"));
        r.put("m2_slots", SLOTS - s.meanMark("Вільні місця B2"));
        r.put("m1_processing", s.meanBuffer("Обробка на B1"));
        r.put("m2_processing", s.meanBuffer("Обробка на B2"));
        r.put("m1_done_waiting", s.meanMark("Оброблена на B1"));
        r.put("m2_done_waiting", s.meanMark("Оброблена на B2"));
        double idle = 0;
        Map<String, Double> calls = new LinkedHashMap<>();
        for (var t : s.model().transitions()) {
            String name = t.getName();
            if (name.startsWith("Виклик") || name.startsWith("Повернення")) {
                idle += s.meanBuffer(name);
            }
            if (name.startsWith("Виклик")) {
                calls.put(name, (double) s.fired(name));
            }
        }
        r.put("idle_moves", idle);
        if (variant == 1) {
            List<Double> load = new ArrayList<>();
            for (int k = 1; k <= 3; k++) {
                load.add(1 - s.meanMark("Робот " + k + " вільний"));
            }
```

```
            r.put("robot_load", load);
        } else {
            r.put("calls", calls);
        }
        return r;
    }

    record Series(Map<String, Stats> stats, List<Map<String, Object>> runs) {
    }

    static Series series(int variant, double arrival, List<String> keys) {
        List<Map<String, Object>> runs = new ArrayList<>();
        for (int i = 0; i < RUNS; i++) {
            runs.add(run(variant, arrival, new Collector(model(variant, arrival), WARMUP)));
        }
        Map<String, Stats> stats = new LinkedHashMap<>();
        for (String k : keys) {
            if (runs.getFirst().get(k) != null) {
                stats.put(k, Stats.of(runs.stream().map(r -> (Double) r.get(k)).toList()));
            }
        }
        return new Series(stats, runs);
    }

    private static final Pattern FAR_CALL = Pattern.compile("Виклик робота: (\\S+) → ([^,\\n]+)");

    static Map<String, Object> main(Output out) {
        out.println("ЗАВДАННЯ 2. Гнучка виробнича система: Петрі-об'єктна модель (PetriObjModel.go)");
        out.printf("Надходження exp(%.0f) с, захоплення/вивільнення %s с, B1 N(60; 10) с, B2 exp(100) с, "
                + "по %d місця на верстаті; ходи вхід-B1 6 с, B1-B2 7 с, B2-склад 5 с%n%n", ARRIVAL, HANDLE,
                SLOTS);
        Map<String, Object> structure = new LinkedHashMap<>();
        for (int variant = 1; variant <= 2; variant++) {
            ObjectModel m = model(variant, ARRIVAL);
            m.build();
            out.printf("Варіант %d: %d Петрі-об'єктів, %d об'єднань позицій; мережа моделі – %d позицій, "
                    + "%d переходів%n", variant, m.objects().size(), m.links().size(), m.places().size(),
                    m.transitions().size());
            structure.put(Integer.toString(variant),
                    Json.obj("objects", m.objects().size(), "links", m.links().size()));
        }
        out.println();
        Map<String, String> traces = new LinkedHashMap<>();
        for (int variant = 1; variant <= 2; variant++) {
            out.printf("Трасування перших подій, варіант %d:%n%n", variant);
            StringBuilder sb = new StringBuilder();
            Collector traced = new Collector(model(variant, ARRIVAL), 0);
            traced.trace(10, s -> {
                out.println(s);
                sb.append(s).append('\n');
            });
            traced.run(300);
            traces.put(Integer.toString(variant), sb.toString());
        }
        // перший виклик робота, що стоїть не на початку шляху (холостий хід)
        List<String> states = new ArrayList<>();
        Collector farTrace = new Collector(model(2, ARRIVAL), 0);
        farTrace.trace(Integer.MAX_VALUE, states::add);
        farTrace.run(3000);
        String far = "";
        for (int i = 1; i < states.size() && far.isEmpty(); i++) {
            String inputs = states.get(i).lines().filter(l -> l.startsWith("  вхід:")).findFirst().orElse("");
            Matcher mt = FAR_CALL.matcher(inputs);
            while (mt.find() && far.isEmpty()) {
                if (!mt.group(2).startsWith(mt.group(1))) {
                    far = states.get(i - 1) + "\n" + states.get(i);
                }
            }
        }
        out.println("Варіант 2, перший виклик робота, що стоїть не на початку шляху (холостий хід):\n");
        out.println(far);

        Map<String, Object> checksDone = new LinkedHashMap<>();
        for (int variant = 1; variant <= 2; variant++) {
            int v = variant;
            Collector checked = new Collector(model(variant, ARRIVAL), 0);
            long[] n = {0};
```

```
    checked.afterEvent(s -> {
        checkInvariants(s, v);
        n[0]++;
    });
    checked.run(TIME);
    out.printf("Перевірка інваріантів, варіант %d: %d подій, порушень немає%n", variant, n[0]);
    checksDone.put(Integer.toString(variant), n[0]);
}

List<String> keys = new ArrayList<>();
for (String[] mtr : METRICS) {
    keys.add(mtr[0]);
}
List<String> baseKeys = new ArrayList<>(keys);
baseKeys.add("arrivals");
Map<String, Map<String, Object>> base = new LinkedHashMap<>();
Map<String, Map<String, Stats>> capacity = new LinkedHashMap<>();
Map<String, Map<String, Object>> sweep = new LinkedHashMap<>();
for (int variant = 1; variant <= 2; variant++) {
    String v = Integer.toString(variant);
    Series s = series(variant, ARRIVAL, baseKeys);
    Map<String, Object> b = new LinkedHashMap<>(s.stats());
    if (variant == 1) {
        List<Stats> load = new ArrayList<>();
        for (int k = 0; k < 3; k++) {
            int kk = k;
            load.add(Stats.of(s.runs().stream().map(r -> ((List<?>) r.get("robot_load")).get(kk))
                    .map(x -> (Double) x).toList()));
        }
        b.put("robot_load", load);
    } else {
        Map<String, Double> calls = new LinkedHashMap<>();
        for (Map<String, Object> r : s.runs()) {
            ((Map<?, ?>) r.get("calls"))
                    .forEach((k, c) -> calls.merge((String) k, (Double) c / RUNS, Double::sum));
        }
        b.put("calls", calls);
    }
    base.put(v, b);
    capacity.put(v, series(variant, 0, keys).stats());
    Map<String, Object> sw = new LinkedHashMap<>();
    for (double a : INTERVALS) {
        sw.put(String.format(Locale.ROOT, "%.1f", a),
                series(variant, a, List.of("throughput", "time_in_system", "input_queue")).stats());
    }
    sweep.put(v, sw);
}

out.printf("%nРЕЗУЛЬТАТИ при надходженні кожні %.0f с (%d прогонів по %.0f с, розгін %.0f с, 95 %% "
        + "довірчий інтервал):%n", ARRIVAL, RUNS, TIME, WARMUP);
out.printf("  %-40s %20s %20s%n", "показник", "варіант 1", "варіант 2");
List<String[]> rows = new ArrayList<>();
rows.add(new String[]{"arrivals", "інтенсивність надходження, деталей/год"});
rows.addAll(List.of(METRICS));
for (String[] mtr : rows) {
    out.printf("  %-40s %20s %20s%n", mtr[1], ((Stats) base.get("1").get(mtr[0])).format(2),
            ((Stats) base.get("2").get(mtr[0])).format(2));
}
@SuppressWarnings("unchecked")
List<Stats> load = (List<Stats>) base.get("1").get("robot_load");
out.println("  завантаження роботів 1–3 (варіант 1): " + String.join(", ",
        load.stream().map(x -> x.format(3)).toList()));

out.println("\nНеобмежене надходження деталей (пропускна здатність системи):");
for (String[] mtr : METRICS) {
    if (capacity.get("1").containsKey(mtr[0])) {
        out.printf("  %-40s %20s %20s%n", mtr[1], capacity.get("1").get(mtr[0]).format(2),
                capacity.get("2").get(mtr[0]).format(2));
    }
}
out.println("\nЗалежність від середнього інтервалу надходження:");
out.printf("  %11s %17s %17s %18s %18s%n", "інтервал, с", "вар. 1, дет./год", "вар. 2, дет./год",
        "вар. 1: час, с", "вар. 2: час, с");
for (double a : INTERVALS) {
    String key = String.format(Locale.ROOT, "%.1f", a);
    @SuppressWarnings("unchecked")
    Map<String, Stats> s1 = (Map<String, Stats>) sweep.get("1").get(key);
```

```
            @SuppressWarnings("unchecked")
            Map<String, Stats> s2 = (Map<String, Stats>) sweep.get("2").get(key);
            out.printf("  %11.0f %17s %17s %18s %18s%n", a, s1.get("throughput").format(2),
                    s2.get("throughput").format(2), s1.get("time_in_system").format(0),
                    s2.get("time_in_system").format(0));
        }

        @SuppressWarnings("unchecked")
        Map<String, Double> calls = (Map<String, Double>) base.get("2").get("calls");
        double total = calls.values().stream().mapToDouble(x -> x).sum();
        out.println("\nВаріант 2, частка викликів робота за місцем, де він стояв:");
        calls.entrySet().stream().sorted((x, y) -> Double.compare(y.getValue(), x.getValue()))
                .filter(e -> e.getValue() > 0)
                .forEach(e -> out.printf("  %-32s %.4f%n", e.getKey(), e.getValue() / total));
        Map<String, Double> idle = new LinkedHashMap<>();
        for (String[] r : ROUTES) {
            double n = 0;
            double d = 0;
            for (String loc : POSITION.keySet()) {
                double c = calls.getOrDefault("Виклик робота: " + loc + " → " + r[0], 0.0);
                n += c;
                d += c * distance(loc, r[1]);
            }
            idle.put(r[0], d / n);
        }
        List<String> idleText = new ArrayList<>();
        idle.forEach((route, d) -> idleText.add(String.format(Locale.ROOT, "%s %.2f", route, d)));
        out.println("  середній холостий хід до деталі, с: " + String.join(", ", idleText));

        double x = capacity.get("1").get("throughput").mean() / 3600;
        out.printf("%nПеревірка (варіант 1, необмежене надходження): за пропускної здатності X = %.2f "
                + "дет./год%n", x * 3600);
        out.printf("  в обробці на B1 X·60 = %.3f (імітація %.3f), на B2 X·100 = %.3f (імітація %.3f)%n",
                x * 60, capacity.get("1").get("m1_processing").mean(), x * 100,
                capacity.get("1").get("m2_processing").mean());
        double cycle = 0;
        for (String[] r : ROUTES) {
            cycle += 2 * 8 + 2 * distance(r[1], r[2]);
        }
        out.printf("  зайняті роботи X·(28 + 30 + 26) = %.3f (імітація %.3f)%n", x * cycle,
                capacity.get("1").get("robots_busy").mean());

        return Json.obj("structure", structure, "traces", traces, "far_call", far, "check_events", checksDone,
                "base", base, "capacity", capacity, "sweep", sweep, "idle", idle, "runs", RUNS,
                "metrics", java.util.Arrays.stream(METRICS).map(List::of).toList());
    }
}
```

Файл Minibus.java

```
package lab7;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Завдання 3: Петрі-об'єктна модель маршруту з двома мікроавтобусами (задача 3 практикуму 5). Об'єкти:
 * два генератори пасажирів, дві зупинки, мікроавтобуси A і B. Пасажир сідає в B, лише коли на зупинці
 * немає A: об'єкт «Мікроавтобус A» має вищий пріоритет, тому першим виконує вхід маркерів у переходи.
 */
public final class Minibus {

    static final Delay ARRIVAL = Delay.unif(0.5, 0.2);
    static final Delay UNLOAD = Delay.unif(5, 1);
    static final Map<String, Delay> TRIP = Map.of("A", Delay.unif(20, 5), "B", Delay.unif(30, 5));
    static final Map<String, Integer> START = Map.of("A", 1, "B", 2);
    static final String[] BUSES = {"A", "B"};
    static final int QUEUE_LIMIT = 30;
    static final int FARE = 20;
    static final double DAY = 600;
    static final int DAYS = 50;
    static final int MAX_SEATS = 25;

    private Minibus() {
    }
```

```
static ObjectModel model(int n) {
    ObjectModel m = new ObjectModel("Маршрут мікроавтобусів, n = " + n);
    Bus a = m.add(new Bus("A", n, START.get("A"), TRIP.get("A"), UNLOAD, 6, 4), 300, 0);
    Bus b = m.add(new Bus("B", n, START.get("B"), TRIP.get("B"), UNLOAD, 5, 3), 300, 720);
    a.priority(2);
    b.priority(1);
    for (int s = 1; s <= 2; s++) {
        Generator g = m.add(new Generator("Генератор З" + s, "Генератор З" + s, "Прихід на З" + s,
                "Пасажир на З" + s, ARRIVAL), s == 1 ? -560 : 1820, 40);
        Stop stop = m.add(new Stop(s, QUEUE_LIMIT), s == 1 ? -300 : 1820, 360);
        m.share(stop, "Пасажир на З" + s, "Черга на З" + s, "Місця в черзі З" + s);
    }
    m.share(a, "Перевезені пасажири");
    return m;
}

/** Показники одного дня (прогону). */
record Day(double meanWait, double waitQueued, double queue, int boarded, int lost, int joined,
           Map<String, Integer> trips) {
    int revenue() {
        return FARE * (boarded - lost);
    }
}

static Day run(int n, double time, Collector s) {
    List<Double> waits = new ArrayList<>();
    List<Double> queued = new ArrayList<>();
    for (int st = 1; st <= 2; st++) {
        for (String b : BUSES) {
            s.onOut("Посадка з черги в " + b + " на З" + st, (t, born) -> {
                waits.add(t - born);
                queued.add(t - born);
            });
            s.onOut("Посадка в " + b + " на З" + st, (t, born) -> waits.add(0.0));
        }
    }
    s.run(time);
    int lost = 0;
    int joined = 0;
    double queue = 0;
    for (int st = 1; st <= 2; st++) {
        lost += s.fired("Відмова на З" + st);
        joined += s.fired("Стати в чергу на З" + st);
        queue += s.meanMark("Черга на З" + st) / 2;
    }
    Map<String, Integer> trips = new LinkedHashMap<>();
    for (String b : BUSES) {
        trips.put(b, s.entered("Рейс " + b + " З1→З2") + s.entered("Рейс " + b + " З2→З1"));
    }
    return new Day(mean(waits), mean(queued), queue, waits.size(), lost, joined, trips);
}

static double mean(List<Double> v) {
    return v.stream().mapToDouble(x -> x).average().orElse(0);
}

/** Інваріанти після кожної події: автобус в одному місці, місця автобуса, черга, баланс пасажирів. */
static void checkInvariants(Collector s, int n) {
    for (String b : BUSES) {
        int where = 0;
        for (int st = 1; st <= 2; st++) {
            String at = " на З" + st;
            where += s.mark(b + at) + s.mark(b + " прибув" + at) + s.buffer("Висадка " + b + at)
                    + s.buffer("Рейс " + b + " З" + st + "→З" + (3 - st));
            int seats = s.mark("Вільні місця " + b + at) + s.mark("Пасажири в " + b + at)
                    + s.buffer("Посадка з черги в " + b + at) + s.buffer("Посадка в " + b + at);
            Checks.check(seats == n * s.mark(b + at), "місця " + b);
        }
        Checks.check(where == 1, "автобус " + b);
    }
    for (int st = 1; st <= 2; st++) {
        int inQueue = s.mark("Черга на З" + st);
        int boarding = 0;
        int boarded = 0;
        for (String b : BUSES) {
            String at = " в " + b + " на З" + st;
```

```
                boarding += s.buffer("Посадка з черги" + at);
                boarded += s.entered("Посадка з черги" + at) + s.entered("Посадка" + at);
                if (inQueue > 0) {
                    Checks.check(s.mark("Вільні місця " + b + " на З" + st) == 0, "черга при вільних місцях");
                }
            }
            int joining = s.buffer("Стати в чергу на З" + st);
            Checks.check(inQueue + s.mark("Місця в черзі З" + st) + joining + boarding == QUEUE_LIMIT,
                    "місця в черзі");
            int passengers = s.mark("Пасажир на З" + st) + joining + inQueue + boarded
                    + s.entered("Відмова на З" + st);
            Checks.check(s.fired("Прихід на З" + st) == passengers, "баланс пасажирів");
        }
        Checks.check(s.tracksConsistently(), "мітки маркерів");
    }

    static Map<String, Object> main(Output out) {
        out.println("ЗАВДАННЯ 3. Маршрут мікроавтобусів: Петрі-об'єктна модель (PetriObjModel.go)");
        out.printf("Пасажири кожні %s хв на кожну з двох кінцевих зупинок, черга до %d, рейс А %s хв, "
                + "В %s хв, висадка %s хв, день %.0f хв%n%n", ARRIVAL, QUEUE_LIMIT, TRIP.get("A"),
                TRIP.get("B"), UNLOAD, DAY);
        ObjectModel shown = model(1);
        shown.build();
        out.printf("%d Петрі-об'єктів, %d об'єднань позицій; мережа моделі – %d позицій, %d переходів%n%n",
                shown.objects().size(), shown.links().size(), shown.places().size(),
                shown.transitions().size());
        out.println("Трасування перших подій (n = 3):\n");
        StringBuilder trace = new StringBuilder();
        Collector traced = new Collector(model(3), 0);
        traced.trace(14, st -> {
            out.println(st);
            trace.append(st).append('\n');
        });
        traced.run(30);

        Collector checked = new Collector(model(10), 0);
        long[] events = {0};
        checked.afterEvent(st -> {
            checkInvariants(st, 10);
            events[0]++;
        });
        checked.run(DAY);
        out.printf("Перевірка інваріантів (n = 10): %d подій, порушень немає%n", events[0]);

        // довгий прогін для перевірки формули Літтла і пропускної здатності
        double longTime = 100 * DAY;
        Day lr = run(MAX_SEATS, longTime, new Collector(model(MAX_SEATS), 0));
        double lam = lr.joined() / longTime / 2;
        out.printf("Довгий прогін %.0f хв, n = %d: середня черга на зупинці %.4f, λ_черги · W_черги = "
                + "%.4f · %.4f = %.4f%n", longTime, MAX_SEATS, lr.queue(), lam, lr.waitQueued(),
                lam * lr.waitQueued());
        double cap = MAX_SEATS * (1.0 / (2 * 20 + 2 * 5) + 1.0 / (2 * 30 + 2 * 5));
        out.printf("Перевезено за хв в одному напрямку %.4f; оцінка при постійній черзі n·(1/50 + 1/70) = "
                + "%.4f; рейсів А %d, В %d (оцінка %.0f і %.0f)%n", lr.boarded() / longTime / 2, cap,
                lr.trips().get("A"), lr.trips().get("B"), longTime / 25, longTime / 35);

        List<Map<String, Object>> sweep = new ArrayList<>();
        out.printf("%nРЕЗУЛЬТАТИ (%d днів для кожного n, 95 %% довірчий інтервал):%n", DAYS);
        out.printf("  %2s %18s %22s %10s %8s %20s%n", "n", "очікування, хв", "серед тих, хто чекав",
                "перевезено", "пішли", "виручка, грн");
        for (int n = 1; n <= MAX_SEATS; n++) {
            List<Day> days = new ArrayList<>();
            for (int d = 0; d < DAYS; d++) {
                days.add(run(n, DAY, new Collector(model(n), 0)));
            }
            Map<String, Object> row = Json.obj("n", n,
                    "wait", Stats.of(days.stream().map(Day::meanWait).toList()),
                    "wait_queued", Stats.of(days.stream().map(Day::waitQueued).toList()),
                    "queue", Stats.of(days.stream().map(Day::queue).toList()),
                    "boarded", Stats.of(days.stream().map(d -> (double) d.boarded()).toList()),
                    "lost", Stats.of(days.stream().map(d -> (double) d.lost()).toList()),
                    "revenue", Stats.of(days.stream().map(d -> (double) d.revenue()).toList()),
                    "income", Stats.of(days.stream().map(d -> (double) FARE * d.boarded()).toList()),
                    "loss", Stats.of(days.stream().map(d -> (double) FARE * d.lost()).toList()),
                    "trips_A", days.stream().mapToInt(d -> d.trips().get("A")).average().orElse(0),
                    "trips_B", days.stream().mapToInt(d -> d.trips().get("B")).average().orElse(0));
            sweep.add(row);
```

```
            out.printf("  %2d %18s %22s %10.1f %8.1f %20s%n", n, ((Stats) row.get("wait")).format(2),
                    ((Stats) row.get("wait_queued")).format(2), ((Stats) row.get("boarded")).mean(),
                    ((Stats) row.get("lost")).mean(), ((Stats) row.get("revenue")).format(0));
        }
        Map<String, Object> best = sweep.stream()
                .min((x, y) -> Double.compare(((Stats) x.get("wait")).mean(), ((Stats) y.get("wait")).mean()))
                .orElseThrow();
        out.printf("%nНайменший середній час очікування при n = %d: %s хв%n", best.get("n"),
                ((Stats) best.get("wait")).format(3));
        out.printf("Виручка за день при n = %d: дохід %.0f, втрати %.0f, виручка %s грн%n", best.get("n"),
                ((Stats) best.get("income")).mean(), ((Stats) best.get("loss")).mean(),
                ((Stats) best.get("revenue")).format(0));

        int objects = shown.objects().size();
        return Json.obj("structure", Json.obj("objects", objects, "links", shown.links().size()),
                "trace", trace.toString(), "sweep", sweep, "best", best.get("n"), "check_events", events[0],
                "little", Json.obj("queue", lr.queue(), "rate", lam, "wait", lr.waitQueued()),
                "capacity", Json.obj("sim", lr.boarded() / longTime / 2, "estimate", cap, "trips", lr.trips(),
                        "time", longTime), "runs", DAYS);
    }
}
```

Файл Conveyor.java

```
package lab7;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.function.ToDoubleFunction;

/**
 * Завдання 4: Петрі-об'єктна модель конвеєрної системи (задача 1 практикуму 5), для якої в звіті
 * записано математичні рівняння. Об'єкти: генератор деталей і п'ять однакових об'єктів «Пристрій».
 */
public final class Conveyor {

    static final int DEVICES = 5;
    static final double ARRIVAL = 0.25;
    static final double SERVICE = 1.0;
    static final double STEP = 1.0;
    static final double BACK = 5.0;
    static final double TIME = 10000;
    static final double WARMUP = 1000;
    static final double TAIL = 300;
    static final int RUNS = 20;

    private Conveyor() {
    }

    /**
     * returnFirst – об'єкт «Пристрій П5» має вищий пріоритет, ніж генератор: якщо деталь, що повертається,
     * і нова деталь з'являються біля П1 одночасно, першою вихід маркера виконує П5 і вільний П1 займає
     * деталь, що повертається (так упорядковано одночасні події в мережі практикуму 5).
     */
    static ObjectModel model(boolean returnFirst) {
        ObjectModel m = new ObjectModel("Конвеєрна система");
        Generator g = m.add(new Generator("Генератор", "Генератор", "Надходження", "Деталь біля П1",
                Delay.fixed(ARRIVAL)), 0, 0);
        List<Device> devices = new ArrayList<>();
        for (int i = 1; i <= DEVICES; i++) {
            boolean last = i == DEVICES;
            String move = last ? "Повернення до П1" : "Рух до П" + (i + 1);
            String next = "Деталь біля П" + (last ? 1 : i + 1);
            devices.add(m.add(new Device(i, Delay.exp(SERVICE), move, last ? BACK : STEP, next),
                    290 + 380 * (i - 1), 0));
        }
        for (Device d : devices) {
            m.share(d, d.name().replace("Пристрій", "Деталь біля"));
        }
        m.share(devices.getFirst(), "Оброблені деталі");
        m.drawAt("Повернення до П1", 290 + 380 * (DEVICES - 1) + 120, 300);
        if (returnFirst) {
            devices.getLast().priority(1);
        }
```

```
        return m;
    }

    static int busyDevices(Collector s) {
        int busy = 0;
        for (int i = 1; i <= DEVICES; i++) {
            busy += 1 - s.mark("П" + i + " вільний");
        }
        return busy;
    }

    /** Деталі в системі: біля пристроїв, на обробці та на конвеєрі (у переходах руху й повернення). */
    static int inSystem(Collector s) {
        int n = 0;
        for (int i = 1; i <= DEVICES; i++) {
            n += s.mark("Деталь біля П" + i);
        }
        for (var t : s.model().transitions()) {
            if (!t.getName().equals("Надходження")) {
                n += t.getBuffer();
            }
        }
        return n;
    }

    record Run(List<Double> times, double meanTime, double[] load, double[] share, double busy,
               Map<Double, Double> busyDist, double throughput, double returnsPerPart, double inSystem) {
    }

    static Run run(Collector s) {
        List<Double> times = new ArrayList<>();
        for (int i = 1; i <= DEVICES; i++) {
            s.onOut("Обробка П" + i, (t, born) -> {
                if (born >= WARMUP && born <= TIME - TAIL) {
                    times.add(t - born);
                }
            });
        }
        Collector.Watch busy = s.watch(Conveyor::busyDevices);
        s.run(TIME);
        double served = 0;
        double[] load = new double[DEVICES];
        double[] share = new double[DEVICES];
        for (int i = 1; i <= DEVICES; i++) {
            served += s.fired("Обробка П" + i);
            load[i - 1] = s.meanBuffer("Обробка П" + i);
        }
        double inSystem = 0;
        for (int i = 1; i <= DEVICES; i++) {
            share[i - 1] = s.fired("Обробка П" + i) / served;
            inSystem += s.meanMark("Деталь біля П" + i);
        }
        for (var t : s.model().transitions()) {
            if (!t.getName().equals("Надходження")) {
                inSystem += s.meanBuffer(t.getName());
            }
        }
        return new Run(times, mean(times), load, share, busy.mean(), busy.distribution(), served / s.period(),
                s.fired("Повернення до П1") / served, inSystem);
    }

    static double mean(List<Double> v) {
        return v.stream().mapToDouble(x -> x).average().orElse(0);
    }

    /** Розподіл часу перебування за деталями всіх прогонів (вибірка впорядкована). */
    static Map<String, Object> distribution(List<Run> runs) {
        List<Double> all = new ArrayList<>();
        runs.forEach(r -> all.addAll(r.times()));
        all.sort(null);
        double m = mean(all);
        double sd = Math.sqrt(all.stream().mapToDouble(x -> (x - m) * (x - m)).sum() / (all.size() - 1));
        return Json.obj("mean", m, "stdev", sd, "min", all.getFirst(), "median", quantile(all, 0.5),
                "p90", quantile(all, 0.9), "p95", quantile(all, 0.95), "max", all.getLast(),
                "count", all.size(), "survival", survival(all));
    }
```

```
static Map<String, Object> main(Output out) {
    out.println("ЗАВДАННЯ 4. Конвеєрна система: Петрі-об'єктна модель (PetriObjModel.go)");
    out.printf("Надходження кожні %.2f хв, обробка exp(%.1f), рух між пристроями %.1f хв, "
            + "повернення %.1f хв, %d пристроїв%n", ARRIVAL, SERVICE, STEP, BACK, DEVICES);
    ObjectModel shown = model(true);
    shown.build();
    out.printf("%d Петрі-об'єктів, %d об'єднань позицій; мережа моделі - %d позицій, %d переходів%n%n",
            shown.objects().size(), shown.links().size(), shown.places().size(),
            shown.transitions().size());
    out.println("Трасування перших подій:\n");
    StringBuilder trace = new StringBuilder();
    Collector traced = new Collector(model(true), 0);
    traced.trace(12, s -> {
        out.println(s);
        trace.append(s).append('\n');
    });
    traced.run(5);

    Collector checked = new Collector(model(true), 0);
    int[] maxInSystem = {0};
    long[] checks = {0};
    checked.afterEvent(s -> {
        for (int i = 1; i <= DEVICES; i++) {
            Checks.check(s.mark("П" + i + " вільний") + s.buffer("Обробка П" + i) == 1, "пристрій П" + i);
        }
        Checks.check(s.mark("Генератор") + s.buffer("Надходження") == 1, "генератор");
        Checks.check(s.fired("Надходження") == s.mark("Оброблені деталі") + inSystem(s), "деталі");
        Checks.check(s.tracksConsistently(), "мітки маркерів");
        maxInSystem[0] = Math.max(maxInSystem[0], inSystem(s));
        checks[0]++;
    });
    checked.run(TIME);
    out.printf("Перевірка інваріантів після кожної з %d подій: порушень немає; найбільше деталей "
            + "у системі %d%n", checks[0], maxInSystem[0]);
    out.printf("Середні значення бібліотеки і власні статистики прогону без розгону: найбільша "
            + "розбіжність %.2e%n", checked.libraryDifference());

    List<Run> runs = new ArrayList<>();
    for (int r = 0; r < RUNS; r++) {
        runs.add(run(new Collector(model(true), WARMUP)));
    }
    Map<String, Stats> table = new LinkedHashMap<>();
    table.put("середній час перебування, хв", stats(runs, Run::meanTime));
    for (int i = 0; i < DEVICES; i++) {
        int k = i;
        table.put("завантаження П" + (i + 1), stats(runs, r -> r.load()[k]));
    }
    table.put("середня кількість зайнятих пристроїв", stats(runs, Run::busy));
    table.put("пропускна здатність, деталей/хв", stats(runs, Run::throughput));
    table.put("повернень до П1 на одну деталь", stats(runs, Run::returnsPerPart));
    table.put("середня кількість деталей у системі", stats(runs, Run::inSystem));
    out.printf("%nРЕЗУЛЬТАТИ (%d прогонів по %.0f хв, розгін %.0f хв, 95 %% довірчий інтервал):%n",
            RUNS, TIME, WARMUP);
    table.forEach((k, v) -> out.printf("  %-40s %s%n", k, v));

    Map<String, Object> times = distribution(runs);
    out.printf("Час перебування деталі (усі прогони): середній %.4f, ст. відхилення %.4f, "
            + "медіана %.4f, 90 %% - %.4f, 95 %% - %.4f, макс. %.4f; деталей %d%n", times.get("mean"),
            times.get("stdev"), times.get("median"), times.get("p90"), times.get("p95"), times.get("max"),
            times.get("count"));
    Map<String, Double> dist = new LinkedHashMap<>();
    List<String> distText = new ArrayList<>();
    for (int k = 0; k <= DEVICES; k++) {
        double key = k;
        double p = runs.stream().mapToDouble(r -> r.busyDist().getOrDefault(key, 0.0)).sum() / RUNS;
        dist.put(Integer.toString(k), p);
        distText.add(String.format(Locale.ROOT, "P(%d) = %.4f", k, p));
    }
    out.println("Розподіл кількості зайнятих пристроїв: " + String.join(", ", distText));
    double[] share = new double[DEVICES];
    List<String> shareText = new ArrayList<>();
    for (int i = 0; i < DEVICES; i++) {
        int k = i;
        share[i] = runs.stream().mapToDouble(r -> r.share()[k]).average().orElse(0);
        shareText.add(String.format(Locale.ROOT, "П%d %.4f", i + 1, share[i]));
    }
    out.println("Частка деталей, оброблених на пристрої: " + String.join(", ", shareText));
```

```
        double lambda = table.get("пропускна здатність, деталей/хв").mean();
        double w = table.get("середній час перебування, хв").mean();
        out.printf("Формула Літтла: λ·W = %.4f · %.4f = %.4f, середня кількість деталей у системі %.4f%n%n",
                lambda, w, lambda * w, table.get("середня кількість деталей у системі").mean());

        // однакові пріоритети об'єктів: одночасні виходи генератора і П5 виконуються у випадковому порядку
        List<Run> equal = new ArrayList<>();
        for (int r = 0; r < RUNS; r++) {
            equal.add(run(new Collector(model(false), WARMUP)));
        }
        Map<String, Object> order = new LinkedHashMap<>();
        for (boolean first : new boolean[]{true, false}) {
            List<Run> rs = first ? runs : equal;
            Map<String, Object> d = distribution(rs);
            Map<String, Object> row = Json.obj("mean_time", stats(rs, Run::meanTime),
                    "returns", stats(rs, Run::returnsPerPart), "load1", stats(rs, r -> r.load()[0]),
                    "in_system", stats(rs, Run::inSystem), "times", d);
            order.put(first ? "return_first" : "equal", row);
            out.printf("%s: час перебування %s (ст. відхилення %.2f, медіана %.2f, 95 %% - %.2f, "
                    + "макс. %.1f), повернень на деталь %s, завантаження П1 %s%n",
                    first ? "Пріоритет П5 вищий (як у ЛР5)" : "Однакові пріоритети об'єктів",
                    row.get("mean_time"), d.get("stdev"), d.get("median"), d.get("p95"), d.get("max"),
                    row.get("returns"), row.get("load1"));
        }

        return Json.obj("table", table, "order", order, "time_stats", times, "busy_distribution", dist,
                "share", share, "check_events", checks[0], "max_in_system", maxInSystem[0],
                "library_difference", checked.libraryDifference(), "trace", trace.toString(), "runs", RUNS,
                "structure", Json.obj("objects", shown.objects().size(), "links", shown.links().size()));
    }

    /** Частка деталей з часом перебування, більшим за t = 0, 2, 4, …, 300 хв (за впорядкованою вибіркою). */
    static List<Double> survival(List<Double> sorted) {
        List<Double> out = new ArrayList<>();
        int i = 0;
        for (int t = 0; t <= 300; t += 2) {
            while (i < sorted.size() && sorted.get(i) <= t) {
                i++;
            }
            out.add((double) (sorted.size() - i) / sorted.size());
        }
        return out;
    }

    static double quantile(List<Double> sorted, double q) {
        double pos = q * (sorted.size() - 1);
        int i = (int) pos;
        double f = pos - i;
        return i + 1 < sorted.size() ? sorted.get(i) * (1 - f) + sorted.get(i + 1) * f : sorted.get(i);
    }

    static Stats stats(List<Run> runs, ToDoubleFunction<Run> f) {
        return Stats.of(runs.stream().map(f::applyAsDouble).toList());
    }
}
```

Файл Export.java

```
package lab7;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics2D;
import java.awt.Point;
import java.awt.Rectangle;
import java.awt.RenderingHints;
import java.awt.geom.Point2D;
import java.awt.geom.RoundRectangle2D;
import java.awt.image.BufferedImage;
import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Collections;
import java.util.IdentityHashMap;
import java.util.LinkedHashMap;
```

```
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.function.Supplier;

import javax.imageio.ImageIO;

import ua.stetsenkoinna.graphnet.GraphArcIn;
import ua.stetsenkoinna.graphnet.GraphArcOut;
import ua.stetsenkoinna.graphnet.GraphPetriNet;
import ua.stetsenkoinna.graphnet.GraphPetriObjModel;
import ua.stetsenkoinna.graphnet.GraphPetriObject;
import ua.stetsenkoinna.graphnet.GraphPetriPlace;
import ua.stetsenkoinna.graphnet.GraphPetriTransition;
import ua.stetsenkoinna.petriobj.ArcIn;
import ua.stetsenkoinna.petriobj.ArcOut;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriObjLink;
import ua.stetsenkoinna.petriobj.PetriObjModel;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;
import ua.stetsenkoinna.pnml.PnmlModelGenerator;
import ua.stetsenkoinna.pnml.PnmlModelParser;
import ua.stetsenkoinna.theme.CanvasColor;
import ua.stetsenkoinna.theme.CanvasPalette;

/**
 * Збереження Петрі-об'єктних моделей засобами PetriObjModelPaint: документ .pnml моделі (одна сторінка
 * на Петрі-об'єкт і перелік об'єднань позицій, як «Export PNML» редактора) з перевіркою зворотного
 * читання, зображення мережі кожного класу Петрі-об'єктів і мережі моделі з рамками об'єктів
 * (GraphPetriNet.paintGraphPetriNet), опис мережі моделі в JSON для порівняння з практикумом 5.
 */
public final class Export {

    private static final int MARGIN = 40;
    private static final int SCALE = 2;
    private static final Color FRAME_FILL = new Color(0xF2F5FA);
    private static final Color FRAME_BORDER = new Color(0x6F86A6);

    private Export() {
    }

    record Frame(Rectangle bounds, String title) {
    }

    static void main(Path nets, Path results) throws Exception {
        Map<String, Supplier<ObjectModel>> models = new LinkedHashMap<>();
        models.put("network", () -> QueueNetwork.model(0, QueueNetwork.P_BACK));
        models.put("fms_variant1", () -> Fms.model(1, Fms.ARRIVAL));
        models.put("fms_variant2", () -> Fms.model(2, Fms.ARRIVAL));
        models.put("minibus", () -> Minibus.model(1));
        models.put("conveyor", () -> Conveyor.model(true));
        Map<String, Integer> classes = new LinkedHashMap<>();
        for (Map.Entry<String, Supplier<ObjectModel>> e : models.entrySet()) {
            exportModel(e.getValue(), nets.resolve(e.getKey()), e.getKey(), results);
            ObjectModel m = e.getValue().get();
            for (PetriObject o : m.objects()) {
                String kind = o.getClass().getSimpleName();
                if (classes.merge(kind, 1, Integer::sum) == 1) {
                    Set<String> linked = interfacePlaces(o, m);
                    GraphPetriNet net = graphNet(o.builder(), o.builder().build(), Map.of(), linked);
                    File png = nets.resolve("objects").resolve(kind + ".png").toFile();
                    ImageIO.write(paint(net, List.of()), "png", png);
                }
            }
        }
        System.out.printf("Зображення мереж %d класів Петрі-об'єктів: %s%n", classes.size(),
                nets.resolve("objects"));
    }

    /** Позиції об'єкта, що беруть участь в об'єднаннях позицій моделі (рисуються сірими, як у редакторі). */
    private static Set<String> interfacePlaces(PetriObject o, ObjectModel m) {
        Set<String> names = new LinkedHashSet<>();
        for (ObjectModel.Link l : m.links()) {
            if (l.copy() == o) {
```

```
                names.add(l.copyPlace());
            }
            if (l.source() == o) {
                names.add(l.sourcePlace());
            }
        }
        return names;
    }

    static void exportModel(Supplier<ObjectModel> factory, Path dir, String file, Path results)
            throws Exception {
        Files.createDirectories(dir);
        Files.createDirectories(dir.getParent().resolve("objects"));
        ObjectModel m = factory.get();
        m.build();
        Output.write(results.resolve("net_" + file + ".json"), Json.write(describe(m)));
        ImageIO.write(paintModel(m), "png", dir.resolve(file + ".png").toFile());

        // документ моделі будується з мереж об'єктів до об'єднання позицій
        ObjectModel fresh = factory.get();
        GraphPetriObjModel graph = new GraphPetriObjModel(fresh.name());
        for (PetriObject o : fresh.objects()) {
            Point2D origin = fresh.origin(o);
            Map<Object, Point2D> at = new IdentityHashMap<>();
            for (PetriP p : o.builder().places()) {
                at.put(p, shift(o.builder().position(p), origin));
            }
            for (PetriT t : o.builder().transitions()) {
                at.put(t, shift(o.builder().position(t), origin));
            }
            GraphPetriNet net = graphNet(o.builder(), o.builder().build(), at, Set.of());
            Rectangle box = bounds(net);
            GraphPetriObject go = new GraphPetriObject(o.name(), net, o.priority(),
                    new Point(box.x - 60, box.y - 60), null);
            go.setSize(box.width + 120, box.height + 120);
            graph.addObject(go);
        }
        for (ObjectModel.Link l : fresh.links()) {
            graph.addLink(PetriObjLink.placeFusion(fresh.objects().indexOf(l.copy()),
                    l.copy().placeIndex(l.copyPlace()), fresh.objects().indexOf(l.source()),
                    l.source().placeIndex(l.sourcePlace())));
        }
        File pnml = dir.resolve(file + ".pnml").toFile();
        new PnmlModelGenerator().generate(graph, pnml);

        PetriObjModel back = new PnmlModelParser().parse(pnml).createPetriObjModel(file);
        boolean same = signature(back.getListObj()).equals(signature(m.model().getListObj()));
        System.out.printf("%s: %d Петрі-об'єктів, %d об'єднань позицій, мережа моделі – %d позицій і %d "
                + "переходів; модель, прочитана з .pnml, %s з вихідною%n", dir.resolve(file),
                m.objects().size(), m.links().size(), m.places().size(), m.transitions().size(),
                same ? "збігається" : "НЕ збігається");
        if (!same) {
            throw new IllegalStateException("PNML " + pnml);
        }
    }

    private static Point2D shift(Point2D p, Point2D origin) {
        return new Point2D.Double(p.getX() + origin.getX(), p.getY() + origin.getY());
    }

    /** Мережа моделі: кожна спільна позиція рисується один раз – там, де її розміщує об'єкт-джерело. */
    static BufferedImage paintModel(ObjectModel m) {
        NetBuilder flat = new NetBuilder(m.name());
        Map<PetriP, PetriP> places = new IdentityHashMap<>();
        Map<PetriT, PetriT> transitions = new IdentityHashMap<>();
        Map<Object, PetriObject> owner = new IdentityHashMap<>();
        for (PetriP p : m.places()) {
            PetriObject o = m.owner(p);
            Point2D at = m.drawAt(p.getName(), shift(o.builder().position(p), m.origin(o)));
            PetriP q = flat.place(p.getName(), p.getMark(), at.getX(), at.getY());
            places.put(p, q);
            owner.put(q, o);
        }
        for (PetriT t : m.transitions()) {
            PetriObject o = m.owner(t);
            Point2D at = m.drawAt(t.getName(), shift(o.builder().position(t), m.origin(o)));
            Delay d = new Delay(t.getDistribution(), t.getParameter(), t.getParamDeviation());
```

```
            PetriT u = flat.transition(t.getName(), d, t.getPriority(), at.getX(), at.getY());
            u.setProbability(t.getProbability());
            transitions.put(t, u);
            owner.put(u, o);
            for (ObjectModel.Arc a : m.inputs(t)) {
                if (a.info()) {
                    flat.arcsIn().add(new ArcIn(places.get(a.place()), u, a.k(), true));
                } else {
                    flat.arc(places.get(a.place()), u, a.k());
                }
            }
            for (ObjectModel.Arc a : m.outputs(t)) {
                flat.arc(u, places.get(a.place()), a.k());
            }
        }
        Set<String> shared = new LinkedHashSet<>();
        for (ObjectModel.Link l : m.links()) {
            shared.add(l.sourcePlace());
        }
        GraphPetriNet graph = graphNet(flat, flat.build(), Map.of(), shared);
        Map<PetriObject, Rectangle> boxes = new LinkedHashMap<>();
        for (GraphPetriPlace g : graph.getGraphPetriPlaceList()) {
            boxes.merge(owner.get(g.getPetriPlace()), g.getBounds(), Rectangle::union);
        }
        for (GraphPetriTransition g : graph.getGraphPetriTransitionList()) {
            boxes.merge(owner.get(g.getPetriTransition()), g.getBounds(), Rectangle::union);
        }
        List<Frame> frames = new ArrayList<>();
        for (PetriObject o : m.objects()) {
            Rectangle r = new Rectangle(boxes.get(o));
            r.grow(46, 0);
            r.y -= 44;
            r.height += 44 + 56;
            frames.add(new Frame(r, o.name() + (o.priority() != 0 ? ", пріоритет " + o.priority() : "")));
        }
        return paint(graph, frames);
    }

    /** Графічна мережа з координатами елементів (так само будує її редактор). */
    static GraphPetriNet graphNet(NetBuilder b, PetriNet net, Map<Object, Point2D> at, Set<String> linked) {
        Map<PetriP, GraphPetriPlace> placeOf = new IdentityHashMap<>();
        Map<PetriT, GraphPetriTransition> transitionOf = new IdentityHashMap<>();
        ArrayList<GraphPetriPlace> places = new ArrayList<>();
        ArrayList<GraphPetriTransition> transitions = new ArrayList<>();
        int id = 0;
        for (PetriP p : net.getListP()) {
            GraphPetriPlace g = new GraphPetriPlace(p, id++);
            g.setNewCoordinates(at.getOrDefault(p, b.position(p)));
            g.setLinkedToAnotherPlace(linked.contains(p.getName()));
            places.add(g);
            placeOf.put(p, g);
        }
        for (PetriT t : net.getListT()) {
            GraphPetriTransition g = new GraphPetriTransition(t, id++);
            g.setNewCoordinates(at.getOrDefault(t, b.position(t)));
            transitions.add(g);
            transitionOf.put(t, g);
        }
        ArrayList<GraphArcIn> arcsIn = new ArrayList<>();
        for (ArcIn arc : net.getArcIn()) {
            GraphArcIn g = new GraphArcIn(arc);
            g.settingNewArc(placeOf.get(net.getListP()[arc.getNumP()]));
            g.finishSettingNewArc(transitionOf.get(net.getListT()[arc.getNumT()]));
            g.setPetriElements();
            g.changeBorder();
            arcsIn.add(g);
        }
        ArrayList<GraphArcOut> arcsOut = new ArrayList<>();
        for (ArcOut arc : net.getArcOut()) {
            GraphArcOut g = new GraphArcOut(arc);
            g.settingNewArc(transitionOf.get(net.getListT()[arc.getNumT()]));
            g.finishSettingNewArc(placeOf.get(net.getListP()[arc.getNumP()]));
            g.setPetriElements();
            g.changeBorder();
            arcsOut.add(g);
            // зустрічні дуги між тими самими елементами редактор розводить паралельно
            for (GraphArcIn in : arcsIn) {
```

```
                if (in.getBeginElement() == g.getEndElement() && in.getEndElement() == g.getBeginElement()) {
                    in.twoArcs(g);
                }
            }
        }
        return new GraphPetriNet(net, places, transitions, arcsIn, arcsOut);
    }

    private static Rectangle bounds(GraphPetriNet graph) {
        Rectangle box = null;
        for (GraphPetriPlace p : graph.getGraphPetriPlaceList()) {
            box = box == null ? new Rectangle(p.getBounds()) : box.union(p.getBounds());
        }
        for (GraphPetriTransition t : graph.getGraphPetriTransitionList()) {
            box = box == null ? new Rectangle(t.getBounds()) : box.union(t.getBounds());
        }
        return box;
    }

    /** Зображення мережі так, як її малює полотно редактора, з рамками Петрі-об'єктів під нею. */
    static BufferedImage paint(GraphPetriNet graph, List<Frame> frames) {
        Rectangle box = bounds(graph);
        box = new Rectangle(box.x - 70, box.y - 40, box.width + 140, box.height + 100);
        for (Frame f : frames) {
            box = box.union(f.bounds());
        }
        int width = box.width + 2 * MARGIN;
        int height = box.height + 2 * MARGIN;
        BufferedImage image = new BufferedImage(width * SCALE, height * SCALE, BufferedImage.TYPE_INT_RGB);
        Graphics2D g = image.createGraphics();
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g.setColor(CanvasPalette.current().get(CanvasColor.CANVAS_BACKGROUND));
        g.fillRect(0, 0, image.getWidth(), image.getHeight());
        g.scale(SCALE, SCALE);
        g.translate(MARGIN - box.x, MARGIN - box.y);
        for (Frame f : frames) {
            Rectangle r = f.bounds();
            RoundRectangle2D shape = new RoundRectangle2D.Double(r.x, r.y, r.width, r.height, 18, 18);
            g.setColor(FRAME_FILL);
            g.fill(shape);
            g.setColor(FRAME_BORDER);
            g.setStroke(new BasicStroke(1.4f));
            g.draw(shape);
            g.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 13));
            g.drawString(f.title(), r.x + 10, r.y + 17);
        }
        g.setStroke(new BasicStroke(1f));
        g.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));
        graph.paintGraphPetriNet(g, g);
        g.dispose();
        return image;
    }

    /** Опис мережі моделі (після об'єднання позицій), Петрі-об'єктів і зв'язків. */
    static Map<String, Object> describe(ObjectModel m) {
        List<Object> places = new ArrayList<>();
        for (PetriP p : m.places()) {
            places.add(Json.obj("name", p.getName(), "mark", p.getMark(), "object", m.owner(p).name()));
        }
        List<Object> transitions = new ArrayList<>();
        for (PetriT t : m.transitions()) {
            List<Object> in = new ArrayList<>();
            List<Object> info = new ArrayList<>();
            List<Object> out = new ArrayList<>();
            for (ObjectModel.Arc a : m.inputs(t)) {
                (a.info() ? info : in).add(Json.obj("place", a.place().getName(), "n", a.k()));
            }
            for (ObjectModel.Arc a : m.outputs(t)) {
                Map<String, Object> arc = Json.obj("place", a.place().getName(), "n", a.k());
                PetriP from = m.carries(t).get(a.place());
                if (from != null) {
                    arc.put("carry", from.getName());
                }
                out.add(arc);
            }
            transitions.add(Json.obj("name", t.getName(), "object", m.owner(t).name(),
```

```
                    "distribution", t.getDistribution() == null ? "const" : t.getDistribution(),
                    "mean", t.getParameter(), "deviation", t.getParamDeviation(), "priority", t.getPriority(),
                    "probability", t.getProbability(), "inputs", in, "info_inputs", info, "outputs", out));
        }
        List<Object> objects = new ArrayList<>();
        for (PetriObject o : m.objects()) {
            objects.add(Json.obj("name", o.name(), "class", o.kind(), "priority", o.priority(),
                    "places", o.builder().places().stream().map(PetriP::getName).toList(),
                    "transitions", o.builder().transitions().stream().map(PetriT::getName).toList()));
        }
        List<Object> links = new ArrayList<>();
        for (ObjectModel.Link l : m.links()) {
            links.add(Json.obj("copy", l.copy().name(), "copy_place", l.copyPlace(),
                    "source", l.source().name(), "source_place", l.sourcePlace()));
        }
        return Json.obj("name", m.name(), "objects", objects, "links", links, "places", places,
                "transitions", transitions);
    }

    /** Склад мережі моделі без урахування порядку: позиції, переходи, дуги (позиції – екземпляри PetriP). */
    static List<String> signature(List<PetriSim> sims) {
        Set<PetriP> seen = Collections.newSetFromMap(new IdentityHashMap<>());
        List<String> items = new ArrayList<>();
        for (PetriSim s : sims) {
            PetriP[] p = s.getNet().getListP();
            PetriT[] t = s.getNet().getListT();
            for (PetriP place : p) {
                if (seen.add(place)) {
                    items.add("P " + place.getName() + " " + place.getMark());
                }
            }
            for (PetriT tr : t) {
                items.add("T " + tr.getName() + " " + tr.getParameter() + " " + tr.getDistribution() + " "
                        + tr.getParamDeviation() + " " + tr.getPriority() + " " + tr.getProbability());
            }
            for (ArcIn a : s.getNet().getArcIn()) {
                items.add("I " + p[a.getNumP()].getName() + " " + t[a.getNumT()].getName() + " "
                        + a.getQuantity() + " " + a.getIsInf());
            }
            for (ArcOut a : s.getNet().getArcOut()) {
                items.add("O " + t[a.getNumT()].getName() + " " + p[a.getNumP()].getName() + " "
                        + a.getQuantity());
            }
        }
        items.sort(null);
        return items;
    }
}
```

Файл Queueing.java

```
package lab7;

/** Формули теорії масового обслуговування для перевірки моделей. */
final class Queueing {

    private Queueing() {
    }

    /** Ймовірність очікування в системі M/M/c (формула Ерланга C), a = λ·s. */
    static double erlangC(int c, double a) {
        double term = 1;
        double sum = 1;
        for (int k = 1; k < c; k++) {
            term *= a / k;
            sum += term;
        }
        double last = term * a / c / (1 - a / c);
        return last / (sum + last);
    }

    /** Середня довжина черги M/M/c. */
    static double queueMMc(int c, double lambda, double s) {
        double a = lambda * s;
        return erlangC(c, a) * a / (c - a);
    }
```

```
    /**
     * Частка вимог, яку обслуговує перший з двох однакових каналів M/M/2, якщо вимога, що застала обидва
     * канали вільними, завжди займає перший: з рівняння балансу стану «зайнятий лише другий канал»
     * μ·p_2 = (λ + μ)·p_{1,2} імовірність зайнятості першого каналу p_1 − p_{1,2} + P(N ≥ 2).
     */
    static double preferredShare(double lambda, double s) {
        double a = lambda * s;
        double rho = a / 2;
        double p0 = 1 / (1 + a + a * a / 2 / (1 - rho));
        double p1 = a * p0;
        double p2 = a * a / 2 * p0;
        double onlySecond = p2 / (1 + a);
        double busyFirst = p1 - onlySecond + p2 / (1 - rho);
        return busyFirst / a;
    }
}
```

Файл Checks.java

```
package lab7;

/** Перевірка інваріантів моделей. */
final class Checks {

    private Checks() {
    }

    static void check(boolean ok, String what) {
        if (!ok) {
            throw new IllegalStateException("порушено інваріант: " + what);
        }
    }
}
```

Файл Stats.java

```
package lab7;

import java.util.Collection;
import java.util.Locale;

/** Середнє, стандартне відхилення та півширина 95 % довірчого інтервалу за t-розподілом Стьюдента. */
public record Stats(double mean, double ci, int n) {

    public static Stats of(Collection<Double> values) {
        int n = values.size();
        double mean = values.stream().mapToDouble(Double::doubleValue).sum() / n;
        if (n < 2) {
            return new Stats(mean, 0.0, n);
        }
        double ss = values.stream().mapToDouble(v -> (v - mean) * (v - mean)).sum();
        return new Stats(mean, tQuantile975(n - 1) * Math.sqrt(ss / (n - 1) / n), n);
    }

    /** Квантиль рівня 0,975 t-розподілу (розклад Корніша – Фішера, похибка < 1e-4 при ν ≥ 5). */
    static double tQuantile975(int nu) {
        double z = 1.959963984540054;
        double z3 = z * z * z;
        double z5 = z3 * z * z;
        double z7 = z5 * z * z;
        return z + (z3 + z) / (4.0 * nu) + (5 * z5 + 16 * z3 + 3 * z) / (96.0 * nu * nu)
                + (3 * z7 + 19 * z5 + 17 * z3 - 15 * z) / (384.0 * nu * nu * nu);
    }

    public String format(int digits) {
        return String.format(Locale.ROOT, "%." + digits + "f ± %." + digits + "f", mean, ci);
    }

    @Override
    public String toString() {
        return format(4);
    }
}
```

Файл Json.java

```java
package lab7;

import java.util.Collection;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.Locale;
import java.util.Map;

/** Запис результатів у JSON (Map, Collection, Number, String, Boolean, Stats). */
public final class Json {

    private Json() {
    }

    /** Впорядкований словник з пар «ключ, значення». */
    public static Map<String, Object> obj(Object... keyValues) {
        Map<String, Object> m = new LinkedHashMap<>();
        for (int i = 0; i < keyValues.length; i += 2) {
            m.put((String) keyValues[i], keyValues[i + 1]);
        }
        return m;
    }

    public static String write(Object value) {
        StringBuilder sb = new StringBuilder();
        append(sb, value, 0);
        return sb.append('\n').toString();
    }

    private static void append(StringBuilder sb, Object v, int indent) {
        switch (v) {
            case null -> sb.append("null");
            case Stats s -> append(sb, obj("mean", s.mean(), "ci", s.ci()), indent);
            case Map<?, ?> m -> container(sb, '{', '}', m.entrySet().iterator(), indent, true);
            case Collection<?> c -> container(sb, '[', ']', c.iterator(), indent, false);
            case double[] a -> append(sb, java.util.Arrays.stream(a).boxed().toList(), indent);
            case Double d -> sb.append(Double.isFinite(d) ? String.format(Locale.ROOT, "%.10g", d) : "null");
            case Number n -> sb.append(n);
            case Boolean b -> sb.append(b);
            default -> sb.append('"').append(v.toString().replace("\\", "\\\\").replace("\"", "\\\"")
                    .replace("\n", "\\n")).append('"');
        }
    }

    private static void container(StringBuilder sb, char open, char close, Iterator<?> it, int indent,
                                  boolean map) {
        sb.append(open);
        boolean first = true;
        while (it.hasNext()) {
            Object item = it.next();
            sb.append(first ? "\n" : ",\n").append(" ".repeat(indent + 1));
            first = false;
            if (map) {
                Map.Entry<?, ?> e = (Map.Entry<?, ?>) item;
                append(sb, String.valueOf(e.getKey()), indent + 1);
                sb.append(": ");
                append(sb, e.getValue(), indent + 1);
            } else {
                append(sb, item, indent + 1);
            }
        }
        if (!first) {
            sb.append('\n').append(" ".repeat(indent));
        }
        sb.append(close);
    }
}
```

Файл Output.java

```java
package lab7;

import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;
```

```java
/** Вивід програми: одночасно на консоль і у файл результатів. */
public final class Output {

    private final StringBuilder text = new StringBuilder();

    public void println(String line) {
        System.out.println(line);
        text.append(line).append('\n');
    }

    public void println() {
        println("");
    }

    public void printf(String format, Object... args) {
        String s = String.format(Locale.ROOT, format, args);
        System.out.print(s);
        text.append(s);
    }

    public void save(Path file) {
        write(file, text.toString());
    }

    public static void write(Path file, String content) {
        try {
            Files.createDirectories(file.getParent());
            Files.writeString(file, content);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}
```

Файл Main.java

```java
package lab7;

import java.nio.file.Path;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/** Запуск моделей: java lab7.Main [network] [fms] [minibus] [conveyor] [export]. */
public final class Main {

    private static final Path RESULTS = Path.of("results");

    private Main() {
    }

    public static void main(String[] args) throws Exception {
        java.util.logging.LogManager.getLogManager().reset(); // журнал бібліотеки не потрібен
        List<String> tasks = args.length > 0 ? List.of(args)
                : List.of("network", "fms", "minibus", "conveyor", "export");
        for (String task : tasks) {
            switch (task) {
                case "network" -> task("task1", QueueNetwork::main);
                case "fms" -> task("task2", Fms::main);
                case "minibus" -> task("task3", Minibus::main);
                case "conveyor" -> task("task4", Conveyor::main);
                case "export" -> Export.main(Path.of("nets"), RESULTS);
                default -> throw new IllegalArgumentException(task);
            }
        }
    }

    private static void task(String name, Function<Output, Map<String, Object>> model) {
        Output out = new Output();
        long start = System.nanoTime();
        Map<String, Object> json = model.apply(out);
        out.save(RESULTS.resolve(name + "_output.txt"));
        Output.write(RESULTS.resolve(name + ".json"), Json.write(json));
        System.out.printf("[%s: %.1f c]%n%n", name, (System.nanoTime() - start) / 1e9);
    }
}
```

Файл compare_lab5.py

```
"""Порівняння Петрі-об'єктних моделей практикуму 7 з мережами та результатами практикуму 5."""
import json
import math
import sys
from pathlib import Path

from scipy import stats

HERE = Path(__file__).resolve().parent
LAB5 = HERE.parent / "lab5" / "results"
RESULTS = HERE / "results"
SWEEP = {"throughput": "пропускна здатність", "time_in_system": "час перебування, с"}
NETS = [("fms_variant1", "net2_fms_variant1"), ("fms_variant2", "net2_fms_variant2"),
        ("minibus", "net3_minibus"), ("conveyor", "net1_conveyor")]


def load(path):
    return json.loads(path.read_text(encoding="utf-8"))


def structure(net):
    """Склад мережі без урахування порядку елементів, разом з дугами, що переносять мітки маркерів."""
    items = {("P", p["name"], p["mark"]) for p in net["places"]}
    for t in net["transitions"]:
        items.add(("T", t["name"], t["distribution"], round(t["mean"], 9), round(t["deviation"], 9),
                   t["priority"], round(t["probability"], 9)))
        items |= {("I", a["place"], t["name"], a["n"]) for a in t["inputs"]}
        items |= {("II", a["place"], t["name"], a["n"]) for a in t["info_inputs"]}
        items |= {("O", t["name"], a["place"], a["n"], a.get("carry")) for a in t["outputs"]}
    return items


def welch(m5, c5, n5, m7, c7, n7):
    """p-значення критерію Велча за середніми і півширинами 95 % довірчих інтервалів."""
    s5 = c5 / stats.t.ppf(0.975, n5 - 1)
    s7 = c7 / stats.t.ppf(0.975, n7 - 1)
    se = math.hypot(s5, s7)
    if se == 0:
        return 1.0
    df = se ** 4 / (s5 ** 4 / (n5 - 1) + s7 ** 4 / (n7 - 1))
    return 2 * stats.t.sf(abs((m7 - m5) / se), df)


def row(metric, old, new, n5, n7):
    m5, c5 = old
    m7, c7 = new["mean"], new["ci"]
    return {"metric": metric, "lab5": [m5, c5], "lab7": [m7, c7], "p": float(welch(m5, c5, n5, m7, c7, n7))}


def show(rows, nd=4):
    for r in rows:
        (m5, c5), (m7, c7) = r["lab5"], r["lab7"]
        print(f"  {r['metric']:46} ЛР5 {m5:10.{nd}f} ± {c5:.{nd}f}   ЛР7 {m7:10.{nd}f} ± {c7:.{nd}f}   "
              f"p = {r['p']:.3f}")


def compare_structure(report):
    ok = True
    print("Структура мережі Петрі-об'єктної моделі (після об'єднання позицій) і мережі практикуму 5:")
    for name, net5 in NETS:
        s5, s7 = structure(load(LAB5 / f"{net5}.json")), structure(load(RESULTS / f"net_{name}.json"))
        same = s5 == s7
        ok &= same
        places = sum(1 for i in s7 if i[0] == "P")
        transitions = sum(1 for i in s7 if i[0] == "T")
        arcs = sum(1 for i in s7 if i[0] in ("I", "II", "O"))
        report[name] = {"same": same, "places": places, "transitions": transitions, "arcs": arcs}
        verdict = "збігається" if same else "НЕ збігається"
        print(f"  {name}: {verdict} ({places} позицій, {transitions} переходів, {arcs} дуг, "
              "перенесення міток теж)")
        for item in sorted(s5 ^ s7, key=str):
            print("     ", "лише ЛР5" if item in s5 else "лише ЛР7", item)
    return ok
```

```
def compare_fms(report):
    t5, t7 = load(LAB5 / "task2.json"), load(RESULTS / "task2.json")
    n = t7["runs"]
    labels = dict(t7["metrics"]) | {"arrivals": "інтенсивність надходження, деталей/год"}
    for variant in ("1", "2"):
        for mode in ("base", "capacity"):
            old, new = t5[mode][variant], t7[mode][variant]
            rows = [row(labels[k], v, new[k], 20, n) for k, v in old.items()
                    if k in new and k in labels]
            if variant == "1" and mode == "base":
                rows += [row(f"завантаження робота {k + 1}", v, new["robot_load"][k], 20, n)
                         for k, v in enumerate(old["robot_load"])]
            report[f"{mode}_{variant}"] = rows
            title = "інтервал 40 с" if mode == "base" else "необмежене надходження"
            print(f"\nТВС, варіант {variant}, {title}:")
            show(rows, 2)
        sweep = []
        for a, v in t5["sweep"][variant].items():
            for k in ("throughput", "time_in_system"):
                label = f"{SWEEP[k]}, інтервал {float(a):.0f} с"
                sweep.append(row(label, v[k], t7["sweep"][variant][a][k], 20, n))
        report[f"sweep_{variant}"] = sweep
        print(f"ТВС, варіант {variant}, залежність від інтервалу надходження:")
        show(sweep, 2)


def compare_minibus(report):
    t5, t7 = load(LAB5 / "task3.json"), load(RESULTS / "task3.json")
    names = {"wait": "очікування, хв", "wait_queued": "очікування тих, хто чекав, хв",
             "queue": "середня черга", "boarded": "перевезено пасажирів", "lost": "пішли до іншого маршруту",
             "revenue": "виручка, грн"}
    rows = []
    for r5, r7 in zip(t5["sweep"], t7["sweep"]):
        for k, label in names.items():
            rows.append({**row(label, r5[k], r7[k], 50, t7["runs"]), "n": r5["n"]})
    small = int(sum(r["p"] < 0.05 for r in rows))
    report.update({"rows": rows, "tests": len(rows), "p_below_005": small})
    print(f"\nМікроавтобуси: {len(rows)} порівнянь (n = 1…25, 6 показників), p < 0,05 - "
          f"{small} (очікувано випадково ≈ {0.05 * len(rows):.1f}); n = 25:")
    show([r for r in rows if r["n"] == 25], 2)


def compare_conveyor(report):
    t5, t7 = load(LAB5 / "task1.json"), load(RESULTS / "task4.json")
    rows = [row(r["metric"], [r["mean"], r["ci"]], t7["table"][r["metric"]], 20, t7["runs"])
            for r in t5["table"]]
    keys = ("mean", "stdev", "median", "p90", "p95", "max")
    times = {k: [t5["time_stats"][k], t7["time_stats"][k]] for k in keys}
    report.update({"rows": rows, "time_stats": times})
    print("\nКонвеєр (задача 1 ЛР5):")
    show(rows)
    print("  розподіл часу перебування, ЛР5 / ЛР7: "
          + ", ".join(f"{k} {a:.2f} / {b:.2f}" for k, (a, b) in times.items()))


def main():
    report = {"structure": {}, "fms": {}, "minibus": {}, "conveyor": {}}
    ok = compare_structure(report["structure"])
    compare_fms(report["fms"])
    compare_minibus(report["minibus"])
    compare_conveyor(report["conveyor"])
    text = json.dumps(report, ensure_ascii=False, indent=1)
    (RESULTS / "lab5_comparison.json").write_text(text, encoding="utf-8")
    sys.exit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

3. Результати роботи програми:

Завдання 1:

Петрі-об'єктне моделювання (І. В. Стеценко) поєднує стохастичну мережу Петрі як опис динаміки з об'єктно-орієнтованим підходом: Петрі-об'єкт – це об'єкт класу, полем якого є мережа Петрі, а модель складається з Петрі-об'єктів так само, як програма – з екземплярів класів. У бібліотеці PetriObjModelPaint Петрі-об'єкт – це `PetriSim`: мережа `PetriNet`, поточний момент найближчої події `timeMin` і перехід `eventMin`, пріоритет і номер об'єкта. Модель `PetriObjModel` – список Петрі-об'єктів і перелік зв'язків `PetriObjLink`. Основний зв'язок – об'єднання позицій (`linkObjectsCombiningPlaces`): елемент масиву позицій мережі одного об'єкта замінюється тим самим екземпляром `PetriP`, що належить іншому об'єкту, тож обидва об'єкти бачать одне маркування. Другий вид зв'язку – дуга від переходу одного об'єкта до позиції іншого (`linkTransitionToPlace`). Перехід завжди бере маркери лише з позицій своєї мережі, тому вимога потрапляє до іншого об'єкта тільки через спільну позицію.

Метод `go(T)` класу `PetriObjModel` реалізує такий алгоритм (перевірено за вихідним кодом і байт-кодом збірки 2.3.0):

1. Об'єкти впорядковуються за спаданням пріоритету, і кожен виконує `input()`: поки в його мережі є переходи з виконаною умовою запуску (`findActiveT`), вибирається перехід з найвищим пріоритетом, серед рівних – випадково з урахуванням ймовірностей (`doConflikt`), і виконується вхід маркерів (`actIn`: маркери забираються з вхідних позицій, у масив `timeOut` переходу додається момент виходу $t + \tau$). Потім `eventMin()` визначає найближчий момент виходу маркерів з переходів об'єкта.
2. Поки поточний час менший за T: обчислюється момент наступної події – мінімум `timeMin` за всіма об'єктами; для кожного об'єкта накопичуються середні значення маркування позицій і кількості маркерів у переходах (`doStatistics`), викликається збирач статистики (якщо він заданий); час просувається.
3. Об'єкти, у яких `timeMin` дорівнює поточному часу, утворюють список конфліктних об'єктів; вибирається один – з найвищим пріоритетом, серед рівних випадково. Лише цей об'єкт виконує `doT()` і `output()`: вихід

маркерів з усіх своїх переходів, момент виходу яких дорівнює поточному часу (з усіх таких каналів багатоканального переходу).

4. Список об'єктів перемішується (`Collections.shuffle`) і знову впорядковується за пріоритетом, після чого всі об'єкти виконують `input()`: вихід маркерів міг змінити маркування спільних позицій, тож нові переходи могли стати дозволеними в будь-якому об'єкті. Інші об'єкти з подією в той самий момент обробляються на наступних ітераціях без просування часу (так само виконуються переходи з нульовою затримкою).

Отже, стан моделі – це стани всіх об'єктів (маркування і масиви моментів виходу), час просувається до найближчої події серед усіх об'єктів, а конфлікт за маркер спільної позиції розв'язується порядком, у якому об'єкти виконують вхід маркерів: першим – об'єкт з вищим пріоритетом, серед рівних – випадковий. Під час вивчення коду і перевірок виявлено такі особливості:

- середні значення бібліотеки оновлюються з вагою $(t_{n+1} - t_n) / t_{n+1}$, тобто це точне середнє за часом (на відміну від `PetriSim.step()`, де вага $\Delta t / T$ давала зміщене середнє, як з'ясовано в практикумі 6), а `go()` не зупиняє модель з генератором після першої події. Розгону бібліотека не має, тому статистику з розгоном збирає власний `Collector`, підключений через інтерфейс бібліотеки `SimulationStatisticCollector`; алгоритм `go()` при цьому не змінюється;
- кожна позиція враховується в статистиці об'єктом, якому вона належить: у мережі об'єкта-копії замінено лише елемент масиву позицій, а список позицій для статистики лишається старим, тож спільна позиція не рахується двічі. Для прогону без розгону власні середні і середні бібліотеки (`PetriP.getMean`, `PetriT.getMean`) збігаються з розбіжністю $4{,}0 \cdot 10^{-10}$;
- у виборі з конфліктних об'єктів за різних пріоритетів верхня межа випадкового номера обчислюється як `i - 1` замість `i`, тобто завжди береться перший об'єкт групи з найвищим пріоритетом; оскільки список щоразу перемішується, вибір серед рівних усе одно випадковий (частка вимог двох однакових об'єктів зі спільною чергою в тесті – 0,5000 ± 0,0006);

- одночасні події різних об'єктів виконуються по одній з входом маркерів між ними, а не «усі виходи, потім усі входи», як в одній мережі. Для неперервних розподілів це несуттєво, а для детермінованих затримок порядок одночасних подій впливає на результат (завдання 4) і задається пріоритетами об'єктів;
- `Math.random`, `Collections.shuffle` і `new Random()` не мають зерна, тому прогони не відтворювані – точність забезпечують серії прогонів з довірчими інтервалами;
- маркери неіменовані. Щоб виміряти час перебування окремої вимоги, `Tracker` зберігає мітки маркерів поза мережею: черга міток для позиції і список міток каналів для переходу в тому самому порядку, що й масив `timeOut`; під час виходу маркера бібліотека повідомляє номер каналу (`PetriT.getNum()`). Узгодженість міток з маркуванням перевіряється після кожної події;
- збирач отримує в `onSimulationEnd` момент наступної, вже не виконаної події, а не T, тому останній інтервал статистики завершується в момент T самим збирачем.

Для тестування побудовано відкриту мережу масового обслуговування, для якої відомі точні результати (мережа Джексона). Модель складено з Петрі-об'єктів трьох класів: `Generator` (надходження вимог з інтервалом exp(1), $\lambda = 1$), `Smo` (СМО з с каналами, експоненційним обслуговуванням і необмеженою чергою; клас використано чотири рази) і `Fork` (розгалуження за ймовірностями; двічі). Вимога обслуговується у вузлі A, з ймовірністю 0,7 переходить у вузол B, з ймовірністю 0,3 – у вузол C; після B з ймовірністю 0,2 повертається у вузол A. Вузол B складено з двох однакових об'єктів «СМО B1» і «СМО B2» зі спільною чергою (об'єднано позиції черг і виходів), тому він є системою M/M/2, а конфлікт двох об'єктів за вимогу перевіряє розв'язання конфліктів між об'єктами.

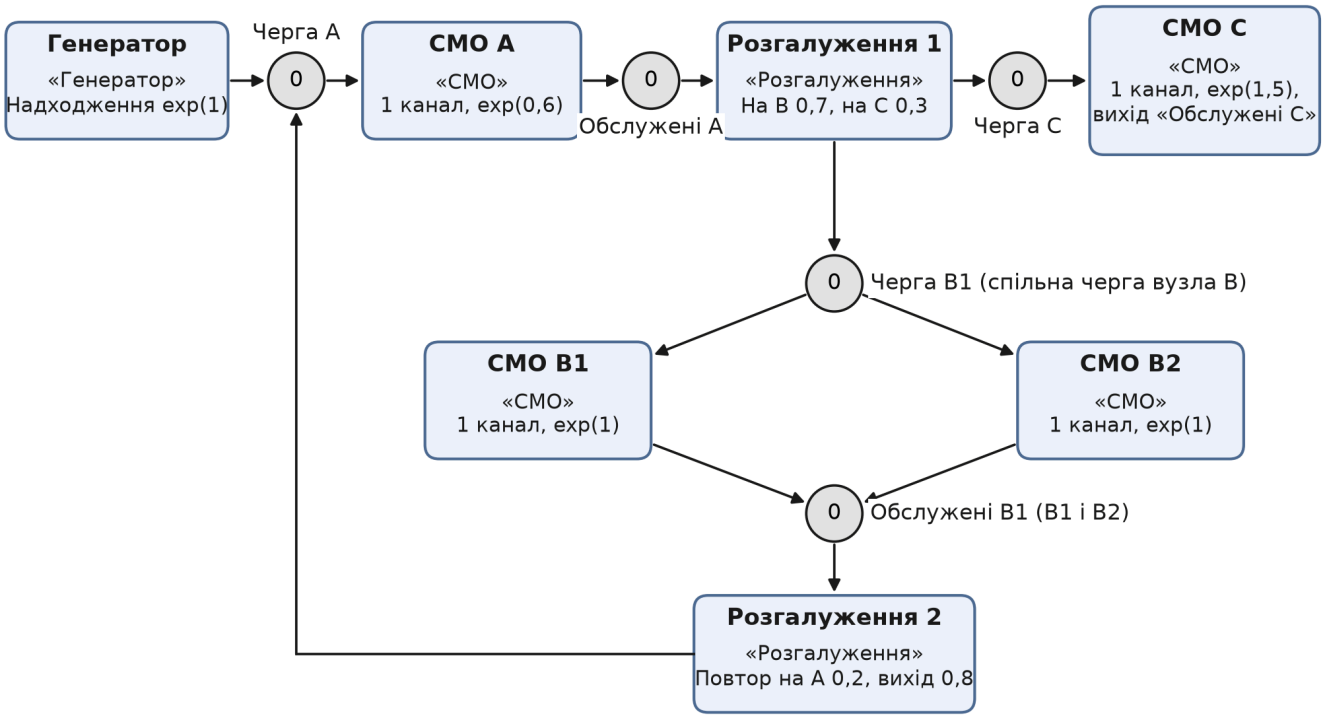


Рисунок 1 – Схема Петрі-об’єктної моделі мережі масового обслуговування (рамки – Петрі-об’єкти, кола – спільні позиції з початковим маркуванням; стрілка від об’єкта – його переходи кладуть маркери в позицію, до об’єкта – забирають)

Таблиця 1 – Вузли тестової мережі

Вузол (Петрі-об'єкти)	**Каналів c**	**Сер. час обслуговування**	**λ_i**	**ρ_i**
A (СМО A)	1	0,6	1,1628	0,6977
B (СМО B1, СМО B2)	2	1,0	0,8140	0,4070
C (СМО C)	1	1,5	0,3488	0,5233

На рис. 1 показано, з яких Петрі-об'єктів складено модель і через які спільні позиції вони передають вимоги; власні позиції й переходи об'єктів не зображено. Повну мережу моделі (усі позиції і переходи всіх об'єктів у рамках) збережено засобами бібліотеки в `nets/network/network.png`, а модель – у `nets/network/network.pnml`. Інтенсивності потоків у вузлах (табл. 1) знайдено з рівнянь балансу $\lambda_A = \lambda + 0{,}2\cdot\lambda_B$, $\lambda_B = 0{,}7\cdot\lambda_A$, $\lambda_C = 0{,}3\cdot\lambda_A$, звідки $\lambda_A = \lambda / 0{,}86$. Кожен вузол мережі Джексона в стаціонарному режимі поводиться як ізольована система M/M/c, тому середня черга обчислюється за формулою Ерланга:

$$L_q = C(c, a)\cdot a / (c - a), \quad a = \lambda_i s_i, \quad L = L_q + a, \quad W = L / \lambda_i, \quad W_{\text{мережі}} = \Sigma L_i / \lambda,$$

де C(c, a) – ймовірність очікування (формула Ерланга C). Початок трасування (у кожному рядку «вихід»/«вхід» указано об'єкт і перехід, у дужках –

найближчий момент виходу маркера з переходу):

```
t = 0.0000
  вхід:  Генератор: Надходження
  маркування: Вільні канали A=1, Вільні канали B1=1, Вільні канали B2=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (0.179)

t = 0.1792
  вихід: Генератор: Надходження
  вхід:  Генератор: Надходження, СМО A: Обслуговування A
  маркування: Вільні канали B1=1, Вільні канали B2=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (0.301), Обслуговування A=1 (0.824)

t = 0.3010
  вихід: Генератор: Надходження
  вхід:  Генератор: Надходження
  маркування: Черга A=1, Вільні канали B1=1, Вільні канали B2=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (0.477), Обслуговування A=1 (0.824)

t = 0.4771
  вихід: Генератор: Надходження
  вхід:  Генератор: Надходження
  маркування: Черга A=2, Вільні канали B1=1, Вільні канали B2=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (1.173), Обслуговування A=1 (0.824)

t = 0.8243
  вихід: СМО A: Обслуговування A
  вхід:  Розгалуження 1: На B, СМО A: Обслуговування A
  маркування: Черга A=1, Вільні канали B1=1, Вільні канали B2=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (1.173), Обслуговування A=1 (0.833), На B=1
(0.824)

t = 0.8243
  вихід: Розгалуження 1: На B
  вхід:  СМО B2: Обслуговування B2
  маркування: Черга A=1, Вільні канали B1=1, Вільні канали C=1
  у переходах (найближчий вихід): Надходження=1 (1.173), Обслуговування A=1 (0.833),
Обслуговування B2=1 (2.234)
```

За одну ітерацію `go()` вихід маркерів виконує один об'єкт: коли «СМО A» завершує обслуговування, вимогу забирає «Розгалуження 1»; переходи розгалуження мають нульову затримку, тож вихід з них – окрема подія в той самий момент часу, після якої вимогу забирає об'єкт СМО наступного вузла.

Таблиця 2 – Тестова мережа: імітація і теорія

Вузол	Показник	Імітація (95 % ДІ)	Теорія	Теорія в ДІ
A	λ_i	1,1615 ± 0,0019	1,1628	так
	ρ	0,6972 ± 0,0016	0,6977	так
	L_q	1,6127 ± 0,0239	1,6100	так
	L	2,3099 ± 0,0251	2,3077	так
	W	1,9886 ± 0,0196	1,9846	так
B	λ_i	0,8128 ± 0,0016	0,8140	так
	ρ	0,4070 ± 0,0011	0,4070	так
	L_q	0,1626 ± 0,0018	0,1616	так
	L	0,9765 ± 0,0037	0,9755	так
	W	1,2014 ± 0,0029	1,1985	ні
C	λ_i	0,3487 ± 0,0007	0,3488	так
	ρ	0,5221 ± 0,0014	0,5233	так
	L_q	0,5686 ± 0,0073	0,5743	так
	L	1,0907 ± 0,0082	1,0976	так
	W	3,1275 ± 0,0226	3,1463	так
мережа	Σ L	4,3771 ± 0,0314	4,3808	так
мережа	$W = \Sigma L / \lambda$	4,3816 ± 0,0268	4,3808	так
мережа	W за мітками вимог	4,3816 ± 0,0268	4,3808	так

Результати 20 прогонів по 100000 одиниць часу з розгоном 1000 (≈ 530 тис. подій за прогін) наведено в табл. 2: теоретичне значення потрапляє в 95 % довірчий інтервал для 17 з 18 показників, для решти відносне відхилення менше 1 %. Середній час перебування в мережі, виміряний за мітками маркерів для кожної вимоги, збігся з оцінкою за формулою Літтла і з теорією. Частки маршрутів дорівнюють заданим: на B 0,6998 ± 0,0005 (задано 0,7), повтор на A 0,2001 ± 0,0006 (задано 0,2). Розгалуження – окремий Петрі-об'єкт, тож конфліктна група `doConflikt` складається лише з переходів вибору маршруту; викривлення ймовірностей, знайдене в практикумі 6 для мережі, де в групу потрапляли й інші переходи того самого пріоритету, тут не виникає.

Після кожної з 530211 подій контрольного прогону перевірялися інваріанти: генератор (маркер у позиції або в переході), канали кожної СМО («Вільні канали» + маркер в «Обслуговуванні» = 1), збереження вимог (створені = вийшли + у мережі) і узгодженість міток маркерів з маркуванням – порушень немає.

Таблиця 3 – Об'єкт «СМО B1» має пріоритет 1

Повтор на A	λ_B	Частка B1	Теорія	ρ_B	Теорія	$L_{q,B}$	Теорія
0,2	0,8140	0,6176 ± 0,0007	0,6162	0,4064 ± 0,0007	0,4070	0,1599 ± 0,0016	0,1616
0,0	0,7000	0,6414 ± 0,0007	0,6416	0,3500 ± 0,0010	0,3500	0,0985 ± 0,0012	0,0977

Пріоритет об'єкта перевірено окремо: якщо «СМО B1» має пріоритет 1, вимога, що застала обидва канали вузла B вільними, завжди потрапляє до B1. Для M/M/2 з упорядкованим зайняттям каналів частку B1 можна знайти з рівняння балансу стану «зайнятий лише другий канал» $\mu \cdot p_2 = (\lambda + \mu) \cdot p_{1,2}$: частка = $(p_1 - p_{1,2} + P(N \geq 2))$ / a. Без повторів на A потік у вузол B пуассонівський, і частка 0,6414 ± 0,0007 збігається з теоретичною 0,6416 (табл. 3). З повторами частка B1 (0,6176 ± 0,0007) трохи більша за 0,6162: потік на виході вузла A, що містить повернення, у мережі зі зворотним зв'язком не є пуассонівським, і формула для B1 стає наближеною, тоді як завантаження і черга вузла B від розподілу вимог між каналами не залежать і збігаються з M/M/2. Отже, тестування підтверджує, що алгоритм `PetriObjModel.go()` правильно відтворює динаміку мережі, складеної з Петрі-об'єктів: часові середні, маршрутизацію за ймовірностями, спільні ресурси і пріоритети об'єктів.

Завдання 2:

Петрі-об'єктна модель гнучкої виробничої системи побудована розкладанням мережі практикуму 5 на Петрі-об'єкти. Тлумачення умови те саме: пункт прибуття, верстати і склад стоять на одній лінії (ходи 6, 7, 5 с, між несусідніми точками – сума), захоплення і вивільнення 8 ± 1 с, місце на верстаті призначення резервується в момент призначення робота і звільняється, коли наступний робот захопить оброблену деталь; у варіанті 1 робот після вивільнення повертається на початок свого шляху, у варіанті 2 – залишається там, куди привіз деталь, і на деталь викликається найближчий вільний робот. Класи Петрі-об'єктів:

Таблиця 4 – Класи Петрі-об'єктів моделі ГВС

Клас	Об'єкти	Власні позиції	Переходи
Generator	Генератор	Генератор, Деталь на вході	Надходження exp(40) с
Machine	Верстат B1, Верстат B2	Деталь на Bk, Вільні місця Bk (3), Оброблена на Bk	Обробка на Bk: N(60; 10) / exp(100) с
Transfer	Шлях вхід–B1, Шлях B1–B2, Шлях B2–склад	Робот біля деталі, Деталь захоплена, Деталь доставлена (Склад – у третього)	Захоплення 8 ± 1, Переміщення 6 / 7 / 5, Вивільнення 8 ± 1 с
AssignedRobot (варіант 1)	Робот 1, Робот 2, Робот 3	Робот k вільний (1), Робот k повертається	Призначення робота k (0), Повернення робота k (6 / 7 / 5 с)
RobotPool (варіант 2)	Парк роботів	Вільний робот: вхід / B1 / B2 / склад (1, 1, 1, 0)	12 переходів «Виклик робота: L → шлях», затримка – холостий хід, пріоритет 100 − хід

Спільні позиції (табл. 4) належать об'єкту, який їх «утримує»: деталь на вході – генератору, деталь на верстаті, оброблена деталь і вільні місця – верстату, «Робот біля деталі» – шляху, вільні роботи – роботу або парку роботів. Інші об'єкти мають у своїй мережі позицію з тим самим ім'ям, яку `ObjectModel.share` об'єднує з позицією-джерелом. Так робот k читає запит з позиції верстата (або генератора), резервує місце на верстаті призначення і передає деталь об'єкту «Шлях», а «Шлях» після захоплення звільняє місце на верстаті, з якого забрав деталь, і повертає робота у його позицію. Варіант 1 містить 9 Петрі-об'єктів і 15 об'єднань позицій, варіант 2 – 7 об'єктів і 15 об'єднань; варіанти відрізняються лише об'єктами роботів, решта об'єктів однакова.

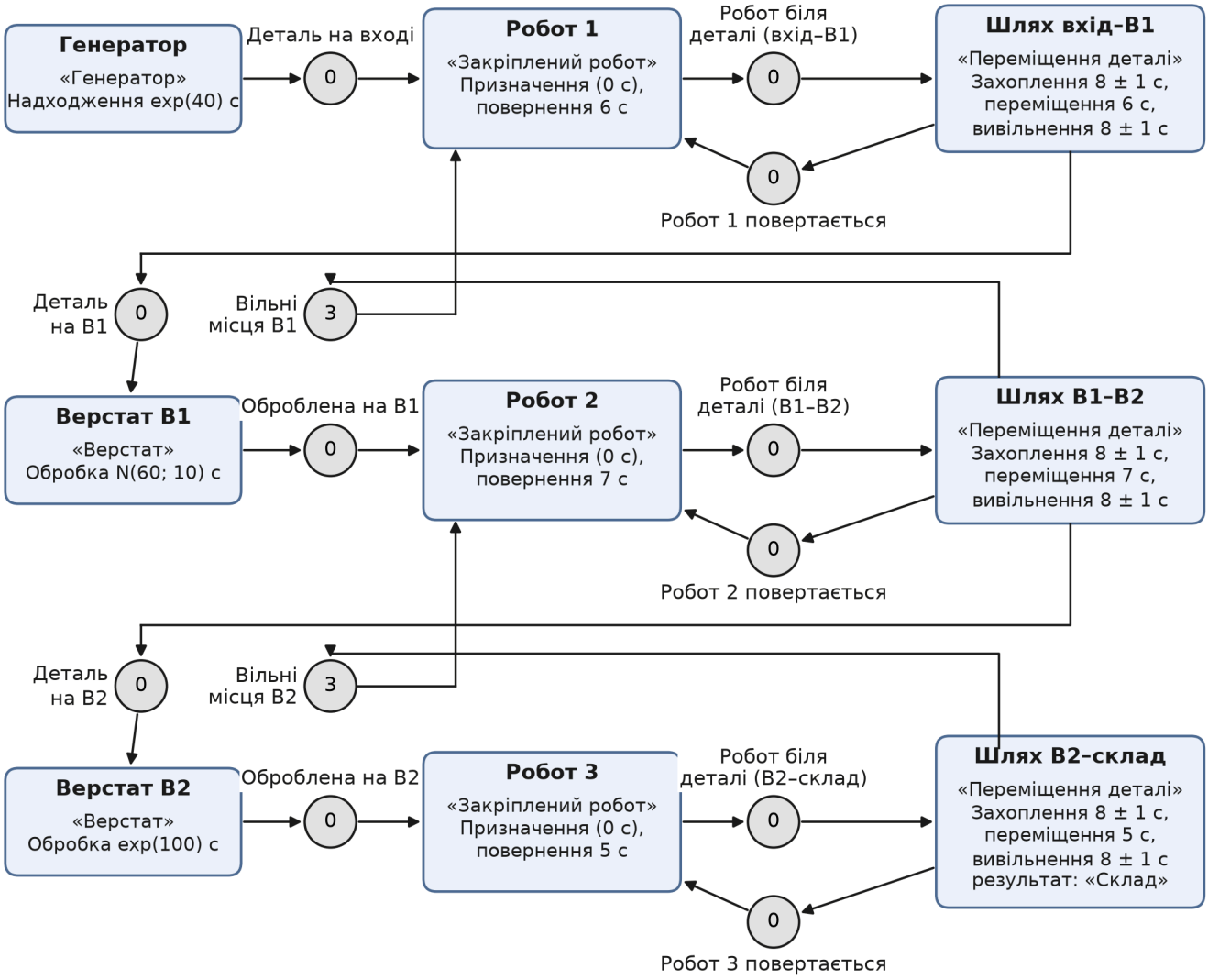


Рисунок 2 – Схема Петрі-об’єктної моделі ГВС, варіант 1 (закріплені роботи; рамки – Петрі-об’єкти, кола – спільні позиції з початковим маркуванням; стрілка від об’єкта – його переходи кладуть маркери в позицію, до об’єкта – забирають)

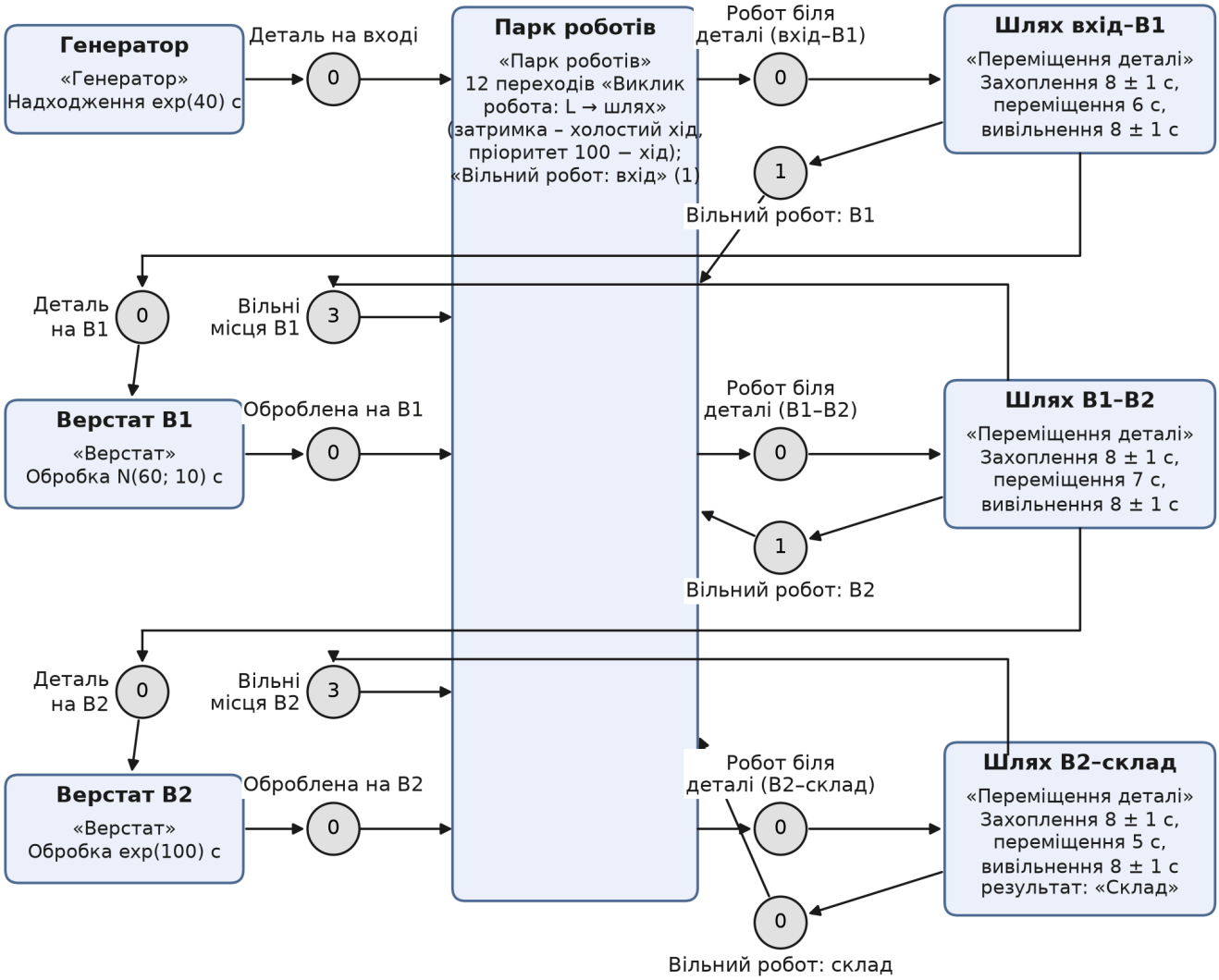


Рисунок 3 – Схема Петрі-об’єктної моделі ГВС, варіант 2 (парк роботів)

Схеми моделей наведено на рис. 2, 3; повні мережі моделей з усіма елементами об'єктів – у `nets/fms_variant1/fms_variant1.png` і `nets/fms_variant2/fms_variant2.png`. Чому в варіанті 2 всі виклики роботів зібрано в одному Петрі-об'єкті, а не розподілено між об'єктами шляхів? Вимога «займається найближчий робот» – це конфлікт переходів різних шляхів за маркер одного робота, який у мережі практикуму 5 розв'язується пріоритетами 100 − (час ходу) серед усіх викликів. Пріоритети переходів діють лише в межах одного об'єкта, а між об'єктами конфлікт розв'язується порядком входу маркерів (пріоритет об'єкта, далі випадково), тож розподілені виклики дали б іншу дисципліну призначення. Об'єкт «Парк роботів» зберігає глобальне правило. Інших відступів від мережі практикуму 5 немає: `compare_lab5.py` порівнює

мережу, яку утворює об'єднання мереж об'єктів після об'єднання позицій, з описом мережі практикуму 5, і вони збігаються повністю (варіант 1 – 24 позицій, 18 переходів, 47 дуг; варіант 2 – 22, 24, 74), включно з дугами, що переносять мітку деталі. Модель збережено в `nets/fms_variant1.pnml` і `nets/fms_variant2.pnml` (по сторінці на Петрі-об'єкт і перелік об'єднань), прочитано назад і перевірено, що відновлена модель збігається з вихідною.

Перший у прогоні виклик робота, що стоїть не на початку шляху (варіант 2):

```
t = 51.6416
  вихід: Шлях вхід–В1: Вивільнення (вхід–В1)
  вхід:  Верстат В1: Обробка на В1
  маркування: Вільні місця В1=2, Вільні місця В2=3, Вільний робот: В1=2, Вільний робот: В2=1
  у переходах (найближчий вихід): Надходження=1 (76.694), Обробка на В1=1 (113.042)

t = 76.6944
  вихід: Генератор: Надходження
  вхід:  Генератор: Надходження, Парк роботів: Виклик робота: В1 → вхід–В1
  маркування: Вільні місця В1=1, Вільні місця В2=3, Вільний робот: В1=1, Вільний робот: В2=1
  у переходах (найближчий вихід): Надходження=1 (196.626), Обробка на В1=1 (113.042), Виклик
робота: В1 → вхід–В1=1 (82.694)
```

Деталь чекає на шляху вхід–В1, біля неї вільного робота немає, тож «Парк роботів» викликає найближчого вільного – з точки «В1»: перехід «Виклик робота: В1 → вхід–В1» триває 6 с холостого ходу, і лише потім об'єкт «Шлях вхід–В1» починає захоплення. Після кожної з 38840 (варіант 1) і 30880 (варіант 2) подій контрольних прогонів перевірялися інваріанти: кожен робот перебуває рівно в одному стані (у варіанті 2 – сума вільних, викликаних і зайнятих роботів дорівнює 3), місця кожного верстата зберігаються (вільні + зарезервовані + на верстаті + оброблені, які ще не забрав робот = 3), кількість деталей зберігається, мітки маркерів узгоджені з маркуванням. Порушень немає.

Таблиця 5 – ГВС: результати Петрі-об'єктної моделі і практикуму 5

Показник	Вар. 1, ЛР5	Вар. 1, ЛР7	Вар. 2, ЛР5	Вар. 2, ЛР7
пропускна здатність, деталей/год	76,08 ± 0,50	75,80 ± 0,62	71,79 ± 0,50	72,15 ± 0,34
середній час перебування деталі, с	9282,23 ± 629,86	9394,85 ± 731,06	11839,97 ± 663,42	11355,09 ± 578,34
середня черга на вході	219,81 ± 16,54	225,29 ± 18,05	282,66 ± 16,52	269,85 ± 15,12
середня кількість зайнятих роботів	1,78 ± 0,01	1,77 ± 0,01	1,71 ± 0,01	1,71 ± 0,01
зайняті місця B1 (з резервом)	3,00 ± 0,00	3,00 ± 0,00	2,95 ± 0,00	2,95 ± 0,00
зайняті місця B2 (з резервом)	2,90 ± 0,00	2,90 ± 0,00	2,79 ± 0,00	2,79 ± 0,01
деталей в обробці на B1	1,27 ± 0,01	1,26 ± 0,01	1,20 ± 0,01	1,20 ± 0,01
деталей в обробці на B2	2,10 ± 0,01	2,11 ± 0,01	2,02 ± 0,01	2,01 ± 0,01
оброблені на B1 чекають робота	1,10 ± 0,01	1,10 ± 0,02	0,81 ± 0,02	0,80 ± 0,01
оброблені на B2 чекають робота	0,14 ± 0,00	0,14 ± 0,00	0,02 ± 0,00	0,02 ± 0,00
роботів у холостому ході	0,38 ± 0,00	0,38 ± 0,00	0,39 ± 0,00	0,39 ± 0,00
інтенсивність надходження, деталей/год	89,43 ± 1,13	90,36 ± 0,95	89,98 ± 0,94	89,62 ± 0,87
завантаження робота 1	0,59 ± 0,00	0,59 ± 0,00	–	–
завантаження робота 2	0,63 ± 0,00	0,63 ± 0,01	–	–
завантаження робота 3	0,55 ± 0,00	0,55 ± 0,00	–	–
пропускна здатність при необмеженому надходженні, деталей/год	75,95 ± 0,38	76,40 ± 0,61	72,13 ± 0,45	72,18 ± 0,39

У табл. 5 – 20 прогонів по 100000 с з розгоном 10000 с при надходженні кожні 40 с, 95 % довірчі інтервали, поруч – результати практикуму 5 за тих самих умов. З 71 порівнянь (разом із залежністю від інтервалу надходження) p-значення критерію Велча менше 0,05 лише в 4 випадках (за збігу моделей випадково очікується ≈ 3,6), найменше – 0,025. Петрі-об'єктна модель відтворює результати мережі практикуму 5.

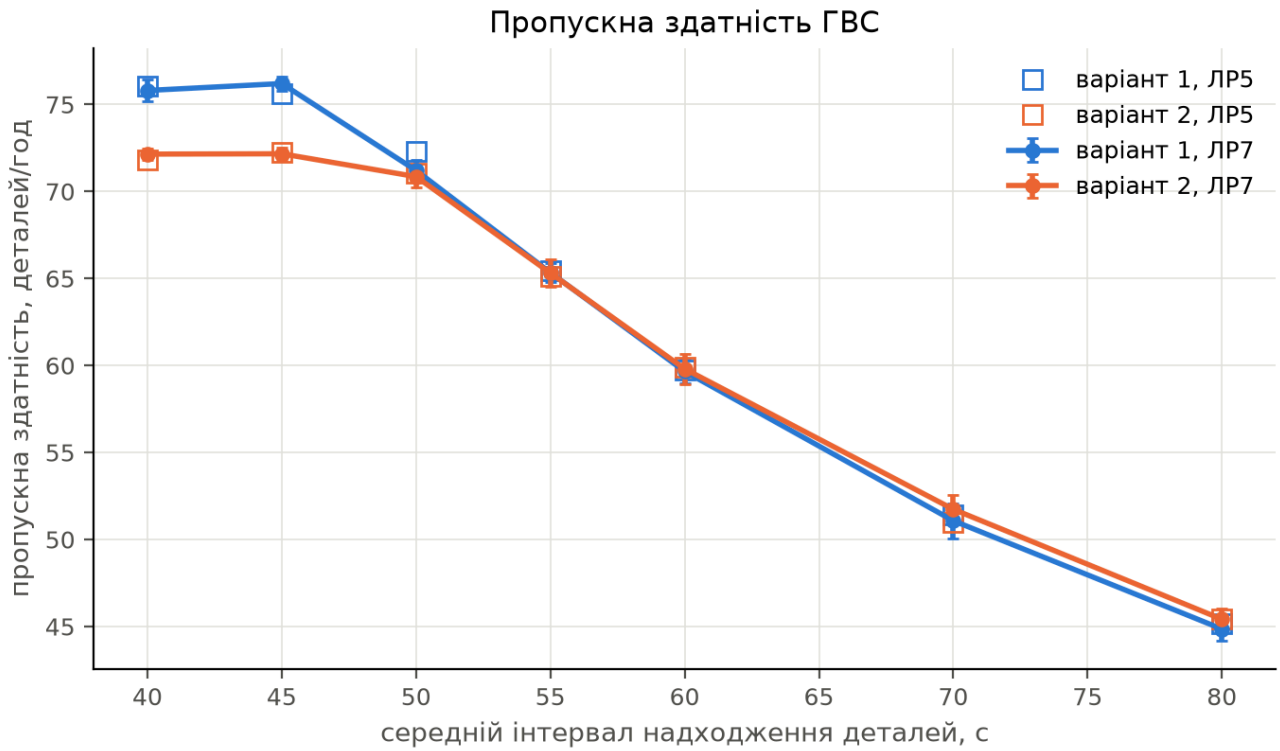


Рисунок 4 – Пропускна здатність ГВС залежно від середнього інтервалу надходження деталей (лінії – Петрі-об’єктна модель, квадрати – практикум 5)

Деталі надходять з інтенсивністю ≈ 90 за годину, а система пропускає лише 75,8 ± 0,6 (варіант 1) і 72,1 ± 0,3 (варіант 2) деталей за годину, тому черга на вході весь час зростає (середня – 225 і 270 деталей), а середній час перебування деталі – 2,6 і 3,2 год. Вузьке місце – верстат B1: його три місця зайняті 100,0 % часу, з них у середньому 1,10 місця тримають уже оброблені деталі, які чекають на робота, бо на B2 немає вільного місця (обробка на B2 триває в середньому 100 с проти 60 с на B1). При необмеженому надходженні пропускна здатність – 76,4 ± 0,6 і 72,2 ± 0,4 деталей/год (рис. 4). Варіант 1 кращий на 3,7 деталі за годину при надходженні кожні 40 с і на 4,2 – при необмеженому. У варіанті 2 робот, звільнившись, залишається в кінці шляху й до наступної деталі мусить їхати холостим ходом (у середньому 12,9 с для шляху вхід–B1, 4,4 с для B1–B2), а закріплений робот повертається на початок свого шляху відразу після вивільнення, поки деталь ще обробляється; до того ж спільний парк може віддати робота на шлях вхід–B1 саме тоді, коли він потрібен для звільнення місць на B1. При рідшому надходженні (60 с і більше) пропускна здатність обох варіантів дорівнює інтенсивності надходження, і варіанти відрізняються лише часом перебування деталі.

Завдання 3:

Модель маршруту мікроавтобусів побудована з Петрі-об'єктів трьох класів (тлумачення умови – як у практикумі 5: дві кінцеві зупинки, на кожну пасажири підходять незалежно через 0,5 ± 0,2 хв, черга до 30 осіб, мікроавтобус відправляється заповненим, після рейсу 5 ± 1 хв висадки і посадка на тій самій кінцевій; А стоїть на З1, В – на З2; день 600 хв; виручка = 20 грн · сіли − 20 грн · пішли):

- `Generator` (двічі) – «Генератор З1», «Генератор З2»: пасажир з'являється в позиції «Пасажир на Зs»;
- `Stop` (двічі) – «Зупинка З1», «Зупинка З2»: позиції «Пасажир на Зs», «Черга на Зs», «Місця в черзі Зs» (30 маркерів), «Пішли з Зs»; переходи «Стати в чергу» і «Відмова»;
- `Bus` (двічі) – «Мікроавтобус А», «Мікроавтобус В»: для кожної зупинки позиції «Х на Зs», «Вільні місця Х на Зs», «Пасажири в Х на Зs», «Х прибув на Зs», переходи посадки з черги і посадки щойно підійшлого пасажира, «Рейс Х Зs→Зt» (дуга кратності n), «Висадка Х на Зt»; позиції пасажирів, черг і місць у черзі об'єднано з позиціями зупинок.

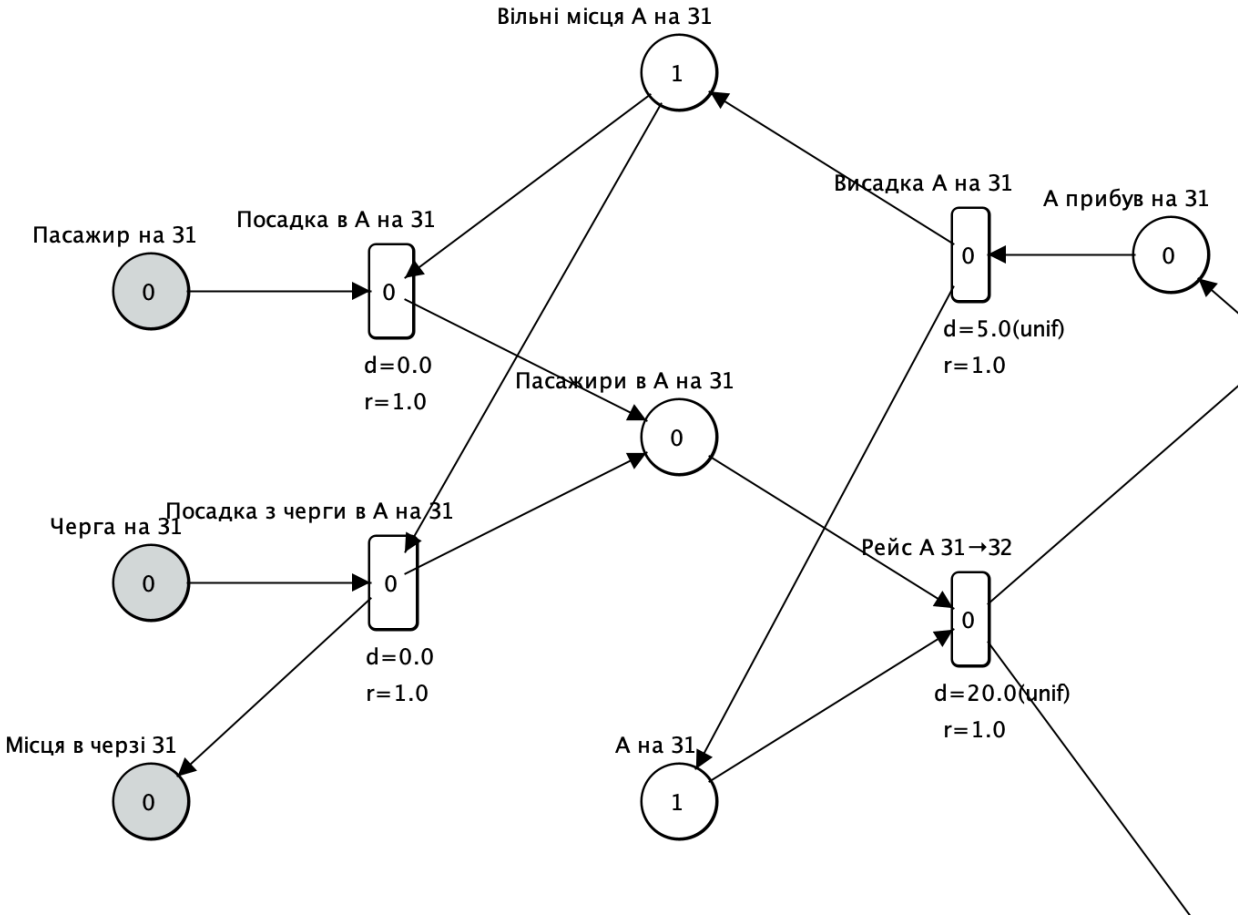


Рисунок 5 – Мережа Петрі-об’єкта «Мікроавтобус», частина для зупинки 31 (n = 1; сірі позиції об’єднуються з позиціями зупинок)

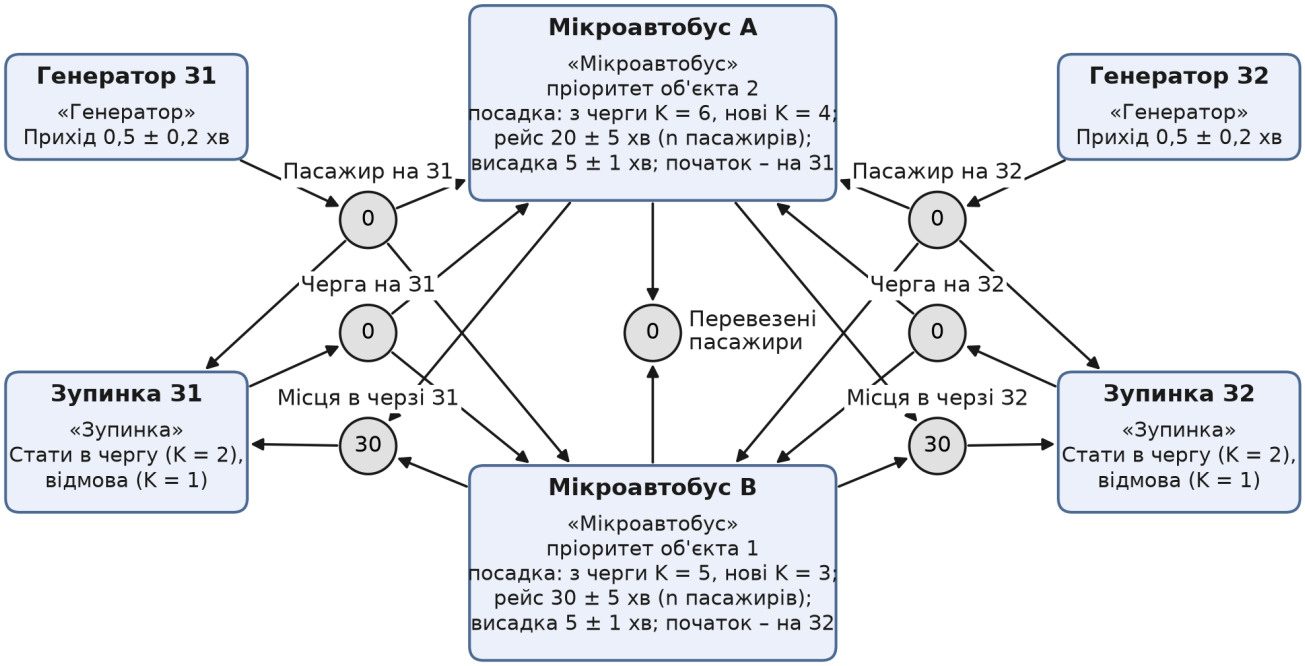


Рисунок 6 – Схема Петрі-об’єктної моделі маршруту мікроавтобусів (рамки – Петрі-об’єкти, кола – спільні позиції з початковим маркуванням; стрілка від об’єкта – його переходи кладуть маркери в позицію, до об’єкта – забирають)

Клас «Мікроавтобус» (рис. 5) описує обидві кінцеві зупинки; частина мережі для 32 – дзеркальне відображення частини для 31 (повна мережа класу – `nets/objects/Bus.png`, повна мережа моделі – `nets/minibus/minibus.png`). Модель (рис. 6) містить 6 Петрі-об'єктів і 15 об'єднань позицій. Правило «пасажир сідає в В, лише коли немає А» – це конфлікт переходів посадки різних об'єктів за маркер позиції «Пасажир на 3s». У мережі практикуму 5 його розв'язували пріоритети переходів: посадка в А (з черги 6, щойно підійшлого 4) вища, ніж у В (5, 3), а «Стати в чергу» (2) і «Відмова» (1) – найнижчі. У Петрі-об'єктній моделі ці пріоритети залишилися всередині об'єктів, а між об'єктами правило задано пріоритетами Петрі-об'єктів: «Мікроавтобус А» – 2, «Мікроавтобус В» – 1, зупинки й генератори – 0. Після кожної події об'єкти виконують вхід маркерів у порядку А, В, далі зупинки, тому пасажира першим забирає А, потім В, і лише залишок стає в чергу або йде. Вхід маркерів лише забирає маркери, тож об'єкт, що виконує вхід пізніше, не може зробити дозволеним перехід об'єкта, що вже виконав вхід, і порядок об'єктів відтворює глобальні пріоритети мережі практикуму 5 (з точністю до розташування посадки з черги в В перед посадкою щойно підійшлого пасажира в А, що вимагало б одночасно черги і вільного місця в А, а цього не буває). Мережа моделі після об'єднання позицій збігається з

мережею практикуму 5 (27 позицій, 22 переходів, 72 дуг для n = 1). Початок трасування для n = 3:

```
t = 0.0000
  вхід:  Генератор З1: Прихід на З1, Генератор З2: Прихід на З2
  маркування: А на З1=1, Вільні місця А на З1=3, В на З2=1, Вільні місця В на З2=3, Місця в черзі
З1=30, Місця в черзі З2=30
  у переходах (найближчий вихід): Прихід на З1=1 (0.521), Прихід на З2=1 (0.475)

t = 0.4753
  вихід: Генератор З2: Прихід на З2
  вхід:  Мікроавтобус В: Посадка в В на З2, Генератор З2: Прихід на З2
  маркування: А на З1=1, Вільні місця А на З1=3, В на З2=1, Вільні місця В на З2=2, Місця в черзі
З1=30, Місця в черзі З2=30
  у переходах (найближчий вихід): Посадка в В на З2=1 (0.475), Прихід на З1=1 (0.521), Прихід на
З2=1 (0.917)

t = 0.4753
  вихід: Мікроавтобус В: Посадка в В на З2
  маркування: А на З1=1, Вільні місця А на З1=3, В на З2=1, Вільні місця В на З2=2, Пасажири в В
на З2=1, Місця в черзі З1=30, Місця в черзі З2=30
  у переходах (найближчий вихід): Прихід на З1=1 (0.521), Прихід на З2=1 (0.917)

...

t = 1.3938
  вихід: Мікроавтобус В: Посадка в В на З2
  вхід:  Мікроавтобус В: Рейс В З2→З1
  маркування: А на З1=1, Вільні місця А на З1=1, Пасажири в А на З1=2, Місця в черзі З1=30, Місця
в черзі З2=30
  у переходах (найближчий вихід): Рейс В З2→З1=1 (33.892), Прихід на З1=1 (1.879), Прихід на З2=1
(1.773)

t = 1.7725
  вихід: Генератор З2: Прихід на З2
  вхід:  Зупинка З2: Стати в чергу на З2, Генератор З2: Прихід на З2
  маркування: А на З1=1, Вільні місця А на З1=1, Пасажири в А на З1=2, Місця в черзі З1=30, Місця
в черзі З2=29
  у переходах (найближчий вихід): Рейс В З2→З1=1 (33.892), Прихід на З1=1 (1.879), Прихід на З2=1
(2.335), Стати в чергу на З2=1 (1.773)
```

Пасажири, що підходять, одразу сідають у мікроавтобуси, які стоять на зупинках (А на З1, В на З2): першим вхід маркерів виконує об'єкт з вищим пріоритетом, а посадка – перехід з нульовою затримкою, тож вихід з неї – окрема подія в той самий момент. Коли в мікроавтобусі стало n = 3 пасажири, перехід рейсу забирає їх (дуга кратності 3), а наступний пасажир на цій зупинці стає в чергу. Після кожної з 4914 подій контрольного прогону (n = 10) перевірялися інваріанти: кожен мікроавтобус в одному місці, вільні місця + пасажири в автобусі на зупинці = n, черга + місця в черзі = 30, черги немає, поки на зупинці є вільне місце, баланс пасажирів, узгодженість міток.

Перевірка формулою Літтла на довгому прогоні 60000 хв (n = 25): середня черга на зупинці 23,66, $\lambda_{черги} \cdot W_{черги}$ = 0,7421·31,89 = 23,66. Час очікування виміряно для кожного пасажира за мітками маркерів (від приходу до посадки, для тих, хто сів одразу, – нуль), як у практикумі 5.

Таблиця 6 – Маршрут мікроавтобусів залежно від n (50 днів)

n	Очікування, хв, ЛР5	Очікування, хв, ЛР7	Тих, хто чекав, хв	Сіли	Пішли	Виручка, грн, ЛР5	Виручка, грн, ЛР7
1	287,69 ± 1,01	287,45 ± 1,20	301,83 ± 1,22	42	2297	-45080 ± 68	-45093 ± 67
2	265,60 ± 0,89	266,14 ± 0,98	279,45 ± 1,06	84	2253	-43411 ± 61	-43376 ± 62
3	210,61 ± 0,82	211,20 ± 0,75	221,73 ± 0,82	126	2214	-41703 ± 72	-41756 ± 86
5	142,85 ± 0,51	142,35 ± 0,58	149,47 ± 0,63	210	2131	-38392 ± 72	-38418 ± 76
10	75,55 ± 0,31	75,53 ± 0,36	79,32 ± 0,40	419	1921	-30113 ± 106	-30048 ± 119
15	49,66 ± 0,21	49,70 ± 0,22	52,21 ± 0,24	625	1719	-21853 ± 130	-21883 ± 137
20	35,40 ± 0,21	35,30 ± 0,24	38,04 ± 0,30	823	1519	-13957 ± 176	-13909 ± 219
25	26,87 ± 0,18	26,69 ± 0,17	31,37 ± 0,40	995	1357	-7374 ± 257	-7240 ± 281

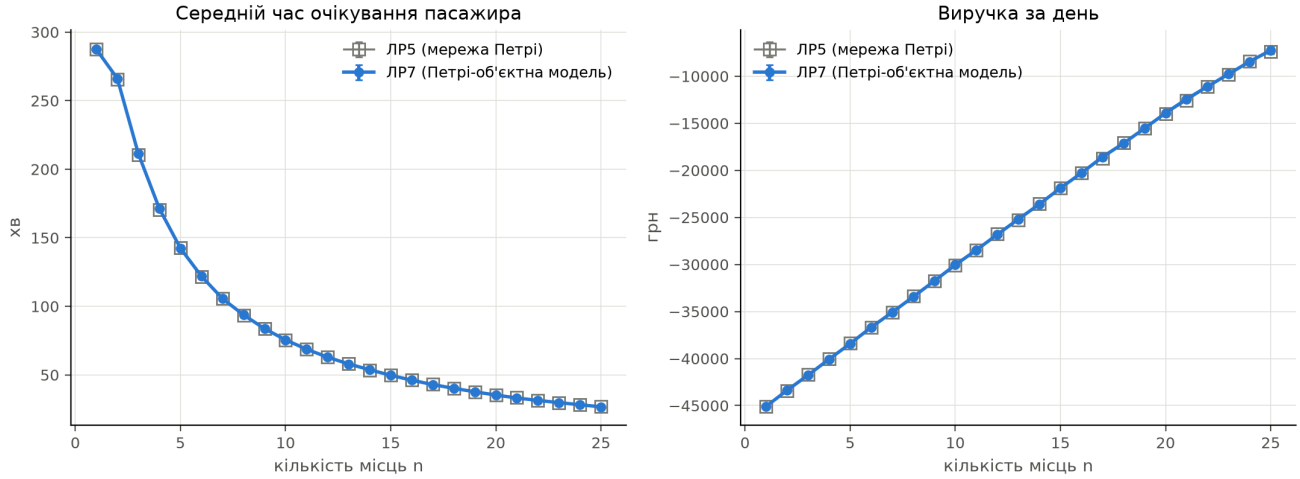


Рисунок 7 – Середній час очікування і виручка за день залежно від n: Петрі-об’єктна модель і практикум 5

Результати для всіх n = 1…25 наведено на рис. 7, для частини n – у табл. 6. З 150 порівнянь з практикумом 5 (шість показників для кожного n) $p < 0,05$ лише в 3 (випадково очікується ≈ 7,5), криві практично збігаються. Середній час очікування монотонно зменшується зі зростанням n і мінімальний при n = 25: 26,69 ± 0,17 хв для всіх пасажирів, що сіли, і 31,37 ± 0,40 хв для тих, хто стояв у черзі. Виручка за день при n = 25 – -7240 ± 281 грн: дохід 19894 грн від 995 пасажирів і втрати 27135 грн від 1357 пасажирів, що пішли. Маршрут перевантажений: на кожну зупинку приходить 2 пасажири за хвилину, а два мікроавтобуси перевозять в одному напрямку не більше $n \cdot (1/50 + 1/70) \approx 0,86$ пасажира за хвилину при n = 25 (у моделі – 0,82), тому черга майже весь час заповнена і понад половина пасажирів іде на інший маршрут; виручка від'ємна за будь-якого $n \leq 25$.

Завдання 4:

Петрі-об'єктну модель конвеєрної системи (задача 1 практикуму 5) складено з Петрі-об'єкта «Генератор» і п'яти однакових Петрі-об'єктів класу `Device`

«Пристрій Пі» (рис. 8, мережа класу – рис. 9); модель реалізовано (`Conveyor.java`) і перевірено, щоб рівняння описували працюючу модель.

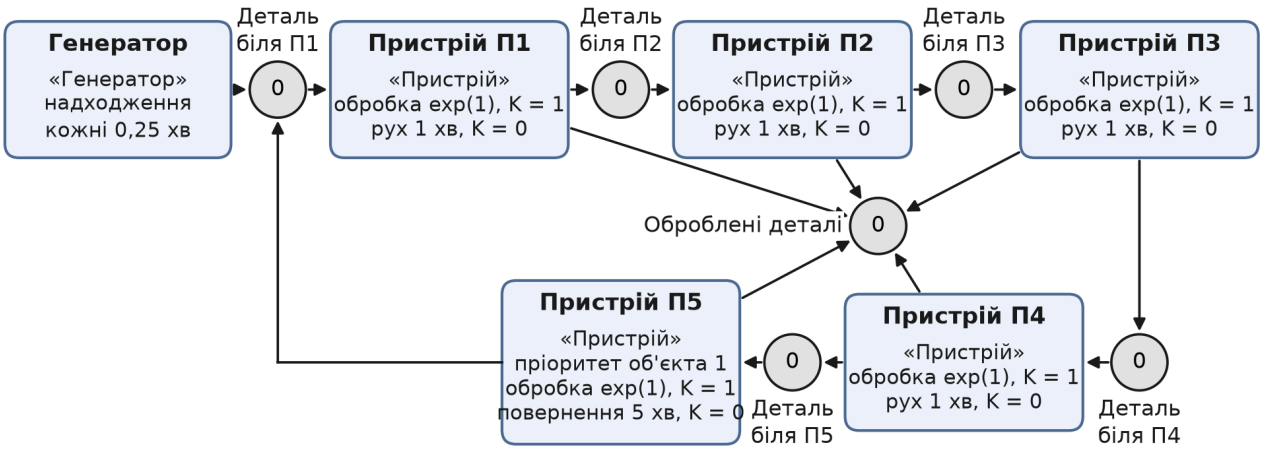


Рисунок 8 – Схема Петрі-об’єктної моделі конвеєрної системи (рамки – Петрі-об’єкти, кола – спільні позиції з початковим маркуванням; стрілка від об’єкта – його переходи кладуть маркери в позицію, до об’єкта – забирають)

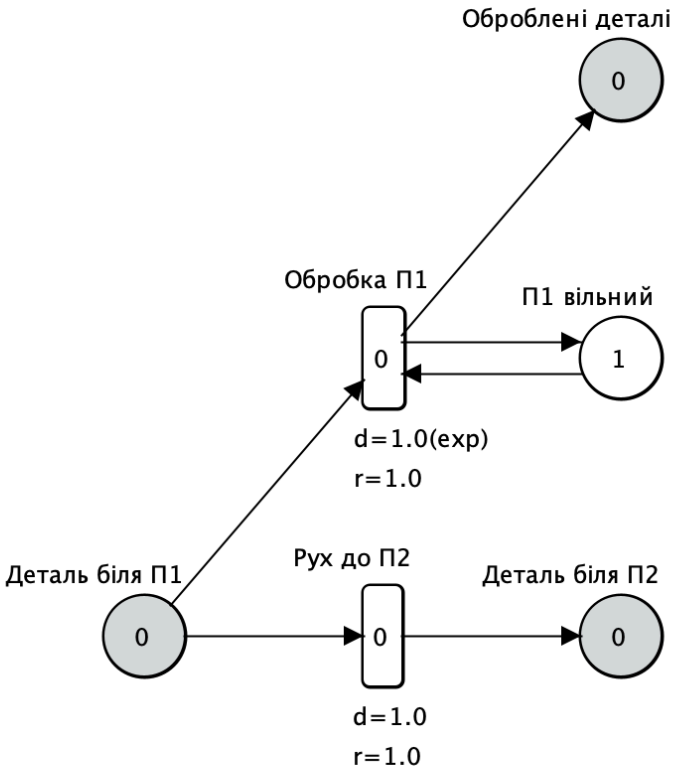


Рисунок 9 – Мережа Петрі-об’єкта «Пристрій» (для П1; сірі позиції об’єднуються з позиціями інших об’єктів)

Позначення. O_0 – генератор, O_i – пристрій Пі, i = 1, …, 5. Мережа O_0: позиції $P_{0,1}$ «Генератор», $P_{0,2}$ «Деталь біля П1» (вихід), перехід T_0 «Надходження». Мережа O_i: позиції $P_{i,1}$ «Деталь біля Пі», $P_{i,2}$ «Пі вільний», $P_{i,3}$ «Оброблені деталі», $P_{i,4}$ – деталь біля наступного пристрою; переходи $T_{i,1}$ «Обробка Пі», $T_{i,2}$ «Рух до Пі+1» (для i = 5 – «Повернення до П1»). Кожна мережа – стохастична мережа Петрі з часовими затримками

$$N_i = (P_i, T_i, A_i, W, K, R, \tau, \boldsymbol{M}_{i,0}),$$

де A_i – дуги, W – їх кратності (усі дорівнюють 1), K – пріоритети переходів, R – ймовірності (усі 1), τ – затримки, $M_{i,0}$ – початкове маркування. Модель – множина об'єктів зі зв'язками об'єднання позицій (у дужках – позиція-джерело):

$$Model = \{O_0, \ldots, O_5\},\ \ O_0.P_{0,2} \equiv O_1.P_{1,1},\ O_i.P_{i,4} \equiv O_{i+1}.P_{i+1,1}\ (i = 1\ldots4),\ O_5.P_{5,4} \equiv O_1.P_{1,1},\ O_i.P_{i,3} \equiv O_1.P_{1,3}\ (i = 2\ldots5).$$

Об'єднання означає, що позиції – той самий об'єкт PetriP: $M_{0,2} \equiv M_{1,1}$, $M_{i,4} \equiv M_{i+1,1}$, $M_{5,4} \equiv M_{1,1}$, $M_{i,3} \equiv M_{1,3}$. Початкове маркування: $M_{0,1} = 1$, $M_{i,2} = 1$, решта 0. Стан об'єкта O_i у момент t – пара $S_i(t) = (M_i(t), E_i(t))$, де $E_i = \{E_{i,j}\}$ – масиви моментів виходу маркерів з каналів переходів $T_{i,j}$; $b_{i,j} = |E_{i,j}|$ – кількість маркерів у переході. Умови запуску переходів:

$$Z_0 = [M_{0,1} \ge 1], \quad Z_{i,1} = [M_{i,1} \ge 1] \cdot [M_{i,2} \ge 1], \quad Z_{i,2} = [M_{i,1} \ge 1],$$

де $[\cdot] = 1$, якщо умова виконана, і 0 інакше. Переходи $T_{i,1}$ і $T_{i,2}$ конфліктують за маркер $P_{i,1}$; пріоритет $K_{i,1} = 1 > K_{i,2} = 0$, тому фактично $T_{i,2}$ запускається лише при $[M_{i,1} \ge 1] \cdot [M_{i,2} = 0]$ – деталь їде далі, якщо пристрій зайнятий. Затримки: $\tau_0 =$ 0,25 хв, $\tau_{i,1} \sim \text{Exp}(1)$, $\tau_{i,2} = 1$ хв (i = 1…4), $\tau_{5,2} = 5$ хв. Рівняння переходів зведено в таблицю:

Таблиця 7 – Рівняння переходів Петрі-об'єктів конвеєра

Об'єкт: перехід	Умова запуску	Вхід маркерів	Вихід маркерів	τ, хв	K
O_0: T_0 Надходження	$M_{0,1} \ge 1$	$M_{0,1} - 1$	$M_{0,1} + 1$, $M_{1,1} + 1$	0,25	0
O_i: $T_{i,1}$ Обробка Пі	$M_{i,1} \ge 1$, $M_{i,2} \ge 1$	$M_{i,1} - 1$, $M_{i,2} - 1$	$M_{i,2} + 1$, $M_{1,3} + 1$	Exp(1)	1
O_i: $T_{i,2}$ Рух до Пі+1 (i ≤ 4)	$M_{i,1} \ge 1$ (при $M_{i,2} = 0$)	$M_{i,1} - 1$	$M_{i+1,1} + 1$	1	0
O_5: $T_{5,2}$ Повернення до П1	$M_{5,1} \ge 1$ (при $M_{5,2} = 0$)	$M_{5,1} - 1$	$M_{1,1} + 1$	5	0

У табл. 7 вихідні дуги записано з урахуванням об'єднання позицій: вихід T_0 у власну позицію $P_{0,2}$ – це поява маркера в $P_{1,1}$ об'єкта O_1. Вхід маркерів у переходи об'єкта (оператор D^-_i) виконується, поки є дозволені переходи; вибирається перехід з найвищим пріоритетом:

$$D^-_i\text{: поки } \exists j\text{: } Z_{i,j} = 1 \text{ — } \boldsymbol{M}_i := \boldsymbol{M}_i - \boldsymbol{W}^-_{i,j}, \quad E_{i,j} := E_{i,j} \cup \{t + \tau_{i,j}\}.$$

Вихід маркерів об'єкта в момент t (оператор D^+_i) – з усіх каналів його переходів, момент виходу яких дорівнює t:

$$D^+_i(t)\text{: } \forall j, \forall e \in E_{i,j}, e = t \text{ — } \boldsymbol{M}_i := \boldsymbol{M}_i + \boldsymbol{W}^+_{i,j}, \quad E_{i,j} := E_{i,j} \setminus \{e\}.$$

Найближча подія об'єкта і просування часу моделі:

$$t^{\min}_i = \min_j \min E_{i,j}, \quad t_{n+1} = \min_{i=0\ldots5} t^{\min}_i, \quad C_{n+1} = \{i\text{: } t^{\min}_i = t_{n+1}\}.$$

З множини конфліктних об'єктів C_{n+1} вибирається об'єкт i* з найбільшим пріоритетом c_i ($c_5 = 1$, інші 0), серед рівних – випадково. Подія моделі в момент

t_{n+1}:

$$S_{i*} := D^{+}{}_{i*}(S_{i*}, t_{n+1}); \quad S_i := D^{-}{}_i(S_i, t_{n+1}) \text{ для всіх } i \text{ у порядку спадання } c_i.$$

Початковий стан – $S_i := D^{-}{}_i(S_{i,0}, 0)$. Якщо $X_{i,j}$ і $Y_{i,j}$ – кількості входів маркерів у перехід $T_{i,j}$ і виходів з нього в межах події, то з урахуванням об'єднання позицій маркування моделі змінюється за рівняннями:

$$M_{1,1}(t_{n+1}) = M_{1,1}(t_n) + Y_0 + Y_{5,2} - X_{1,1} - X_{1,2}, \quad M_{i,1}(t_{n+1}) = M_{i,1}(t_n) + Y_{i-1,2} - X_{i,1} - X_{i,2} \ (i = 2\ldots5),$$

$$M_{i,2}(t_{n+1}) = M_{i,2}(t_n) + Y_{i,1} - X_{i,1}, \quad M_{1,3}(t_{n+1}) = M_{1,3}(t_n) + \Sigma_i\, Y_{i,1}, \quad M_{0,1}(t_{n+1}) = M_{0,1}(t_n) + Y_0 - X_0.$$

З них випливають інваріанти, які перевірялися після кожної з 200798 подій контрольного прогону: $M_{0,1} + b_0 = 1$, $M_{i,2} + b_{i,1} = 1$ і збереження деталей – кількість створених деталей дорівнює $M_{1,3} + \Sigma\, M_{i,1} + \Sigma\, (b_{i,1} + b_{i,2})$. Вихідні характеристики на інтервалі збору статистики $[T_0, T]$:

$$\rho_i = \overline{b}_{i,1} = (1 / (T - T_0)) \int b_{i,1}(t)\, dt, \quad \overline{N}_{\text{зайн}} = \Sigma_i\, \rho_i, \quad \lambda = \Sigma_i\, Y_{i,1}(T_0, T) / (T - T_0),$$

$$L = \Sigma_i\, (\overline{M}_{i,1} + \overline{b}_{i,1} + \overline{b}_{i,2}), \quad W = (1 / K)\, \Sigma_k\, (t^{\text{вих}}{}_k - t^{\text{вх}}{}_k) \approx L / \lambda,$$

де $t^{\text{вх}}{}_k$, $t^{\text{вих}}{}_k$ – моменти появи k-ї деталі і виходу з переходу обробки (за мітками маркерів). Бібліотека обчислює середні рекурентно: $\bar{M}(t_{n+1}) = \bar{M}(t_n) + (M(t_n) - \bar{M}(t_n)) \cdot (t_{n+1} - t_n) / t_{n+1}$.

Таблиця 8 – Конвеєр: Петрі-об'єктна модель і практикум 5

Показник	ЛР5	ЛР7	p
середній час перебування, хв	9,6205 ± 0,2386	9,8133 ± 0,3254	0,32
завантаження П1	0,8851 ± 0,0007	0,8854 ± 0,0007	0,50
завантаження П2	0,8543 ± 0,0010	0,8559 ± 0,0010	0,02
завантаження П3	0,8132 ± 0,0017	0,8153 ± 0,0018	0,09
завантаження П4	0,7582 ± 0,0025	0,7612 ± 0,0039	0,18
завантаження П5	0,6883 ± 0,0035	0,6934 ± 0,0055	0,11
середня кількість зайнятих пристроїв	3,9991 ± 0,0082	4,0113 ± 0,0118	0,09
пропускна здатність, деталей/хв	3,9998 ± 0,0007	3,9998 ± 0,0008	0,91
повернень до П1 на одну деталь	0,7488 ± 0,0257	0,7682 ± 0,0348	0,36
середня кількість деталей у системі	38,4688 ± 0,9470	39,1969 ± 1,2762	0,34

Результати моделі (табл. 8, 20 прогонів по 10000 хв з розгоном 1000 хв) збігаються з практикумом 5: p-значення менше 0,05 лише для 1 з 10 показників, а відхилення не перевищують десятих часток відсотка; мережа моделі після об'єднання позицій ідентична мережі практикуму 5. Пріоритет $c_5 = 1$ у рівняннях не випадковий. Деталі рухаються з детермінованими затримками, кратними 0,25 хв, тож деталь, що повертається від П5, і нова деталь від генератора з'являються біля П1 в той самий момент. У мережі практикуму 5 спершу виконувались усі виходи, і вільний П1 займала деталь, яка раніше увійшла в перехід, – та, що повертається. У Петрі-об'єктній моделі одночасні події різних об'єктів виконуються по одній, з входом маркерів між ними: за однакових пріоритетів П1 у половині таких випадків займає нова деталь, а деталь, що повертається, іде на ще одне коло. Середні значення від цього не змінюються (завантаження і кількість деталей у системі визначає лише процес зайнятості пристроїв), але розподіл часу перебування змінюється (рис. 10): за однакових пріоритетів стандартне відхилення 15,95 хв замість 11,86, 95-й процентиль 39,5 хв замість 32,0, максимум 284 хв замість 175. Пріоритет об'єкта «Пристрій П5» вищий за пріоритет генератора відновлює порядок практикуму 5.

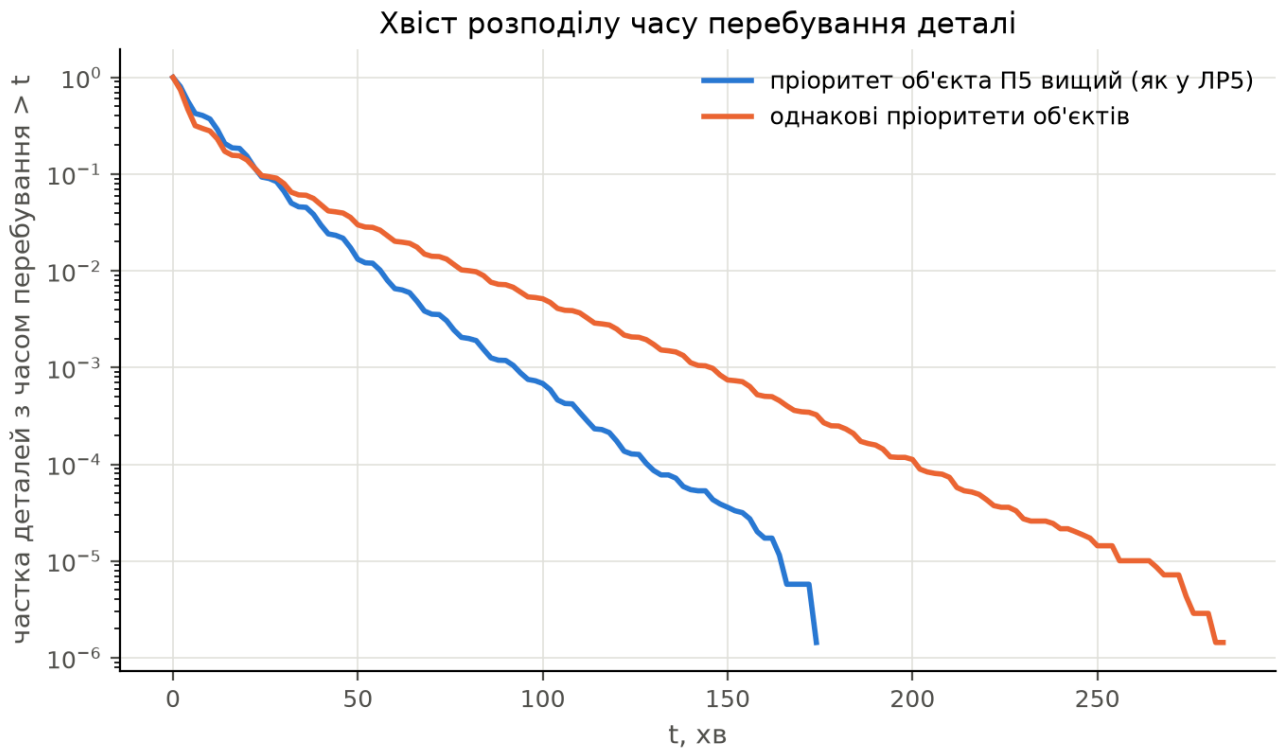


Рисунок 10 – Частка деталей з часом перебування більше t за різних пріоритетів Петрі-об’єктів (логарифмічна шкала)

Завдання 5:

Переваги технології Петрі-об'єктного моделювання, що проявилися в роботі:

- повторне використання: мережа класу описується один раз і використовується багато разів – «СМО» чотири рази, «Пристрій» п'ять, «Шлях» три, «Мікроавтобус», «Зупинка», «Верстат» – по два, «Генератор» – у всіх моделях; опис моделі масштабується з кількістю класів, а не елементів мережі;
- модель зручно змінювати: два варіанти закріплення роботів відрізняються лише об'єктами роботів (три «Закріплені роботи» або «Парк роботів»), решта об'єктів не змінювалась;
- формальна строгість: мережа, яку утворює об'єднання мереж об'єктів, – звичайна стохастична мережа Петрі (для всіх трьох задач вона збіглася з мережами практикуму 5), тож рівняння моделі – це рівняння об'єктів і зв'язків між ними, а результати можна перевіряти інваріантами;
- конфлікти локалізуються в об'єктах: розгалуження як окремий об'єкт дало точні частки маршрутів без окремого пріоритету, потрібного в одній мережі практикуму 6;
- пріоритети об'єктів природно описують правила переваги між підсистемами (А перед В, бажаний канал);
- підтримка інструментами: модель зберігається в PNML по сторінці на об'єкт, відкривається в редакторі з рамками об'єктів, статистика адресується за об'єктами.

Недоліки:

- семантика одночасних подій залежить від декомпозиції: події різних об'єктів в один момент виконуються по одній з входом маркерів між ними, тож одна й та сама мережа, розбита на об'єкти, може поводитись інакше, ніж ціла (конвеєр), і порядок треба задавати пріоритетами об'єктів;
- конфлікт між об'єктами за маркер спільної позиції розв'язується лише порядком об'єктів, а не пріоритетами і ймовірностями переходів; глобальне правило («найближчий робот») довелося зосередити в одному об'єкті, тобто декомпозицію диктують конфлікти, а не предметна область;
- зв'язки задаються індексами позицій; маркування позиції-копії втрачається, порядок об'єднань має значення, імена елементів у моделі

мають бути унікальними – для надійності довелося написати обгортку, що з'єднує позиції за іменами і перевіряє ці умови;

- маркери неіменовані: час перебування окремої вимоги не вимірюється засобами бібліотеки, потрібен зовнішній облік міток за номерами каналів переходів;
- накладні витрати: після кожної події всі об'єкти виконують вхід маркерів, список об'єктів перемішується і сортується, статистика оновлюється для всіх об'єктів; на мережі з семи об'єктів це ≈ 0,3 с на прогін з 530 тис. подій, але витрати зростають з кількістю об'єктів;
- особливості бібліотеки: немає періоду розгону, генератори випадкових чисел без зерна, кожен перехід мусить мати вхідну позицію своєї мережі.

Висновок

У роботі розглянуто алгоритм Петрі-об'єктного моделювання бібліотеки PetriObjModelPaint і за ним побудовано чотири Петрі-об'єктні моделі. Модель складається з Петрі-об'єктів (`PetriSim`), з'єднаних об'єднанням позицій; метод `PetriObjModel.go()` просуває час до найближчої події серед усіх об'єктів, виконує вихід маркерів у вибраному об'єкті, після чого всі об'єкти в порядку пріоритету виконують вхід маркерів. Статистику з розгоном, трасування, перевірки і мітки маркерів зібрано через інтерфейс збирача статистики бібліотеки, не змінюючи її алгоритму.

Тестування на мережі масового обслуговування (вузли M/M/1, M/M/2 з двох об'єктів зі спільною чергою, розгалуження і зворотний зв'язок) показало збіг з формулами мережі Джексона в межах довірчих інтервалів, правильні частки маршрутів і розв'язання конфліктів між об'єктами за пріоритетами; середні значення бібліотеки є точними середніми за часом.

Для гнучкої виробничої системи розроблено Петрі-об'єкти «Генератор», «Верстат», «Шлях», «Закріплений робот» і «Парк роботів»; мережа моделі ідентична мережі практикуму 5, а результати статистично не відрізняються. Пропускна здатність – 75,8 ± 0,6 деталей/год при закріпленні робота за шляхом і 72,1 ± 0,3 при спільному парку, тому кращий варіант 1; система не встигає за потоком 90 деталей/год, обмеження – верстат B1, місця якого займають оброблені деталі в очікуванні місця на B2.

Для маршруту мікроавтобусів розроблено Петрі-об'єкти «Генератор», «Зупинка» і «Мікроавтобус»; правило переваги A над B реалізовано пріоритетом Петрі-об'єкта. Мінімальний середній час очікування – при n = 25: 26,69 ± 0,17 хв, виручка за день -7240 ± 281 грн; результати збігаються з практикумом 5.

Для Петрі-об'єктної моделі конвеєра записано рівняння: стани об'єктів, умови запуску, оператори входу і виходу маркерів, просування часу, вибір з конфліктних об'єктів, рівняння маркування з урахуванням об'єднання позицій, інваріанти і вихідні характеристики. Модель відтворює результати практикуму 5, але показала головну відмінність технології: порядок одночасних подій різних об'єктів впливає на розподіл часу перебування і задається пріоритетами об'єктів.

Технологія дає повторне використання, наочну структуру і формальну строгість моделей; її обмеження – семантика одночасних подій, конфлікти між об'єктами лише за порядком об'єктів, зв'язки за індексами і неіменовані маркери.